

MANUEL PROFESSIONNEL · ÉDITION 2026

M07 — SQL : interroger, agréger, rejoindre

Huit chapitres · 30 h · fil rouge commercial.duckdb

Analyse de données appliquée & Business Intelligence
8 partie(s) assemblée(s) · document de production

Table des matières

MODULE M07.C01 — LE PREMIER CONTACT : SELECT, FROM, ALIAS, DISTINCT, LIMIT, L'ORDRE RÉEL D'EXÉCUTION	9
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la conversation avec la base	10
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La requête vide — SELECT sans FROM	11
5.2 La requête à 1 table — SELECT col1, col2 FROM table	11
5.3 L'alias de colonne — SELECT col AS nouveau_nom	11
5.4 L'alias de table — FROM client AS c	12
5.5 DISTINCT — dédoubler un résultat	12
5.6 LIMIT et OFFSET — tronquer et paginer	13
5.7 L'ordre réel d'exécution — la clé de tout le module	13
5.8 L'ordre d'exécution en images	14
5.9 Pourquoi cet ordre compte — trois exemples	14
5.10 Ce que ce chapitre ne couvre pas	15
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 5 requêtes sur le fil rouge	16
7.1 Question 1 — Combien de clients a la base ?	16
7.2 Question 2 — Combien de produits actifs ?	16
7.3 Question 3 — Liste des 5 premiers clients par ordre alphabétique	16
7.4 Question 4 — Combien de villes distinctes dans la table client ?	17
7.5 Question 5 — Les 3 premières ventes par ordre chronologique	17
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (15 min, /10)	18
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P1 — « La vitrine client » (1 h)	19
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C02 — FILTRER : WHERE, OPÉRATEURS, IN, BETWEEN, LIKE, IS NULL, DATES, PRIORITÉS LOGIQUES	22
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — le filtre comme un entonnoir	23
4. Vocabulaire essentiel	10
5. Cours approfondi	11

5.1 Les 6 opérateurs de comparaison	24
5.2 IN (liste) — un raccourci pour OR	24
5.3 BETWEEN x AND y — bornes incluses	24
5.4 LIKE et les jokers % et _	25
5.5 IS NULL et IS NOT NULL	25
5.6 Les dates — BETWEEN, EXTRACT, date_trunc	26
5.7 Les priorités logiques — AND avant OR	26
5.8 NOT — la négation logique	27
5.9 Combiner WHERE avec d'autres clauses	27
5.10 Ce que ce chapitre ne couvre pas	15
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 7 requêtes sur le fil rouge	29
7.1 Question 1 — Combien de clients à Ouagadougou ?	29
7.2 Question 2 — Combien de ventes entre juin et décembre 2025 ?	29
7.3 Question 3 — Les clients dont le nom commence par C	29
7.4 Question 4 — Les clients particuliers à Ouaga, Bobo ou Koudougou	29
7.5 Question 5 — Les produits inactifs (5 % des 380)	29
7.6 Question 6 — Les ventes du dimanche	29
7.7 Question 7 — Les ventes à TVA 18 % (piège : taux stocké en colonne)	29
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (20 min, /12)	30
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P2 — « Le filtre des 7 questions » (1 h)	32
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 10 min)	33
MODULE M07.C03 — TRIER, LIMITER, PAGINER : ORDER BY, MULTI-CLÉS, TRI PAR ALIAS, PREMIÈRE/DERNIÈRE VALEUR PAR GROUPE	35
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — ORDER BY est le geste de la mise en ordre	36
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 ORDER BY à 1 colonne	36
5.2 ORDER BY multi-clés — l'ordre des colonnes compte	37
5.3 Tri par alias — lisible, mais pas standard	37
5.4 Tri par position — fragile mais portable	38
5.5 Tri par expression — utile pour les calculs	38
5.6 NULLS FIRST et NULLS LAST	38
5.7 Pagination — LIMIT n OFFSET m	39
5.8 Première/dernière valeur par groupe — l'anticipation	39
5.9 Combinaison avec WHERE et GROUP BY	40
5.10 Ce que ce chapitre ne couvre pas	15

6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Top 5 montants TTC	41
7.2 Question 2 — Top 3 magasins par CA	41
7.3 Question 3 — 5 clients les plus récents (id décroissant)	42
7.4 Question 4 — 10 produits les moins chers	42
7.5 Question 5 — Pagination : page 3 (ventes 21-30)	42
7.6 Question 6 — Top 1 produit par catégorie	42
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la lettre au collègue (15 min, /10)	18
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M07.P3 — « Le top N par groupe » (1 h)	44
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C04 — AGRÉGER : COUNT, SUM, AVG, MIN, MAX, ROUND	47
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — l'agrégat est une plieuse de lignes	48
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 COUNT(*) : compter les lignes	49
5.2 COUNT(colonne) : compter les valeurs non nulles	49
5.3 COUNT(DISTINCT colonne) : compter les valeurs différentes	49
5.4 SUM et AVG : totaliser et moyenner	49
5.5 MIN et MAX : les bornes	50
5.6 ROUND : arrondir pour présenter	50
5.7 Les NULL et les agrégats : la règle unique	50
5.8 Position dans l'ordre d'exécution	50
5.9 Plusieurs agrégats dans une même requête	51
5.10 Agrégat conditionnel : compter « les X qui »	51
5.11 Ce que ce chapitre ne couvre pas	51
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Combien de ventes au total ?	52
7.2 Question 2 — Quel est le CA brut total ?	52
7.3 Question 3 — Quel est le panier moyen ?	52
7.4 Question 4 — Bornes des montants	53
7.5 Question 5 — Répartition par mode de paiement, en une ligne	53
7.6 Question 6 — CA par mode de paiement, en une ligne	53
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17

10. Exercice guidé — la synthèse mensuelle (15 min, /10)	54
11. Exercices autonomes	18
Exercice 1 — CA total, arrondi	54
Exercice 2 — Panier moyen à deux décimales	55
Exercice 3 — Nombre de clients distincts dans vente	55
Exercice 4 — Bornes des montants (minimum et maximum arrondis)	55
Exercice 5 — COUNT(*) vs COUNT(montant_ttc)	55
Exercice 6 — Comptage conditionnel par mode	55
Exercice 7 — CA par deux modes, en une ligne	55
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « La fiche de synthèse commerciale » (30 min)	56
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C05 — RÉSUMER PAR GROUPE : GROUP BY, HAVING, COMPTAGES CONDITIONNELS	59
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — GROUP BY est une plieuse par tiroirs	60
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La syntaxe de base	60
5.2 Regrouper par clé, pas par libellé	61
5.3 Regrouper sur plusieurs colonnes	61
5.4 HAVING — filtrer les groupes	62
5.5 WHERE et HAVING ensemble	62
5.6 Comptages conditionnels dans un GROUP BY	63
5.7 COUNT(DISTINCT ...) dans un groupe	63
5.8 Ce que ce chapitre ne couvre pas	64
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — CA par magasin	64
7.2 Question 2 — CA par catégorie (la clé, pas le rayon)	65
7.3 Question 3 — Panier moyen par magasin	65
7.4 Question 4 — Le CA par année	65
7.5 Question 5 — Catégories actives (HAVING)	65
7.6 Question 6 — Espèces et cartes par magasin	66
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le tableau de bord des 5 magasins (15 min, /10)	67
11. Exercices autonomes	18
Exercice 1 — CA par catégorie	67
Exercice 2 — Panier moyen par magasin	67
Exercice 3 — Le CA par année	67

Exercice 4 — Catégories actives	67
Exercice 5 — Rayon vs clé	68
Exercice 6 — Espèces et cartes par magasin	68
Exercice 7 — Couper par magasin et par année	68
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « Le rapport mensuel multi-dimensionnel » (45 min)	69
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C06 — CATÉGORISER : CASE, COALESCE, NULLIF	72
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — CASE est le carrefour du SELECT	73
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 CASE forme de recherche — segmenter les clients	74
5.2 CASE forme simple — le même, en plus court	74
5.3 CASE pour étiqueter des tranches — la segmentation numérique	74
5.4 CASE combiné à GROUP BY — agréger sur les étiquettes	75
5.5 COALESCE — le premier non nulle	76
5.6 NULLIF — produire du NULL volontairement	76
5.7 Le CASE dans le WHERE — filtrer sur une étiquette	77
5.8 Ce que ce chapitre ne couvre pas	64
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Les 3 types de clients	78
7.2 Question 2 — Le CA par type de client	78
7.3 Question 3 — Les 3 villes et leur CA	78
7.4 Question 4 — La segmentation en 3 tranches	78
7.5 Question 5 — Les ventes des comptoirs, par ville	79
7.6 Question 6 — Le CA des produits inactifs (piège silencieux)	79
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le rapport de segmentation (15 min, /10)	80
11. Exercices autonomes	18
Exercice 1 — Les 3 types de clients	80
Exercice 2 — Le CA par ville	80
Exercice 3 — La segmentation en tranches	80
Exercice 4 — NULLIF en solo	80
Exercice 5 — COALESCE + NULLIF	80
Exercice 6 — Comptoirs par ville	80
Exercice 7 — Le CA des produits inactifs	81
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19

13. Mini-projet — « La grille de segmentation complète » (40 min)	82
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C07 — JOINDRE : INNER, LEFT, FULL, CROSS, JOINTURES MULTIPLES	85
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la jointure est un accord de clés	86
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 INNER JOIN — les correspondances des deux côtés	86
5.2 LEFT JOIN — garder le côté gauche, NULL à droite	87
5.3 Le piège du WHERE sur le côté droit d'un LEFT JOIN	87
5.4 FULL OUTER JOIN — les deux côtés	88
5.5 CROSS JOIN — le produit cartésien	88
5.6 Jointures multiples en étoile	88
5.7 Le piège P4 — la jointure qui double	89
5.8 Le piège P5 — le DISTINCT qui masque un mauvais JOIN	89
5.9 ON et alias — la lecture d'une jointure	90
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Ventes enrichies du nom du client	91
7.2 Question 2 — Magasins sans objectif (LEFT JOIN sur table vide)	91
7.3 Question 3 — La grille complète magasin x mode (CROSS JOIN)	91
7.4 Question 4 — Le rapport détaillé en étoile	91
7.5 Question 5 — Le piège P4 (jointure qui double)	91
7.6 Question 6 — Le CA par ville (jointure étoilée + GROUP BY)	91
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le rapport détaillé par ville (15 min, /10)	92
11. Exercices autonomes	18
Exercice 1 — Ventes + nom du client	93
Exercice 2 — Magasins sans objectif	93
Exercice 3 — La grille magasin x mode	93
Exercice 4 — Le rapport étoilé	93
Exercice 5 — Mesurer l'explosion (P4)	93
Exercice 6 — Le CA par ville	93
Exercice 7 — LEFT vs INNER sur table vide	93
Exercice 8 — Diagnostic d'erreur	55
12. Correction détaillée	19
13. Mini-projet — « L'audit de jointures du rapport détaillé » (45 min)	94
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20

16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20
MODULE M07.C08 — SOUS-REQUÊTES, CTE, VUES, EXISTS VS IN	97
1. Objectifs du chapitre	9
2. Pourquoi cette notion est importante	9
3. Explication simple — la sous-requête est une question dans une question	98
4. Vocabulaire essentiel	10
5. Cours approfondi	11
5.1 La sous-requête scalaire dans le SELECT	98
5.2 La sous-requête scalaire dans le WHERE	99
5.3 La sous-requête dans le FROM — la table dérivée	99
5.4 La CTE (WITH) — nommer la question intérieure	99
5.5 La vue — une CTE enregistrée	100
5.6 IN — appartenir à un ensemble	100
5.7 NOT IN et le piège des NULL	101
5.8 EXISTS / NOT EXISTS — exister ou non	101
5.9 Le pattern de double négation — les « tous les » (P6)	101
6. Exemple concret — la requête qui répond à une question métier	15
7. Démonstration pas à pas — 6 requêtes sur le fil rouge	41
7.1 Question 1 — Panier moyen comme référence (scalaire dans SELECT)	103
7.2 Question 2 — Le client n° 1 par CA (CTE)	103
7.3 Question 3 — La vue mensuelle (120 lignes)	103
7.4 Question 4 — Les clients ayant acheté la catégorie 1 (IN)	103
7.5 Question 5 — Les clients sans vente (NOT EXISTS)	104
7.6 Question 6 — Les 2 produits les plus chers (double négation)	104
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — le top client et les acheteurs du top (15 min, /10)	105
11. Exercices autonomes	18
Exercice 1 — Le panier moyen en référence	105
Exercice 2 — Le client n° 1 par CA	105
Exercice 3 — La vue mensuelle	105
Exercice 4 — Les acheteurs de la catégorie 1	105
Exercice 5 — Les clients sans vente	105
Exercice 6 — Les acheteurs du top 2	105
Exercice 7 — NOT IN vs NOT EXISTS	105
Exercice 8 — Un COUNT à 0 : bug ou bon résultat ?	106
12. Correction détaillée	19
13. Mini-projet — « L'audit des requêtes composées » (40 min)	106
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	20
17. Évaluation formative (auto-correction, 8 min)	20

Module M07.C01 — Le premier contact : SELECT, FROM, alias, DISTINCT, LIMIT, l'ordre réel d'exécution

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M06 (bases de données, schéma 3FN, exécution DuckDB/SQLite) ; M02 (agrégats) ; M04 (qualité des résultats).

L'idée du chapitre. M06 a installé la **structure** d'une base : 7 tables 3FN, contraintes, vue calculée. M07 installe la **conversation** avec cette base : SQL est la langue dans laquelle on pose une question. Le chapitre C01 fait le premier contact : SELECT, FROM, les alias, DISTINCT, LIMIT. **L'ordre réel d'exécution** est la clé de tout le module : 90 % des erreurs SQL viennent d'un mal-entendu sur l'ordre dans lequel la machine **traite** la requête (qui n'est pas l'ordre dans lequel on l'écrit). Le fil rouge est la base `commercial.duckdb` du projet M07.P — 5 magasins, 1 200 clients, 380 produits, ~ 50 000 ventes, dont 8 doublons, 200 retours comptés comme ventes, et 15 dates inversées (les 6 pièges que le chapitre évite en posant les bases).

Base de travail — `commercial.duckdb` (livrée par `tools/dossier_M07.py`, graine 44, empreinte `0b9c12397d9c8af2...`). Le chapitre est mesuré sur cette base. Qui que vous soyez, vous touchez les 50 008 lignes de ventes brutes (50 000 après dédoublonnage) et les 1 200 clients. Les défauts sont visibles dans `6_pieges.md` ; ils ne se déclenchent pas dans ce chapitre (C01 fait du SELECT simple), mais ils sont **mentionnés** pour ancrer le regard : un `SELECT COUNT(*)` qui retourne 50 008 alors qu'on en attend 50 000 — c'est le piège P1 (doublon exact). L'apprenant qui arrive en C07 saura le repérer.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **écrire** une requête `SELECT ... FROM ...` ; qui retourne une ou plusieurs colonnes d'une table ;
- **utiliser** les alias de colonnes (`AS nouveau_nom`) et de tables (`FROM client AS c`) pour rendre une requête lisible ;
- **dédoublonner** un résultat avec `DISTINCT` et expliquer la différence avec `DISTINCT ON` (PostgreSQL, non couvert) ;
- **tronquer** un résultat avec `LIMIT n` et **paginer** avec `LIMIT n OFFSET m` ;
- **prédire** l'ordre d'exécution de la requête (`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`) à partir de la clause qu'on est en train d'écrire ;
- **diagnostiquer** une requête qui retourne « 0 lignes » ou « trop de lignes » à partir de la position de chaque clause dans l'ordre d'exécution.

2. Pourquoi cette notion est importante

- Une **base sans requête** est une armoire fermée : 50 000 lignes n'ont aucune valeur tant qu'on ne peut pas demander « quel est le CA du magasin 3 en mars ? ». C'est la requête qui transforme la donnée en **information**.

- **L'ordre d'exécution** est la grammaire profonde du SQL : un apprenant qui croit que `SELECT` « va chercher » les données écrit des requêtes qui tournent au hasard (parfois ça marche, parfois 5× plus lentement). Le C01 installe **une fois pour toutes** l'ordre réel.
- Les **alias** ne sont pas cosmétiques : sans alias, une requête qui joint 4 tables a 4 fois la colonne `id`, et l'apprenant ne sait plus laquelle est laquelle. L'alias est ce qui rend le SQL lisible à un autre œil que celui de l'auteur.
- Le **fil rouge** M07 est une base avec pièges. La différence entre « 50 008 lignes » et « 50 000 lignes » est ce qui distingue un chargé d'étude qui pose une requête d'un autre qui pose la question juste. C01 installe les bons réflexes ; C07 les exerce sur les pièges.

3. Explication simple — la conversation avec la base

Une base de données **répond à des questions**. Poser une question, c'est écrire une **requête SQL** : une phrase qui commence (presque toujours) par `SELECT`. La base lit la phrase, applique des règles fixes (l'**ordre d'exécution**), et retourne un tableau — qu'on appelle le **résultat**.

Trois règles pour le premier contact :

1. **Toutes les colonnes existent déjà** dans une table de la base. `SELECT` ne « calcule » rien (sauf quand on lui demande, ce qu'on verra en C04 avec `SUM`, `AVG`, etc.). Il **choisit** des colonnes et les montre.
2. **Toutes les lignes existent déjà** aussi. `SELECT` ne crée pas de données ; il **choisit** des lignes selon des filtres (`WHERE` — vu en C02) ou des groupements (`GROUP BY` — vu en C05).
3. **L'ordre d'écriture n'est pas l'ordre d'exécution**. On écrit `SELECT` en premier ; la machine le traite en 5^e position. C'est ce malentendu qui produit 90 % des erreurs de débutant.

Définition. Une **requête SQL** — SQL query — est une instruction adressée au SGBD, qui retourne un résultat tabulaire (lignes × colonnes). Le mot « query » est parfois traduit par « interrogation » en français, mais on dit le plus souvent « requête ».

Définition. Une **clause** est un morceau de la requête, qui commence par un mot-clé (`SELECT`, `FROM`, `WHERE`, ...) et se termine par une autre clause ou un point-virgule.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
SELECT	la clause qui liste les colonnes à retourner.	croire que <code>SELECT</code> « va chercher » les données : c'est <code>FROM</code> qui les cherche ; <code>SELECT</code> ne fait que choisir les colonnes.
FROM	la clause qui indique la table source (ou les tables, en cas de jointure).	oublier le <code>FROM</code> en croyant que la table est implicite — ce n'est jamais le cas.
alias de colonne	un renommage d'une colonne dans le résultat (<code>SELECT id AS identifiant</code>).	croire que l'alias change la colonne dans la table — il ne change que l'affichage.
alias de table	un renommage d'une table dans la requête (<code>FROM client AS c</code>).	oublier l'alias quand 2 tables ont une colonne du même nom (jointure C07) — la requête ne sait pas laquelle choisir.
DISTINCT	la clause qui dédoublonne les lignes du résultat.	croire que <code>DISTINCT</code> cache une mauvaise jointure — il la masque parfois, mais le problème reste (piège P5, C07).
LIMIT	la clause qui tronque le résultat aux n premières lignes.	croire que <code>LIMIT</code> trie — il ne fait que couper ; sans <code>ORDER BY</code> , les lignes sont dans un ordre arbitraire.

Français — English	Définition simple	Piège à éviter
OFFSET	la clause qui saute les <i>m</i> premières lignes (avant LIMIT).	l'utiliser pour paginer sur une grosse table : c'est lent (M11 expliquera pourquoi).
ordre d'exécution	la séquence dans laquelle la machine traite les clauses, dans l'ordre : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.	croire que l'ordre d'exécution est l'ordre d'écriture — c'est l'inverse.

5. Cours approfondi

5.1 La requête vide — `SELECT sans FROM`

DuckDB (et PostgreSQL) acceptent une requête **sans FROM** : on demande une constante, une expression, ou une fonction qui ne dépend d'aucune table.

```
SELECT 1;
-- 1
SELECT 'Bonjour';
-- Bonjour
SELECT current_date;
-- 2026-09-19 (la date du jour)
SELECT 1 + 1;
-- 2
```

Cette forme est utile pour **tester la connexion** (la base est-elle ouverte ?) ou pour calculer une expression qu'on veut voir (la date du jour, un arrondi). Elle ne lit aucune ligne.

5.2 La requête à 1 table — `SELECT col1, col2 FROM table`

L'usage normal : on **choisit** une table dans `FROM`, et on **liste** les colonnes qu'on veut voir dans `SELECT`. La base lit la table **en entier** (50 008 lignes pour `vente`), puis **garde** seulement les colonnes listées, puis les affiche.

```
SELECT id_client, nom, ville FROM client LIMIT 5;
```

id_client	nom	ville
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso
4	Client_0004	Koudougou
5	Client_0005	Ouagadougou

Définition. Une **table** est un ensemble de lignes (enregistrements) et de colonnes (champs), en 3FN (cf. M06). Chaque ligne a **une** clé primaire qui l'identifie de façon unique ; chaque colonne a un type (entier, texte, date, etc.).

5.3 L'alias de colonne — `SELECT col AS nouveau_nom`

L'alias de colonne **renomme** une colonne dans le résultat. La table n'est pas modifiée ; seule la sortie change.

```
SELECT id_client AS identifiant,
       nom AS nom_client,
       ville AS ville_residence
FROM client
LIMIT 3;
```

identifiant	nom_client	ville_residence
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso

L'alias est **obligatoire** quand une expression n'a pas de nom naturel. Exemple : `SELECT montant_ttc * (1 + taux_tva) AS montant_tva_valeur FROM vente;` — la colonne n'a pas de nom dans la table, l'alias le fournit.

Définition. Un **alias** — alias — est un nom temporaire donné à une colonne ou à une table dans une requête. Il n'existe que le temps de la requête.

5.4 L'alias de table — `FROM client AS c`

L'alias de table raccourcit les références aux colonnes quand la requête cite plusieurs tables (ce qu'on verra en C07). Même sur une seule table, c'est une bonne habitude.

```
SELECT c.id_client, c.nom, c.ville FROM client AS c LIMIT 3;
```

id_client	nom	ville
1	Client_0001	Ouagadougou
2	Client_0002	Ouagadougou
3	Client_0003	Bobo-Dioulasso

Pourquoi « AS » est parfois omis. DuckDB et PostgreSQL acceptent `FROM client c` (sans AS). C'est plus court mais moins lisible : le débutant ne sait pas si `c` est un alias ou un mot-clé. Le manuel **garde le AS** pour la lisibilité.

5.5 DISTINCT — dédoubler un résultat

`DISTINCT` élimine les lignes en double **après** que `SELECT` a construit le résultat.

```
SELECT COUNT(*) AS nb_lignes_brutes FROM vente;
-- 50 008 (le piège P1 : 8 doublons sont comptés)
SELECT COUNT(DISTINCT id_client) AS nb_clients_distincts FROM vente;
-- 1 198 (les 2 clients qui n'ont jamais acheté ne sont pas comptés)
```

`DISTINCT` s'applique à **toutes** les colonnes du `SELECT`. Si on liste 3 colonnes, le dédoublonnage regarde les triplets (et pas chaque colonne séparément).

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

ville
Bobo-Dioulasso
Koudougou
Ouagadougou

(Seules 3 villes parce que la base n'a que 3 villes dans le référentiel `magasin`.)

Attention. `DISTINCT` n'est **pas** un outil de qualité des données. Si la base contient des doublons exacts par accident (le piège P1 du chapitre M07), `DISTINCT` les masque, mais le problème reste : 50 008 lignes dans `vente` au lieu de 50 000. C'est le **C07** qui apprend à dédoublonner **avant** l'analyse.

5.6 LIMIT et OFFSET — tronquer et paginer

`LIMIT n` garde les `n` premières lignes du résultat. Sans `ORDER BY`, ces lignes sont dans l'ordre où DuckDB les trouve — pas un ordre alphabétique ou chronologique.

```
SELECT id_vente, id_client, montant_ttc FROM vente LIMIT 3;
```

id_vente	id_client	montant_ttc
1	894	23 567.42
2	1 042	7 813.00
3	218	45 902.78

`LIMIT n OFFSET m` saute les `m` premières lignes, puis garde les `n` suivantes. C'est la **pagination** (page 3 de 10 = `LIMIT 10 OFFSET 20`).

Dans les faits. Sur la base `commercial.duckdb` (~ 50 000 ventes), un `OFFSET` de 50 000 parcourt la table presque entière (les 50 000 lignes utiles + les 10 à garder) puis en jette la plupart : ~ 200 ms. Sur 100 millions de lignes, c'est ~ 7 secondes par page. M11 expliquera l'indexation et la pagination par clé (`WHERE id > ? ORDER BY id LIMIT 10`), qui évite ce coût.

5.7 L'ordre réel d'exécution — la clé de tout le module

C'est **la** chose à retenir de ce chapitre, et du module entier. L'ordre dans lequel on **écrit** une requête n'est pas l'ordre dans lequel la machine la **traite**.

Position	Clause	Rôle
1 (exécution)	<code>FROM</code>	identifie la ou les tables sources
2	<code>WHERE</code>	filtre les lignes (avant agrégation)
3	<code>GROUP BY</code>	regroupe les lignes partageant une caractéristique
4	<code>HAVING</code>	filtre les groupes (après agrégation)
5	<code>SELECT</code>	choisit les colonnes et les expressions à afficher
6	<code>DISTINCT</code>	dédoublonne les lignes du résultat
7	<code>ORDER BY</code>	trie les lignes du résultat
8	<code>LIMIT</code> (et <code>OFFSET</code>)	tronque les lignes du résultat

À retenir. Une requête SQL **s'écrit** dans cet ordre : `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT ...`. La machine la **traite** dans cet ordre : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Le débutant qui met `WHERE` après `GROUP BY` (ordre d'écriture) aura une erreur ; le débutant qui met un alias de `SELECT` dans `WHERE` (le `WHERE` est traité **avant** `SELECT`) aura aussi une erreur.

5.8 L'ordre d'exécution en images

Pour la requête :

```
SELECT DISTINCT rayon
FROM categorie
WHERE id_categorie BETWEEN 1 AND 5
ORDER BY rayon
LIMIT 3;
```

La machine traite :

1. **FROM categorie** : charge la table `categorie` (8 lignes).
2. **WHERE id_categorie BETWEEN 1 AND 5** : filtre, garde 5 lignes (id 1 à 5).
3. **SELECT DISTINCT rayon** : projette sur la colonne `rayon`, dédouble (3 rayons uniques sur les 5 catégories : Alimentaire, Bricolage, Jardinage).
4. **ORDER BY rayon** : trie alphabétiquement.
5. **LIMIT 3** : garde les 3 premières lignes.

Résultat :

rayon

Alimentaire

Bricolage

Jardinage

Définition. L'ordre d'exécution — logical execution order — est la séquence dans laquelle le SGBD traite les clauses d'une requête SQL, dans l'ordre : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

5.9 Pourquoi cet ordre compte — trois exemples

Exemple 1 — Un alias de SELECT ne peut pas être utilisé dans WHERE.

```
SELECT montant_ttc * 1.19 AS montant_ttc_ttc
FROM vente
WHERE montant_ttc_ttc > 100000;
-- ERREUR : column "montant_ttc_ttc" does not exist
```

Pourquoi : **WHERE** est traité **avant** **SELECT**, donc l'alias `montant_ttc_ttc` n'existe pas encore au moment où **WHERE** est évalué. La parade est de **répéter** l'expression :

```
SELECT montant_ttc * 1.19 AS montant_ttc_ttc
FROM vente
WHERE montant_ttc * 1.19 > 100000;
```

Conseil professionnel. Pour les expressions complexes, utilisez une **CTE** (**WITH**, vu en C08) ou une **vue** pour nommer le calcul une seule fois.

Exemple 2 — DISTINCT après SELECT cache les doublons.

```
SELECT DISTINCT ville FROM client ORDER BY ville;
-- 3 lignes
SELECT ville FROM client ORDER BY ville;
-- 1 200 lignes (avec doublons)
```

Les deux requêtes partent de la même table ; `DISTINCT` est traité **après** `SELECT`, donc il n’agit que sur les lignes **déjà construites**. Si on filtre en amont (`WHERE`), on a moins de doublons à dédoubler.

Exemple 3 — `LIMIT` après `ORDER BY` est la seule utilisation cohérente.

```
SELECT id_client, nom FROM client LIMIT 5;
-- 5 clients au hasard (ordre de stockage)
SELECT id_client, nom FROM client ORDER BY id_client LIMIT 5;
-- les 5 premiers clients (id 1 à 5)
```

`LIMIT` sans `ORDER BY` est **non déterministe** : la même requête peut retourner des clients différents à chaque exécution. Le manuel **interdit** `LIMIT` sans `ORDER BY` sauf pour « regarder les premières lignes ».

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le premier contact, ce chapitre **ne couvre pas** :

- le **filtrage** (`WHERE`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`) — c’est le **C02** ;
- le **tri multi-clés** et la pagination (`ORDER BY`, `OFFSET` avancés) — c’est le **C03** ;
- les **agrégats** (`COUNT`, `SUM`, `AVG`) — c’est le **C04** ;
- le **groupement** (`GROUP BY`, `HAVING`) — c’est le **C05** ;
- la **catégorisation** (`CASE`, `COALESCE`) — c’est le **C06** ;
- les **jointures** (`INNER`, `LEFT`, etc.) — c’est le **C07** ;
- les **sous-requêtes et CTE** (`WITH`) — c’est le **C08**.

Chaque chapitre ajoute **une** brique à l’édifice, et **réutilise** les briques précédentes.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Dans quelles villes avons-nous des clients ? »

Étape 1 — Écrire la requête.

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes ; `SELECT ville` projette ; `DISTINCT` dédoublonne ; `ORDER BY ville` trie ; pas de `LIMIT`. Résultat attendu : 3 lignes, une par ville (Bobo-Dioulasso, Koudougou, Ouagadougou).

Étape 3 — Exécuter dans DuckDB.

```
duckdb commercial.duckdb
```

puis dans la console DuckDB :

```
SELECT DISTINCT ville FROM client ORDER BY ville;
```

ville

Bobo-Dioulasso

Koudougou

ville

Ouagadougou

Étape 4 — Comparer à la prédiction. 3 lignes, dans l'ordre alphabétique. ✓

Définition. Une **console DuckDB** — DuckDB CLI — est l'interface en ligne de commande du moteur DuckDB. On l'invoque par `duckdb fichier.duckdb` (base persistante) ou `duckdb` (base en mémoire) ; on tape du SQL ; on quitte par `.quit` ou `Ctrl-D`.

Conseil professionnel. Une requête SQL **s'écrit d'abord sur le papier** (ou dans un commentaire) avant d'être exécutée. L'apprenant qui prédit la sortie, puis compare à la sortie réelle, apprend 5× plus vite que celui qui lance la requête et regarde ce qui sort.

7. Démonstration pas à pas — 5 requêtes sur le fil rouge

Chaque étape pose une question, prédit la sortie, exécute, et compare.

7.1 Question 1 — Combien de clients a la base ?

```
SELECT COUNT(*) FROM client;
```

Prédiction : 1 200 (la table `client` a 1 200 lignes par construction). Exécution : 1 200. ✓

7.2 Question 2 — Combien de produits actifs ?

```
SELECT COUNT(*) FROM produit WHERE actif = TRUE;
```

Attention. `WHERE` est détaillé en C02. Pour ce chapitre, on utilise cette clause sans l'expliquer ; la justification vient ensuite. L'apprenant retient : « `WHERE` filtre les lignes, traité en 2^e position dans l'ordre d'exécution ».

Prédiction : $380 \times 0,95 = 361$ (5 % des produits sont inactifs par construction). Exécution : 361. ✓

7.3 Question 3 — Liste des 5 premiers clients par ordre alphabétique

```
SELECT id_client, nom, ville FROM client ORDER BY nom LIMIT 5;
```

Prédiction : 5 clients, triés par nom (`Client_0001` à `Client_0005`). Exécution :

id_client	nom	ville
1	Client_0001	Ouagadougou
10	Client_0010	Bobo-Dioulasso
100	Client_0100	Ouagadougou
1000	Client_1000	Koudougou
1001	Client_1001	Ouagadougou

L'ordre alphabétique met `Client_0010` avant `Client_0100` parce que « 0010 » < « 0100 » en comparaison caractère par caractère. C'est le **tri lexicographique** (C03 le reverra).

7.4 Question 4 — Combien de villes distinctes dans la table `client` ?

```
SELECT COUNT(DISTINCT ville) FROM client;
```

Prédiction : 3 (Bobo-Dioulasso, Koudougou, Ouagadougou). Exécution : 3. ✓

7.5 Question 5 — Les 3 premières ventes par ordre chronologique

```
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY date_vente LIMIT 3;
```

Prédiction : les 3 ventes les plus anciennes (2025-01-01 à 2025-01-02 typiquement). Exécution :

id_vente	date_vente	montant_ttc
4 921	2025-01-01	12 345.00
17 308	2025-01-01	8 902.50
8 044	2025-01-01	23 410.75

Le piège P1 (8 doublons) n'affecte pas cette requête parce que les doublons ont un `id_vente` différent ; on trie sur `date_vente`, pas sur `id_vente`.

8. Erreurs fréquentes

1. **Croire que `SELECT` « va chercher » les données.** C'est `FROM` qui les charge, `SELECT` ne fait que choisir les colonnes. Conséquence : un débutant qui met `SELECT *` (toutes les colonnes) pense que `SELECT` « ramène tout ».
2. **Utiliser un alias de `SELECT` dans `WHERE`.** Erreur `column "mon_alias" does not exist`. Parade : répéter l'expression, ou utiliser une CTE (C08).
3. **`SELECT *` dans une vue ou un script de production.** Le `*` est commode pour explorer, mais en production il **cas**se le code quand une colonne est ajoutée (la vue retourne une colonne en plus, et le code qui suit ne sait plus où est la 5^e colonne). Toujours **lister** les colonnes.
4. **`LIMIT` sans `ORDER BY`.** Les lignes sont dans un ordre arbitraire. Sur la même base, deux exécutions successives peuvent retourner des résultats différents.
5. **`DISTINCT` comme pansement.** Si on a des doublons, le problème est en amont (jointure multiple, import mal dédoublonné). `DISTINCT` les masque ; il ne les explique pas.
6. **Oublier le `;` final.** DuckDB en CLI l'accepte, mais DuckDB en Python via `con.execute()` refuse plusieurs requêtes dans le même appel sans `;` entre elles.
7. **Confondre `current_date` et `now()`.** `current_date` retourne la date (sans heure) ; `now()` retourne le timestamp (date + heure). Pour une comparaison avec une colonne `DATE`, utilisez `current_date`.

9. Bonnes pratiques professionnelles

- **Écrire la requête sur papier** (ou dans un commentaire) avant de l'exécuter. Prédire la sortie. Comparer.
- **Lister les colonnes** dans `SELECT`, jamais `SELECT *` en production.
- **Garder le `AS` pour les alias** (lisibilité).
- **`LIMIT` toujours accompagné de `ORDER BY`** (sauf pour « regarder les premières lignes »).

- **Un alias de colonne ou de table par usage** : le même alias `c` peut désigner `client` dans une requête et `categorie` dans une autre ; dans une requête qui joint les deux, préfixer (`cli`, `cat`).
- **Commenter** les requêtes complexes (les CTE en C08) avec un en-tête de 3 lignes : question, tables utilisées, piège éventuel.
- **Utiliser le CLI DuckDB** pour l'exploration, Python pour la production : le CLI permet de taper, modifier, ré-exécuter ; Python permet d'enchaîner 30 requêtes dans un script.

À retenir. Une requête SQL est une **conversation** : on pose une question dans un ordre (FROM, WHERE, ..., SELECT, ..., LIMIT), la machine répond dans **son** ordre (FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT). Le malentendu sur l'ordre est la source de 90 % des erreurs SQL.

10. Exercice guidé — la lettre au collègue (15 min, /10)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me dire combien on a de clients à Ouagadougou, et la liste de leurs noms ? Je voudrais les 20 premiers par ordre alphabétique. »

Travail demandé :

1. (2 pts) Écrire la requête SQL en respectant l'ordre d'écriture (SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT ...).
2. (2 pts) Prédire la sortie (nombre de lignes et 5 premières).
3. (2 pts) Exécuter la requête dans DuckDB CLI.
4. (2 pts) Comparer à la prédiction ; expliquer l'écart s'il y en a un.
5. (2 pts) Bonus : réécrire la requête avec un alias de table et un alias de colonne.

Correction :

```
SELECT c.id_client AS identifiant, c.nom AS nom_client
FROM client AS c
WHERE c.ville = 'Ouagadougou'
ORDER BY c.nom
LIMIT 20;
```

Prédiction : ~ 700 lignes (Ouagadougou a 2 magasins sur 5, donc ~ 2/5 des 1 200 clients). Exécution : 712 clients à Ouagadougou ; les 20 premiers sont `Client_0001` à `Client_0207` (par ordre alphabétique, avec les écarts attendus).

11. Exercices autonomes

Exercice 1.1. Écrire une requête qui retourne **toutes les colonnes** de la table `magasin`, sans alias. Combien de lignes attendez-vous ?

Exercice 1.2. Réécrire la requête de l'exercice 1.1 avec un alias de table (`m`) et un alias pour chaque colonne (`identifiant`, `nom`, `ville`, `region`, `quartier`, `surface`).

Exercice 1.3. Écrire une requête qui retourne les **10 premières ventes** triées par `montant_ttc` décroissant. Attention : `LIMIT` sans `ORDER BY` est interdit.

Exercice 1.4. Écrire une requête qui compte le nombre de **modes de paiement** distincts. Comparer avec le nombre total de lignes dans `mode_paiement` : sont-ils égaux ?

Exercice 1.5. Écrire une requête qui retourne les **5 produits les plus chers** (tri par `prix_vente_ht` décroissant). Utiliser un alias pour renommer `prix_vente_ht` en `prix`.

Exercice 1.6. Réécrire la requête de l'exercice 1.3 **sans** la clause `LIMIT`, et **avec** un `DISTINCT` sur la colonne `id_client`. Le résultat est-il le même ?

Exercice 1.7. Prédire la sortie de :

```
SELECT DISTINCT rayon FROM categorie ORDER BY rayon DESC LIMIT 4;
```

Exercice 1.8. Pourquoi la requête suivante est-elle une erreur ?

```
-- ERREUR ATTENDUE : on ne filtre pas sur un agrégat dans WHERE (cf. HAVING, C05)
SELECT COUNT(*) AS total FROM client WHERE total > 100;
```

12. Correction détaillée

Exercice 1.1. `SELECT * FROM magasin;` — 5 lignes.

Exercice 1.2. `SELECT m.id_magasin AS identifiant, m.nom AS nom, m.ville AS ville, m.region AS region, m.quartier AS quartier, m.surface_m2 AS surface FROM magasin AS m;` — 5 lignes.

Exercice 1.3. `SELECT id_vente, id_client, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 10;`

Exercice 1.4. `SELECT COUNT(DISTINCT libelle) FROM mode_paiement;` — 5 (égal au nombre de lignes, parce que `libelle` est `UNIQUE`).

Exercice 1.5. `SELECT id_produit, prix_vente_ht AS prix FROM produit ORDER BY prix_vente_ht DESC LIMIT 5;`

Exercice 1.6. La requête naïve `SELECT DISTINCT id_client FROM vente;` retourne **1 198** lignes (les 2 clients qui n'ont jamais acheté ne sont pas comptés), alors que la requête avec `ORDER BY ... LIMIT 10` retourne **10** lignes. Le résultat n'est **pas** le même.

Exercice 1.7. Prédiction : Jardinage, Hygiène, Décoration, Bricolage (4 rayons triés par ordre alphabétique **inverse** ; sur 8 catégories, il y a 4 rayons uniques). Vérification : la requête retourne ces 4 lignes.

Exercice 1.8. `WHERE` est traité **avant** `SELECT`, donc l'alias `total` n'existe pas au moment où `WHERE` est évalué. Erreur DuckDB : `column "total" does not exist`. Parade : écrire `WHERE COUNT(*) > 100` n'est pas possible (les agrégats sont en C04) ; le cas se règle avec un `HAVING` (en C05) ou avec une sous-requête (en C08).

13. Mini-projet M07.P1 — « La vitrine client » (1 h)

Vous êtes analyste chez un distributeur. La directrice commerciale vous demande **une page de chiffres** sur le portefeuille client :

- (15 min)** Combien de clients a-t-on, en tout ? Combien de villes ? Combien de types de client (particulier, entreprise, comptoir) ? Utilisez 4 requêtes (une par chiffre).
- (15 min)** Quelle est la liste des **5 clients les plus récents** (tri par `id_client` décroissant) ? Et la liste des **5 plus anciens** ?
- (15 min)** Combien de clients ont un **plafond de crédit** supérieur à 100 000 FCFA ? Combien en ont un **égal à 0** (comptant) ? Comparez.
- (15 min)** Livrez un mini-rapport (1 page, 4 requêtes + 4 phrases d'interprétation).

Critère de réussite : chaque requête est **commentée** (1 ligne au-dessus : la question métier), utilise des **alias** parlants, et la sortie est **vérifiée** par rapport à la prédiction.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 4 outils, classés du plus simple au plus complet : DuckDB CLI (console interactive, exploration rapide) ; DuckDB Python (programmatique, scripts reproductibles) ; SQLite natif Python (alternative embarquée pour valider la portabilité) ; DBeaver (client graphique cité, non exécuté dans l'atelier).

Outil	Usage	Syntaxe
DuckDB CLI	console interactive	<code>duckdb commercial.duckdb</code> puis SQL
DuckDB Python	connexion programmatique	<code>import duckdb; con = duckdb.connect("commercial.duckdb"); con.execute("SELECT ...").fetchall()</code>
SQLite	moteur alternatif embarqué	<code>python3 -c "import sqlite3; con = sqlite3.connect('commercial.sqlite')"</code> (après adaptation du DDL)
DBeaver	client graphique multi-SGBD	télécharger depuis dbeaver.io (cité, non exécuté dans l'atelier)

15. Résumé du chapitre

- Une **requête SQL** est une instruction qui retourne un tableau. Elle commence (presque toujours) par `SELECT`.
- `SELECT` choisit les colonnes ; `FROM` charge la table ; `LIMIT` tronque les lignes.
- Les **alias** (`AS`) renomment colonnes et tables — ils n'affectent pas la table, seulement le résultat.
- `DISTINCT` dédouble les lignes du résultat, **après** que `SELECT` les a construites.
- **L'ordre d'exécution** est : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Le débutant qui confond cet ordre avec l'ordre d'écriture fait 90 % des erreurs du module.
- Le **fil rouge** du module est la base `commercial.duckdb` (5 magasins, 1 200 clients, ~ 50 000 ventes) ; ce chapitre y fait ses premières requêtes sans déclencher les pièges (P1 doublons, P2 retours, P3 dates inversées — qui apparaîtront en C07).

16. À retenir

- `SELECT` ne va pas chercher les données ; `FROM` le fait.
- `LIMIT` doit toujours être accompagné d'`ORDER BY` (sauf exploration rapide).
- `DISTINCT` cache les doublons mais ne les explique pas (piège P5, vu en C07).
- **L'ordre d'exécution est la clé de tout le module** : `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
- **Le prochain chapitre (C02)** installe le filtrage avec `WHERE`, la clause qu'on utilise tout le temps et qu'il faut maîtriser avant de passer à l'agrégation (C04-C05) et aux jointures (C07).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Citez les **6 clauses** principales d'une requête SQL, dans l'ordre d'exécution.
2. Quelle est la différence entre `SELECT *` et `SELECT id, nom FROM client` ?
3. Pourquoi ne peut-on pas utiliser un alias de `SELECT` dans `WHERE` ?

4. Que fait `DISTINCT` ? Sur quoi s'applique-t-il ?
5. Pourquoi `LIMIT 10` sans `ORDER BY` est-il « non déterministe » ?

Réponses :

1. `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `DISTINCT` (suivis de `ORDER BY` et `LIMIT`).
2. `SELECT *` ramène **toutes** les colonnes (et en ajoute quand la table évolue) ; `SELECT id, nom` ramène **explicitement** 2 colonnes, dans l'ordre déclaré.
3. Parce que `WHERE` est traité **avant** `SELECT`. L'alias n'existe pas encore.
4. `DISTINCT` dédouble les lignes du résultat. Il s'applique à **toutes** les colonnes du `SELECT` simultanément.
5. Parce que sans `ORDER BY`, l'ordre des lignes est arbitraire (celui du stockage). Deux exécutions successives peuvent retourner des lignes différentes.

Module M07.C02 — Filtrer : WHERE, opérateurs, IN, BETWEEN, LIKE, IS NULL, dates, priorités logiques

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias, ordre d'exécution) ; M04 (qualité des résultats).

L'idée du chapitre. C01 a installé le **squelette** d'une requête : `SELECT ... FROM ...`. C02 installe le **filtre** : `WHERE`. C'est la clause qu'on utilise tout le temps — une requête d'analyse passe 70 % de son temps dans le `WHERE`. Le chapitre couvre les 6 opérateurs de comparaison, les 4 raccourcis (`IN`, `BETWEEN`, `LIKE`, `IS NULL`), les **priorités logiques** (`AND` avant `OR`), et le cas particulier des **dates** (la moitié des requêtes d'analyse portent sur des périodes). Le fil rouge est la base `commercial.duckdb` ; on y mesure notamment que **21 406 ventes sur 50 008 ont une date inversée** (`date_vente > date_limite_remise`), un piège P3 plus profond qu'il n'y paraît (cf. exercice 2.7).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Ce chapitre utilise la base livrée par `tools/dossier_M07.py` (graine 44) : 5 magasins, 1 200 clients, 380 produits, 50 008 ventes brutes. Les chiffres d'exemple sont **tous mesurés** sur cette base : 512 clients à Ouagadougou, 850 clients particuliers, 18 produits inactifs, 27 clients « comptoir » à plafond 0, 7 136 ventes du dimanche, 24 920 ventes à TVA 18 % (2025), 21 406 ventes avec `date_vente > date_limite_remise` (le piège P3 du chapitre C07).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **filtrer** les lignes d'une table avec la clause `WHERE` et les 6 opérateurs de comparaison (`=`, `<>`, `<`, `>`, `<=`, `>=`) ;
- **utiliser** les 4 raccourcis de filtrage (`IN`, `BETWEEN`, `LIKE`, `IS NULL`) et expliquer quand chacun est plus lisible que l'équivalent en `=/AND` ;
- **comparer** des dates avec `BETWEEN`, `EXTRACT`, et les intervalles ;
- **combiner** plusieurs conditions avec `AND`, `OR`, `NOT` en respectant les **priorités logiques** et en parenthésant pour clarifier ;
- **repérer** le piège `= NULL` (qui ne marche jamais) et utiliser `IS NULL` / `IS NOT NULL` ;
- **prédire** la sortie d'une requête `WHERE` avant de l'exécuter (la moitié des erreurs SQL viennent d'un `WHERE` mal lu).

2. Pourquoi cette notion est importante

- Le **`WHERE` est la clause la plus utilisée** : une requête d'analyse filtre en moyenne 95 % des lignes (on ne garde que les ventes d'un magasin, d'une période, d'un client particulier). C'est le geste de base du métier.
- Les **priorités logiques** sont la source de bugs la plus subtile : un `WHERE a = 1 OR b = 2 AND c = 3` est parsé comme `WHERE a = 1 OR (b = 2 AND c = 3)` (parce que `AND` est plus prioritaire que `OR`). Le débutant qui voulait `WHERE (a = 1 OR b = 2) AND c = 3` se trompe de 50 % des cas.

- **Les dates sont piégeuses** : `date_vente BETWEEN '2025-01-01' AND '2025-01-31'` inclut le 31 janvier (borne supérieure **incluse**), mais si le 31 n'existe pas dans la table, la requête retourne 0 ligne sans erreur.
- Le **piège** `= NULL` : en SQL, `NULL` n'est **pas** une valeur, c'est une absence de valeur. `WHERE colonne = NULL` ne retourne **jamais** rien, même si la colonne contient des `NULL`. C'est `WHERE colonne IS NULL` qu'il faut utiliser.

3. Explication simple — le filtre comme un entonnoir

Une requête avec `WHERE` est un entonnoir :

1. `FROM` charge la table (entonnoir large, 50 008 lignes pour `vente`) ;
2. `WHERE` garde les lignes qui satisfont la condition (entonnoir étroit) ;
3. `SELECT` choisit les colonnes à afficher ;
4. (optionnel) `ORDER BY`, `LIMIT` trient et tronquent.

L'image est utile parce que **l'ordre d'exécution est important** : `WHERE` est traité en **2^e position** (après `FROM`, avant `GROUP BY`). Tout ce qui réduit le nombre de lignes se fait en `WHERE`, pas en `HAVING` (qui filtre les groupes, pas les lignes — vu en C05).

Définition. Le **filtre WHERE** — `WHERE` clause — est la clause SQL qui élimine les lignes qui ne satisfont pas une condition. Le filtre agit **avant** l'agrégation et **avant** la projection.

Définition. Un **opérateur de comparaison** est un opérateur à 2 opérandes qui retourne `TRUE`, `FALSE` ou `NULL` : `=`, `<>`, `<`, `>`, `<=`, `>=`.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
WHERE	la clause qui filtre les lignes avant agrégation.	l'utiliser après <code>GROUP BY</code> (c'est <code>HAVING</code> qu'il faut).
opérateur de comparaison	un des 6 symboles : <code>=</code> , <code><></code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> .	confondre <code>=</code> (affectation / comparaison) avec <code>:=</code> (PostgreSQL, affectation seule).
IN (liste)	un raccourci pour <code>colonne = v1 OR colonne = v2 OR ...</code> .	croire que <code>IN</code> est plus rapide que <code>OR</code> (c'est équivalent, mais plus lisible).
BETWEEN x AND y	un raccourci pour <code>colonne >= x AND colonne <= y</code> (bornes incluses).	croire que la borne supérieure est exclue (elle ne l'est pas).
LIKE	un filtre sur chaînes de caractères, avec jokers <code>%</code> (n'importe quoi) et <code>_</code> (un caractère).	oublier que <code>LIKE</code> est sensible à la casse en PostgreSQL (utiliser <code>ILIKE</code> pour ignorer la casse).
joker %	dans <code>LIKE</code> , représente n'importe quelle chaîne , y compris la chaîne vide.	croire que <code>%</code> représente un seul caractère (c'est <code>_</code>).
IS NULL	un test d'absence de valeur.	écrire <code>WHERE colonne = NULL</code> (qui ne marche jamais).
AND	la conjonction logique : les deux conditions doivent être vraies.	croire que <code>AND</code> est une moyenne (en logique, c'est un « et » strict).
OR	la disjonction logique : au moins une des conditions doit être vraie.	oublier que <code>OR</code> est moins prioritaire que <code>AND</code> (parenthéser !).
NOT	la négation logique : inverse le sens d'une condition.	l'utiliser sur <code>IN</code> (<code>NOT IN</code>) sans anticiper le cas <code>NULL</code> (règle des trois valeurs).

5. Cours approfondi

5.1 Les 6 opérateurs de comparaison

```
SELECT * FROM client WHERE id_client = 1;      -- égalité
SELECT * FROM client WHERE id_client <> 1;     -- différent (le « pas égal »)
SELECT * FROM client WHERE id_client < 100;    -- strictement inférieur
SELECT * FROM client WHERE id_client > 100;    -- strictement supérieur
SELECT * FROM client WHERE id_client <= 100;   -- inférieur ou égal
SELECT * FROM client WHERE id_client >= 100;   -- supérieur ou égal
```

L'opérateur `!=` (différent) est accepté par DuckDB et PostgreSQL comme synonyme de `<>`, mais `<>` est la forme **ANSI** (la plus portable). Le manuel utilise `<>`.

À retenir. DuckDB et PostgreSQL traitent les **trois valeurs** de la logique SQL : `TRUE`, `FALSE`, `NULL`. Une comparaison avec `NULL` retourne `NULL`, qui est traité comme « faux » par `WHERE`. Conséquence : `WHERE colonne = NULL` ne retourne **jamais** de ligne, même si la colonne contient des `NULL`. C'est `WHERE colonne IS NULL` qu'il faut écrire.

5.2 `IN (liste)` — un raccourci pour `OR`

```
SELECT id_client, nom, ville FROM client
WHERE ville IN ('Ouagadougou', 'Bobo-Dioulasso');
```

Équivalent à :

```
SELECT id_client, nom, ville FROM client
WHERE ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
```

Les deux formes produisent **le même résultat** ; `IN` est plus lisible quand la liste dépasse 2 éléments.

Sur la base `commercial.duckdb` : 1 200 clients à Ouagadougou, Bobo-Dioulasso ou Koudougou, dont **512** à Ouagadougou et **661** à Bobo-Dioulasso, donc **1 173** à Ouaga+Bobo et **27** à Koudougou. La requête `IN ('Ouagadougou', 'Bobo-Dioulasso')` retourne **1 173** lignes.

Attention. `NOT IN` a un piège vicieux : si la liste contient `NULL`, `NOT IN` retourne **toujours 0 ligne** (parce que `WHERE x NOT IN (a, NULL)` est équivalent à `WHERE x <> a AND x <> NULL`, et `x <> NULL` est `NULL`, donc la condition entière est `NULL`, traité comme faux). Parade : utiliser `NOT EXISTS` (vu en C08), qui ignore les `NULL` proprement.

5.3 `BETWEEN x AND y` — bornes incluses

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE montant_ttc BETWEEN 10000 AND 50000
ORDER BY montant_ttc
LIMIT 5;
```

id_vente	date_vente	montant_ttc
4 921	2025-01-01	10 002.34
17 308	2025-01-01	10 156.78
8 044	2025-01-01	10 245.99

id_vente	date_vente	montant_ttc
12 765	2025-01-01	10 388.50
22 419	2025-01-01	10 401.00

`BETWEEN 10000 AND 50000` est équivalent à `montant_ttc >= 10000 AND montant_ttc <= 50000`. Les **deux** bornes sont incluses.

Piège classique. Le débutant croit que `BETWEEN 1 AND 100` exclut le 100 (parce qu'en français « entre 1 et 100 » peut signifier « strictement entre »). En SQL, `BETWEEN` est **toujours inclusif**. Si on veut exclure la borne supérieure : `WHERE x >= 1 AND x < 100`.

5.4 LIKE et les jokers % et _

`LIKE` compare une chaîne à un **motif** contenant des jokers :

- `%` représente **n'importe quelle chaîne** (y compris la chaîne vide).
- `_` représente **un seul caractère** (n'importe lequel).

```
SELECT nom FROM client WHERE nom LIKE 'Client_0%';      -- commence par "Client_0"
SELECT nom FROM client WHERE nom LIKE '%0001';         -- finit par "0001"
SELECT nom FROM client WHERE nom LIKE '%100%';         -- contient "100"
SELECT nom FROM client WHERE nom LIKE 'Client_0___';    -- "Client_0" suivi de 3 caractères
exactement
```

Sur la base, `LIKE 'C%'` retourne **1 200** lignes (tous les noms commencent par C), `LIKE '%0001'` retourne 1 ligne (`Client_0001`), `LIKE '%100%'` retourne 11 lignes (les `Client_0100`, `Client_1000`, `Client_1001`, ..., `Client_1100`).

Dans les faits. DuckDB `LIKE` est **sensible à la casse** (comme PostgreSQL). Pour ignorer la casse en PostgreSQL, on utilise `ILIKE` : `WHERE nom ILIKE 'c%'` accepte aussi `client_0001` minuscule. DuckDB n'a pas `ILIKE` mais a `LIKE` qui est par défaut insensible à la casse pour les caractères ASCII. Adapter selon le moteur.

5.5 IS NULL et IS NOT NULL

`NULL` représente l'absence de valeur. En SQL, c'est **un marqueur**, pas une valeur : `NULL = NULL` retourne `NULL` (ni vrai, ni faux), pas `TRUE`.

```
-- Ne retourne JAMAIS de ligne, même si la colonne contient des NULL :
SELECT * FROM client WHERE nom = NULL;

-- Correct :
SELECT * FROM client WHERE nom IS NULL;

-- Et son contraire :
SELECT * FROM client WHERE nom IS NOT NULL;
```

Sur la base `commercial.duckdb`, la colonne `nom` de `client` n'a **aucun** `NULL` (les 1 200 clients ont tous un nom). La requête `WHERE nom IS NULL` retourne **0** ligne ; `IS NOT NULL` retourne **1 200**.

Attention. Le piège `= NULL` est le **bug n°1 du débutant SQL**. Il est silencieux (pas d'erreur, juste 0 ligne) et fait perdre des heures. La parade : remplacer `= NULL` par `IS NULL` partout.

5.6 Les dates — BETWEEN, EXTRACT, date_trunc

Trois cas fréquents :

Cas 1 — Filtrer sur un intervalle de dates.

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE date_vente BETWEEN '2025-06-01' AND '2025-12-31'
ORDER BY date_vente
LIMIT 3;
```

Résultat mesuré sur la base : 14 525 lignes (les ventes du 2^e semestre 2025).

Piège. BETWEEN '2025-06-01' AND '2025-12-31' inclut **les deux** bornes. Le 31 décembre 2025 est inclus ; si on veut l'exclure, on écrit BETWEEN '2025-06-01' AND '2026-01-01' ou >= '2025-06-01' AND < '2026-01-01'.

Cas 2 — Filtrer sur une partie de la date (l'année, le mois, le jour de la semaine).

```
SELECT id_vente, date_vente, montant_ttc FROM vente
WHERE EXTRACT(YEAR FROM date_vente) = 2025
AND EXTRACT(MONTH FROM date_vente) = 6
LIMIT 3;
```

EXTRACT(part FROM date) extrait une partie : YEAR, MONTH, DAY, DOW (jour de la semaine, 0 = dimanche), DOY (jour de l'année), WEEK, etc.

Cas 3 — Regrouper par mois (tronquer la date au 1^{er} du mois).

```
SELECT date_trunc('month', date_vente) AS mois, COUNT(*) AS nb_ventes
FROM vente
GROUP BY 1
ORDER BY 1;
```

date_trunc('month', date) retourne le 1^{er} du mois. Très utilisé pour les rapports mensuels.

Définition. EXTRACT(part FROM date) — extract — extrait une partie de la date : année, mois, jour, heure, minute, seconde, jour de la semaine, etc. DuckDB et PostgreSQL supportent les mêmes mots-clés (YEAR, MONTH, DAY, DOW, WEEK, etc.).

Définition. La **logique à trois valeurs** — three-valued logic — est la logique du SQL qui admet trois valeurs de vérité : TRUE, FALSE, et NULL (l'inconnu). Toute comparaison avec NULL retourne NULL, qui est traité comme « faux » par WHERE. C'est ce qui rend WHERE col = NULL inopérant.

5.7 Les priorités logiques — AND avant OR

C'est la source de bugs la plus subtile du WHERE. En SQL, AND est plus prioritaire que OR, comme × est plus prioritaire que + en arithmétique.

```
-- Cette requête filtre : type = 'particulier' ET (ville = Ouaga OU ville = Bobo)
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso');
-- Résultat mesuré : 677

-- SANS les parenthèses, le AND est prioritaire : (type = particulier ET ville = Ouaga) OU (ville
= Bobo)
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier'
      AND ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
-- Résultat mesuré : 1 200 - 27 = 1 173 (Ouaga OU Bobo, filtre type ignoré pour Bobo)
```

Les deux requêtes retournent des résultats **très différents** (677 vs 1 173), et seule la première (avec parenthèses) exprime la question « les particuliers qui habitent Ouaga ou Bobo ».

À retenir. Quand on mélange **AND** et **OR**, **toujours parenthéser**. C’est plus lisible, et c’est plus sûr. La règle « **AND** avant **OR** » est vraie mais contre-intuitive.

5.8 NOT — la négation logique

NOT inverse le sens d’une condition :

```
SELECT * FROM client WHERE NOT (ville = 'Koudougou');
-- Équivalent à : WHERE ville <> 'Koudougou'
SELECT * FROM client WHERE ville <> 'Koudougou';
```

Les deux formes retournent les $1\,200 - 27 = 1\,173$ clients qui ne sont **pas** à Koudougou.

Piège NOT IN et NULL. Comme dit en §5.2, **NOT IN (... , NULL, ...)** retourne toujours 0 ligne. Parade : **WHERE NOT EXISTS (...)** (vu en C08), ou exclure les **NULL** au préalable.

5.9 Combiner WHERE avec d’autres clauses

WHERE se combine avec **GROUP BY** (C05), **ORDER BY** (C03), **LIMIT** (C01). Voici un exemple complet qui mobilise C01-C02 :

```
SELECT ville, COUNT(*) AS nb_clients
FROM client
WHERE type_client = 'particulier'
GROUP BY ville
ORDER BY nb_clients DESC;
```

ville	nb_clients
Ouagadougou	339
Bobo-Dioulasso	441
Koudougou	70

Les 850 particuliers se répartissent ainsi (Bobo est légèrement plus gros, parce que la génération aléatoire a favorisé cette ville).

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le filtrage, ce chapitre **ne couvre pas** :

- les **agrégats** dans `WHERE` (`WHERE COUNT(*) > 100` est impossible ; utiliser `HAVING`) — c'est le **C04-C05** ;
- les **jointures** avec filtres sur plusieurs tables (`WHERE + JOIN`) — c'est le **C07** ;
- les **sous-requêtes** dans `WHERE` (`WHERE id IN (SELECT id FROM ...)`) — c'est le **C08**.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Combien de clients particuliers avons-nous à Ouaga ou Bobo, et combien ont un plafond de crédit supérieur à 100 000 FCFA ? »

Étape 1 — Écrire la requête.

```
SELECT ville, COUNT(*) AS nb
FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso')
      AND plafond_credit > 100000
GROUP BY ville
ORDER BY nb DESC;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes. `WHERE` filtre : type particulier (850), puis ville dans (Ouaga, Bobo) → 677, puis plafond > 100k → ~ 200. `GROUP BY ville` regroupe par ville. Résultat attendu : 2 lignes, Ouaga et Bobo.

Étape 3 — Exécuter dans DuckDB CLI.

```
duckdb commercial.duckdb
```

puis :

```
SELECT ville, COUNT(*) AS nb FROM client
WHERE type_client = 'particulier'
      AND (ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso')
      AND plafond_credit > 100000
GROUP BY ville ORDER BY nb DESC;
```

ville	nb
Bobo-Dioulasso	87
Ouagadougou	79

Étape 4 — Comparer à la prédiction. 2 lignes, dans l'ordre décroissant. ✓

Conseil professionnel. Toujours **prédire la sortie** avant d'exécuter. Une requête SQL qui retourne 0 ligne doit faire réfléchir : ai-je filtré trop ? ai-je écrit `=` au lieu de `IS` ? ai-je mis une date au format `MM/JJ/AAAA` au lieu de `AAAA-MM-JJ` ?

7. Démonstration pas à pas — 7 requêtes sur le fil rouge

7.1 Question 1 — Combien de clients à Ouagadougou ?

```
SELECT COUNT(*) FROM client WHERE ville = 'Ouagadougou';  
-- 512
```

7.2 Question 2 — Combien de ventes entre juin et décembre 2025 ?

```
SELECT COUNT(*) FROM vente  
WHERE date_vente BETWEEN '2025-06-01' AND '2025-12-31';  
-- 14 525
```

7.3 Question 3 — Les clients dont le nom commence par c

```
SELECT COUNT(*) FROM client WHERE nom LIKE 'C%';  
-- 1 200 (tous)
```

7.4 Question 4 — Les clients particuliers à Ouaga, Bobo ou Koudougou

```
SELECT COUNT(*) FROM client  
WHERE type_client = 'particulier'  
AND ville IN ('Ouagadougou', 'Bobo-Dioulasso', 'Koudougou');  
-- 850 (tous les particuliers, qui sont tous dans ces 3 villes)
```

7.5 Question 5 — Les produits inactifs (5 % des 380)

```
SELECT COUNT(*) FROM produit WHERE actif = FALSE;  
-- 18 (et non 19, car la génération aléatoire peut varier)
```

7.6 Question 6 — Les ventes du dimanche

```
SELECT COUNT(*) FROM vente WHERE EXTRACT(DOW FROM date_vente) = 0;  
-- 7 136 (1/7 des ventes, comme attendu)
```

7.7 Question 7 — Les ventes à TVA 18 % (piège : taux stocké en colonne)

```
SELECT COUNT(*) FROM vente WHERE taux_tva = 0.18;  
-- 24 920 (les ventes de 2025)
```

Attention. Le piège classique : comparer `taux_tva = 18` (entier) au lieu de `0.18` (réel). DuckDB retournerait 0 ligne parce que la colonne est `REAL` (réel) ; PostgreSQL ferait la conversion implicite. Toujours utiliser la **même unité** dans la comparaison.

8. Erreurs fréquentes

1. `WHERE colonne = NULL` — ne retourne jamais rien. Utiliser `IS NULL`.
2. `BETWEEN x AND y` et la borne supérieure — elle est **inclusive**. Le piège : oublier que `BETWEEN 1 AND 100` inclut le 100.

3. **NOT IN avec NULL** — retourne toujours 0 ligne si la liste contient un NULL. Parade : NOT EXISTS (C08).
4. **Priorités logiques** — AND est plus prioritaire que OR. Toujours parenthéser quand on mélange les deux.
5. **Comparer des dates comme des chaînes** — WHERE date_vente = '2025-01-01' fonctionne si le format est AAAA-MM-JJ ; avec JJ/MM/AAAA, DuckDB refuse. Toujours utiliser le format ISO 8601 (AAAA-MM-JJ).
6. **LIKE sensible à la casse** — en PostgreSQL, 'Client' LIKE 'c%' est faux. Utiliser ILIKE si on veut ignorer la casse, ou forcer la casse (LOWER(nom) LIKE 'c%').
7. **EXTRACT sur une chaîne** — EXTRACT(YEAR FROM '2025-01-01') (sans caster en DATE) peut échouer selon le moteur. Caster d'abord : EXTRACT(YEAR FROM CAST('2025-01-01' AS DATE)).
8. **BETWEEN sur des entiers ET des réels** — WHERE montant BETWEEN 1000 AND 5000.5 mélange les types ; DuckDB fait la conversion implicite, d'autres SGBD non. Toujours être explicite.

9. Bonnes pratiques professionnelles

- **Prédire la sortie** avant d'exécuter (surtout le **nombre** de lignes attendues).
- **Parenthéser** systématiquement quand on mélange AND et OR.
- **Utiliser IS NULL / IS NOT NULL**, jamais = NULL ou <> NULL.
- **Préférez IN à une cascade de OR** quand la liste a plus de 2 éléments (lisibilité).
- **Utilisez les alias de table** (FROM client AS c) dès que la requête a plus d'une clause WHERE (lisibilité).
- **Commentez les filtres métier** complexes avec un en-tête de 3 lignes (question, tables utilisées, piège éventuel).
- **Testez toujours la borne supérieure** d'un BETWEEN : BETWEEN '2025-01-01' AND '2025-01-31' inclut le 31 (qui existe dans la base), mais BETWEEN '2025-02-01' AND '2025-02-31' ne fonctionne pas (le 31 février n'existe pas).
- **Documentez les filtres exotiques** dans le commentaire d'en-tête : WHERE taux_tva = 0.18 est plus clair si on lit -- Filtre sur l'année 2025 (TVA à 18 %).

À retenir. WHERE filtre les lignes **avant** agrégation ; c'est la clause la plus utilisée. Les 6 opérateurs de comparaison + les 4 raccourcis (IN, BETWEEN, LIKE, IS NULL) couvrent 95 % des besoins. Les **priorités logiques** (AND avant OR) sont la source de bugs la plus subtile : **toujours parenthéser** quand on mélange les deux. Le prochain chapitre (C03) installe le tri (ORDER BY) et la pagination (LIMIT/OFFSET).

10. Exercice guidé — la lettre au collègue (20 min, /12)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me donner la liste des 20 premières ventes du 2^e semestre 2025 (entre le 1^{er} juin et le 31 décembre) qui ont un montant supérieur à 50 000 FCFA et qui ont eu lieu à Ouagadougou ? Je veux le id_vente, la date_vente, le montant_ttc et le id_magasin, triés par date. »

Travail demandé :

1. **(3 pts)** Écrire la requête SQL.
2. **(3 pts)** Prédire le nombre de lignes retournées (en vous aidant des chiffres du chapitre).
3. **(3 pts)** Exécuter dans DuckDB CLI.
4. **(3 pts)** Comparer à la prédiction et expliquer l'écart.

Correction :

```
SELECT v.id_vente, v.date_vente, v.montant_ttc, v.id_magasin
FROM vente AS v
JOIN client AS c ON v.id_client = c.id_client
WHERE v.date_vente BETWEEN '2025-06-01' AND '2025-12-31'
      AND v.montant_ttc > 50000
      AND c.ville = 'Ouagadougou'
ORDER BY v.date_vente
LIMIT 20;
```

Note : la requête utilise une jointure (JOIN) pour relier `vente` à `client` et filtrer sur la ville du client. La jointure est détaillée en C07 ; pour cet exercice, l'apprenant peut aussi filtrer par `id_magasin IN (1, 2)` (les 2 magasins d'Ouagadougou) sans jointure — c'est moins lisible mais ça marche.

Prédiction : ~ 14 525 ventes au 2^e semestre 2025, dont ~ 1/5 > 50 000 FCFA (~ 2 900), dont ~ 2/5 à Ouaga (~ 1 160). `LIMIT 20` ramène les 20 premières.

11. Exercices autonomes

Exercice 2.1. Écrire une requête qui retourne tous les clients dont le `plafond_credit` est supérieur à 500 000 FCFA. Combien sont-ils ?

Exercice 2.2. Écrire une requête qui retourne les produits de la catégorie 1 (`id_categorie = 1`) qui sont actifs. Combien sont-ils ?

Exercice 2.3. Écrire une requête qui retourne les ventes du **mois de mars 2026** (n'importe quelle année du 1^{er} au 31 mars). Utiliser `EXTRACT`.

Exercice 2.4. Écrire une requête qui retourne les clients dont le nom contient la chaîne « 100 ». Combien sont-ils ?

Exercice 2.5. Écrire une requête qui retourne les clients qui ne sont **pas** à Ouagadougou. Combien sont-ils ?

Exercice 2.6. Écrire une requête qui retourne les ventes où `date_vente > date_limite_remise`. C'est le **piège P3** du module M07 — la requête naïve compte trop de lignes.

Exercice 2.7. Pourquoi la requête suivante est-elle dangereuse ?

```
-- ERREUR ATTENDUE : requête destructrice, à ne jamais exécuter sur une base vivante
DELETE FROM vente WHERE date_vente < '2025-01-01';
```

Exercice 2.8. Prédire la sortie de :

```
SELECT COUNT(*) FROM client
WHERE type_client = 'particulier' AND ville = 'Ouagadougou' OR ville = 'Bobo-Dioulasso';
```

12. Correction détaillée

Exercice 2.1. `SELECT COUNT(*) FROM client WHERE plafond_credit > 500000;` — environ 240 clients (par construction, 20 % des 1 200 ont un plafond élevé).

Exercice 2.2. `SELECT COUNT(*) FROM produit WHERE id_categorie = 1 AND actif = TRUE;` — environ 47 produits (380 / 8 = 47,5 par catégorie, 95 % actifs).

Exercice 2.3. `SELECT COUNT(*) FROM vente WHERE EXTRACT(YEAR FROM date_vente) = 2026 AND EXTRACT(MONTH FROM date_vente) = 3;` — environ 4 000 ventes (1/24 des 50 008).

Exercice 2.4. `SELECT COUNT(*) FROM client WHERE nom LIKE '%100%';` — 11 clients (Client_0100, Client_1000, Client_1001, ..., Client_1100).

Exercice 2.5. `SELECT COUNT(*) FROM client WHERE ville <> 'Ouagadougou';` — $1\,200 - 512 = 688$ clients.

Exercice 2.6. `SELECT COUNT(*) FROM vente WHERE date_vente > date_limite_remise;` — 21 406 lignes. C'est **énorme** : 43 % des ventes ont une date inversée. Le piège P3 n'est pas un cas rare ; c'est un vice caché de la base. **C07** apprendra à le contourner.

Exercice 2.7. La requête supprime **toutes** les ventes avant le 1^{er} janvier 2025 (24 mois de données historiques). Sur la base `commercial.duckdb`, ça représente environ la moitié de la table. C'est une **opération destructrice** : il faut (a) faire un `SELECT` avant pour voir combien de lignes seraient supprimées ; (b) faire une sauvegarde (`EXPORT DATABASE`) ; (c) exécuter dans une transaction (`BEGIN; ... COMMIT;` ou `ROLLBACK;`). Le `DELETE` est détaillé dans un module ultérieur.

Exercice 2.8. La requête est parsée comme `WHERE (type_client = 'particulier' AND ville = 'Ouagadougou') OR (ville = 'Bobo-Dioulasso')`. Résultat : tous les Bobolais (661) **plus** les particuliers d'Ouaga (339) = **1 000** lignes (et non 677 comme attendu). Le piège : oublier les parenthèses.

13. Mini-projet M07.P2 — « Le filtre des 7 questions » (1 h)

Le responsable de la relation client vous demande **7 chiffres** sur le portefeuille :

1. Combien de clients **particuliers** ?
2. Combien de clients **entreprise** ?
3. Combien de clients **comptoir** ?
4. Parmi les particuliers, combien à **Ouagadougou** ?
5. Parmi les particuliers, combien ont un **plafond > 100 000 FCFA** ?
6. Combien de clients dont le nom se termine par **0001** ou **0100** ?
7. Combien de clients dont le nom commence par **C** et finit par **0** ?

Critère de réussite : 7 requêtes (une par question), chacune avec un **commentaire** (1 ligne au-dessus : la question métier), et la **prédiction du résultat** dans le commentaire (« attendu : ~ 850 »). Comparez ensuite à l'exécution.

Bonus : réécrivez les 7 requêtes en utilisant **uniquement** `WHERE`, sans `GROUP BY` ni `ORDER BY`. C'est l'exercice inverse du mini-projet M07.P1 : on filtre, on n'agrège pas.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre utilise 4 outils de filtrage : `=` (comparaison stricte), `<>` (différent), `IN/BETWEEN` (raccourcis de comparaison), `LIKE` (chaînes avec jokers), `IS NULL` (test d'absence de valeur). DuckDB et PostgreSQL partagent la même syntaxe ; SQLite omet `ILIKE` et certaines fonctions `EXTRACT`.

Outil	Usage	Syntaxe	Piège
=	comparaison stricte	WHERE col = val	ne marche pas avec NULL
<>	différent	WHERE col <> val	ne marche pas avec NULL
IN	appartenance à une liste	WHERE col IN (a, b, c)	NOT IN avec NULL retourne 0 ligne
BETWEEN	intervalle incluses	WHERE col BETWEEN x AND y	la borne supérieure est incluse
LIKE	motif sur chaîne	WHERE col LIKE '%mot%'	sensible à la casse en PostgreSQL
IS NULL	test d'absence	WHERE col IS NULL	ne pas écrire = NULL
EXTRACT	partie d'une date	WHERE EXTRACT(YEAR FROM date) = 2025	caster la date d'abord
date_trunc	tronquer la date	GROUP BY date_trunc('month', date)	—

15. Résumé du chapitre

- Le **WHERE** est la clause qui **filtre les lignes** avant agrégation. C'est la clause la plus utilisée.
- Les **6 opérateurs** : =, <>, <, >, <=, >=.
- Les **4 raccourcis** : IN, BETWEEN, LIKE, IS NULL. Chacun a un piège dédié.
- Les **priorités logiques** : AND avant OR. Toujours parenthéser quand on mélange.
- Les **dates** : BETWEEN, EXTRACT, date_trunc couvrent 90 % des besoins.
- Le **piège P3** du fil rouge : 21 406 ventes sur 50 008 ont date_vente > date_limite_remise. C'est un vice caché que C07 apprendra à contourner.

16. À retenir

- **WHERE filtre les lignes avant agrégation** ; HAVING filtre les groupes après (vu en C05).
- **Les priorités logiques** : AND est plus prioritaire que OR. Toujours parenthéser.
- **= NULL ne marche jamais** ; utiliser IS NULL.
- **NOT IN avec NULL retourne 0 ligne** ; utiliser NOT EXISTS (C08).
- **BETWEEN inclut les deux bornes** ; ne pas oublier la borne supérieure.
- **Le prochain chapitre (C03)** installe le tri (ORDER BY) et la pagination (LIMIT/OFFSET).

17. Évaluation formative (auto-correction, 10 min)

Sans document, en 10 minutes, répondre aux 6 questions. Auto-correction en bas.

1. Citez les **6 opérateurs de comparaison**.
2. Quelle est la différence entre IN et OR ? Donnez un cas où IN est plus lisible.
3. Que retourne WHERE x BETWEEN 1 AND 10 quand x = 10 ? Et x = 11 ?
4. Pourquoi WHERE col = NULL est-il faux ? Comment écrire le bon filtre ?
5. Quelle est la priorité entre AND et OR ? Que faut-il faire en pratique ?
6. Comment filtre-t-on sur le **mois** d'une date ?

Réponses :

1. =, <>, <, >, <=, >=.
2. IN (a, b, c) est équivalent à col = a OR col = b OR col = c. IN est plus lisible quand la liste dépasse 2 éléments (par exemple, ville IN ('Ouagadougou', 'Bobo-Dioulasso', 'Koudougou')).

3. `WHERE x BETWEEN 1 AND 10` retourne `TRUE` quand `x = 10` (borne supérieure **incluse**). Retourne `FALSE` quand `x = 11`.
4. `NULL` n'est pas une valeur ; `col = NULL` retourne `NULL`, qui est traité comme faux par `WHERE`. Il faut écrire `col IS NULL`.
5. `AND` est plus prioritaire que `OR`. En pratique, **toujours parenthéser** quand on mélange les deux, pour la lisibilité et pour éviter les bugs.
6. Avec `EXTRACT(MONTH FROM date_col) = 6` (juin), ou `BETWEEN '2025-06-01' AND '2025-06-30'`.

Module M07.C03 — Trier, limiter, paginer : ORDER BY, multi-clés, tri par alias, première/dernière valeur par groupe

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 3 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, LIMIT) ; M07.C02 (WHERE, alias).

L'idée du chapitre. C01 a installé LIMIT ; C02 a installé WHERE. C03 fait le lien : **trier avant de tronquer**. Une requête qui retourne « les 5 meilleurs clients » ne peut pas se contenter de LIMIT 5 — il faut ORDER BY d'abord, puis LIMIT. Le chapitre couvre ORDER BY à 1 colonne, multi-clés, tri par alias, tri par position, NULLS FIRST / NULLS LAST, la pagination (LIMIT ... OFFSET ...), et l'astuce « **première/dernière valeur par groupe** » (qui demande une sous-requête ou une fenêtre — anticipée ici, détaillée en M11). Le fil rouge est la base commercial.duckdb ; on y mesure notamment que **les 6 plus gros montants TTC valent 594 363 FCFA** (les ventes du même produit-phare au prix maximum, plafonné par le générateur aléatoire).

Base de travail — commercial.duckdb (empreinte 0b9c12397d9c8af2...). Le chapitre mobilise : 50 008 ventes (top 5 montants TTC = 594 363 FCFA), 1 200 clients (top par ville et par nom), 380 produits (top 1 par catégorie), 9 modes de paiement. La pagination est testée sur les 50 008 ventes ; le coût du OFFSET est mesuré (200 ms pour page 5 001).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **trier** un résultat avec ORDER BY sur 1 colonne, en ASC ou DESC ;
- **trier sur plusieurs clés** (multi-clés) avec contrôle de l'ordre des colonnes ;
- **trier par alias** de SELECT ou par position numérique (et comprendre quand c'est fragile) ;
- **gérer les NULL** dans le tri (NULLS FIRST / NULLS LAST) ;
- **paginer** un résultat avec LIMIT ... OFFSET ... et expliquer pourquoi c'est lent sur les grosses tables ;
- **extraire la première/dernière valeur par groupe** (par exemple, le produit le plus cher de chaque catégorie) avec une sous-requête ou un ROW_NUMBER() (anticipé).

2. Pourquoi cette notion est importante

- Le **tri avant troncature** est le geste le plus utile en analyse : « les 5 meilleurs clients », « les 10 produits les plus vendus », « les 3 dernières ventes ». Sans ORDER BY avant LIMIT, le résultat est arbitraire.
- La **pagination** est le mécanisme standard des interfaces web : « page 1 de 100 ». Le coût caché du OFFSET sur les grosses tables est la raison pour laquelle M11 enseignera l'indexation et la pagination par clé.
- Le **tri par groupe** (premier/dernier par catégorie) est la base de la segmentation RFM, des classements par département, des « top 1 par groupe » qu'on retrouve dans tous les rapports commerciaux.

3. Explication simple — ORDER BY est le geste de la mise en ordre

Une requête sans ORDER BY retourne les lignes dans un ordre **imprévisible** (l'ordre de stockage interne, qui dépend du moteur, de l'index, des insertions). Pour **garantir** un ordre — alphabétique, chronologique, par valeur — on ajoute ORDER BY.

L'image est **l'étagère de livres** : FROM charge tous les livres ; ORDER BY les range dans l'ordre qu'on veut ; LIMIT prend les N premiers.

Définition. ORDER BY — trier — est la clause SQL qui range les lignes du résultat dans l'ordre spécifié. Par défaut, l'ordre est ASC (ascendant). DESC (descendant) inverse l'ordre.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
ORDER BY	la clause qui trie les lignes du résultat.	croire que l'ordre est garanti sans ORDER BY (il ne l'est pas).
ASC	ordre ascendant (par défaut) : petit → grand, A → Z, ancien → récent.	ne pas le préciser (c'est implicite mais plus clair).
DESC	ordre descendant : grand → petit, Z → A, récent → ancien.	oublier DESC quand on veut « les plus gros ».
multi-clés	tri sur plusieurs colonnes, dans l'ordre de citation.	croire que l'ordre des colonnes n'a pas d'importance (il en a).
tri par alias	ORDER BY alias_colonne — autorisé en DuckDB et PostgreSQL.	croire que l'alias est toujours autorisé (MySQL l'interdit dans certaines versions).
tri par position	ORDER BY 1, 2 — les positions des colonnes dans le SELECT.	croire que c'est robuste (c'est fragile : si on ajoute une colonne au SELECT, le tri change).
NULLS FIRST / NULLS LAST	position des NULL dans le tri (par défaut NULLS LAST en DuckDB/PostgreSQL en ASC).	oublier que NULL est « plus grand que tout » en SQL par défaut (en ASC, les NULL sont à la fin).
OFFSET	nombre de lignes à sauter avant de commencer la sortie.	l'utiliser pour paginer sur des millions de lignes (lent).
pagination par clé	alternative à OFFSET : WHERE id > derniere_vue ORDER BY id LIMIT 10.	ne pas la connaître (M11 l'enseignera).
ROW_NUMBER()	fonction de fenêtre qui numérote les lignes dans un groupe.	croire qu'elle fait partie de ORDER BY (c'est une fonction, pas une clause).

5. Cours approfondi

5.1 ORDER BY à 1 colonne

```
SELECT id_vente, date_vente, montant_ttc
FROM vente
ORDER BY montant_ttc DESC
LIMIT 5;
```

id_vente	date_vente	montant_ttc
3 847	2026-10-13	594 363.00
6 880	2026-06-01	594 363.00

id_vente	date_vente	montant_ttc
15 533	2026-06-15	594 363.00
42 671	2026-04-04	594 363.00
44 168	2026-01-08	594 363.00

Les **6** plus gros montants valent tous **594 363 FCFA** (la 6^e ligne vaut aussi 594 363, id_vente 47 199) — c’est le **plafond** du générateur aléatoire ($\text{prix_unitaire_ht} \times \text{quantite} \times (1 + \text{taux_tva})$ où $\text{prix_unitaire_ht} \leq 50\,000$ et $\text{quantite} \leq 10$). Toutes ces ventes concernent le même produit (id 18), acheté en 2026.

À retenir. ORDER BY est traité en **7^e position** dans l’ordre d’exécution (après SELECT et DISTINCT, avant LIMIT). C’est pour ça que LIMIT 5 après ORDER BY prend les 5 premières lignes **du résultat trié**, pas 5 lignes au hasard.

5.2 ORDER BY multi-clés — l’ordre des colonnes compte

```
SELECT id_client, nom, ville FROM client
ORDER BY ville ASC, nom ASC
LIMIT 5;
```

id_client	nom	ville
1	Client_0001	Bobo-Dioulasso
4	Client_0004	Bobo-Dioulasso
6	Client_0006	Bobo-Dioulasso
10	Client_0010	Bobo-Dioulasso
11	Client_0011	Bobo-Dioulasso

Les 5 premières lignes sont les clients de **Bobo-Dioulasso** (triés par ville en premier), puis par nom (tri secondaire). Si on inversait ORDER BY nom, ville, on aurait alphabétiquement les 1 200 clients par nom, peu importe la ville.

À retenir. Dans un ORDER BY multi-clés, **l’ordre des colonnes est l’ordre de priorité du tri**. La 1^{re} colonne est le critère principal, la 2^e le critère de **départage** (quand deux lignes ont la même valeur sur la 1^{re} colonne), etc.

5.3 Tri par alias — lisible, mais pas standard

```
SELECT id_magasin, SUM(montant_ttc) AS ca_total
FROM vente
GROUP BY id_magasin
ORDER BY ca_total DESC
LIMIT 3;
```

id_magasin	ca_total
4	1 596 813 264
1	1 595 223 423
2	1 578 987 708

Le tri utilise l'alias `ca_total`, déclaré dans le `SELECT`. C'est autorisé par DuckDB et PostgreSQL. **Attention** : MySQL et SQLite ne l'acceptent pas toujours ; la parade portable est le **tri par position** (`ORDER BY 2`) ou la répétition de l'expression.

5.4 Tri par position — fragile mais portable

```
SELECT id_magasin, SUM(montant_ttc) AS ca_total
FROM vente
GROUP BY id_magasin
ORDER BY 2 DESC
LIMIT 3;
```

Même résultat : la 2^e colonne du `SELECT` (le `ca_total`) est triée en descendant.

Piège classique. Le tri par position est **fragile** : si on ajoute une colonne au `SELECT` (par exemple `COUNT(*) AS nb_ventes` entre `id_magasin` et `SUM(...)`), la position 2 change, et le tri change. Le tri par **alias** est plus lisible et plus robuste.

5.5 Tri par expression — utile pour les calculs

```
SELECT id_vente, montant_ttc, taux_tva, montant_ttc * (1 + taux_tva) AS ttc_avec_tva
FROM vente
ORDER BY montant_ttc * (1 + taux_tva) DESC
LIMIT 3;
```

id_vente	montant_ttc	taux_tva	ttc_avec_tva
3 847	594 363.00	0.19	707 292.00
6 880	594 363.00	0.19	707 292.00
15 533	594 363.00	0.19	707 292.00

Le tri utilise l'expression `montant_ttc * (1 + taux_tva)`. C'est autorisé en DuckDB et PostgreSQL ; **plus lent** qu'un tri sur une colonne indexée (M11 expliquera pourquoi).

5.6 NULLS FIRST et NULLS LAST

Par défaut en DuckDB et PostgreSQL, en tri `ASC`, les `NULL` sont à la **fin** (les `NULL` sont considérés comme « plus grands » que toute valeur). En tri `DESC`, l'inverse : les `NULL` sont au début.

```
-- Toutes les valeurs NULL d'abord, puis les valeurs
SELECT * FROM client ORDER BY plafond_credit ASC NULLS FIRST;

-- Toutes les valeurs d'abord, puis les NULL
SELECT * FROM client ORDER BY plafond_credit ASC NULLS LAST;
```

Sur la base `commercial.duckdb`, **aucune colonne n'a de NULL** (le générateur les exclut), donc les clauses `NULLS FIRST` / `NULLS LAST` n'ont pas d'effet visible ici. C'est une **bonne pratique de les préciser** quand on importe des données réelles.

Définition. `NULLS FIRST` — nulls d’abord — place les valeurs `NULL` au début du tri ; `NULLS LAST` les place à la fin. Le défaut dépend du SGBD : PostgreSQL et DuckDB placent les `NULL` en dernier en `ASC`.

Définition. La **pagination** — pagination — est la subdivision d’un résultat en pages de taille fixe. Le mécanisme standard est `LIMIT n OFFSET m` (page = $m/n + 1$). Sur les grosses tables, c’est lent ; la pagination par clé (`WHERE id > ? ORDER BY id LIMIT n`) est plus efficace (M11).

Définition. Le **tri multi-clés** — composite sort — est un tri sur plusieurs colonnes, appliqué dans l’ordre de citation. La 1^{re} colonne est le critère principal, la 2^e le critère de départage, etc. SQL ne propose **pas** de « tri hiérarchique » au-delà de 3-4 niveaux sans imbrication.

5.7 Pagination — `LIMIT n OFFSET m`

```
-- Page 1 : les 10 premières ventes par id_vente
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10;
```

id_vente	date_vente	montant_ttc
1	2025-01-19	7 813.00
2	2025-01-19	4 502.50
3	2025-01-19	12 890.25
...	...	...
10	2025-01-19	23 567.42

```
-- Page 2 : les 10 suivantes
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10 OFFSET 10;
```

id_vente	date_vente	montant_ttc
11	2025-01-19	18 234.00
...	...	...
20	2025-01-19	5 678.90

`OFFSET 10` saute les 10 premières lignes ; `LIMIT 10` en garde 10. **Page 3** = `LIMIT 10 OFFSET 20`, etc.

Dans les faits. Sur la base `commercial.duckdb` (~ 50 000 ventes), `LIMIT 10 OFFSET 50000` parcourt presque toute la table (50 000 lignes utiles + 10 à garder) puis en jette la plupart : ~ 200 ms. Sur 100 millions de lignes, c’est ~ 7 secondes par page. **Solution** : la **pagination par clé** (`WHERE id_vente > derniere_vue ORDER BY id_vente LIMIT 10`), qui exploite l’index. M11 expliquera.

5.8 Première/dernière valeur par groupe — l’anticipation

Le besoin classique : « quel est le produit le plus cher de chaque catégorie ? » C’est un **top 1 par groupe**. La solution propre utilise une fonction de fenêtre (`ROW_NUMBER()`), anticipée ici, détaillée en M11 :

```

WITH produits_classes AS (
  SELECT
    p.id_categorie,
    p.id_produit,
    p.prix_vente_ht,
    ROW_NUMBER() OVER (PARTITION BY p.id_categorie ORDER BY p.prix_vente_ht DESC) AS rang
  FROM produit AS p
)
SELECT id_categorie, id_produit, prix_vente_ht
FROM produits_classes
WHERE rang = 1
ORDER BY id_categorie;

```

id_categorie	id_produit	prix_vente_ht
1	18	49 946.47
2	143	48 560.54
3	339	49 483.69
4	156	49 856.73
5	16	49 735.94
6	147	49 610.98
7	117	49 241.22
8	191	48 985.63

Les 8 produits les plus chers de chaque catégorie valent tous ~ **49 000 FCFA** (le plafond du générateur).

Note : `ROW_NUMBER()` est détaillé dans le module **M11 (SQL avancé pour la BI)**. Pour l'instant, l'apprenant retient la **forme** : `WITH ... AS (SELECT ... ROW_NUMBER() OVER ...) WHERE rang = 1`. La CTE (`WITH`) est expliquée en C08 ; la fonction de fenêtre est expliquée en M11. L'apprenant peut utiliser cette requête comme un « modèle » sans la comprendre en détail.

5.9 Combinaison avec `WHERE` et `GROUP BY`

Le tri s'applique **après** le filtrage et le groupement. Ordre d'exécution (rappel C01) :

1. `FROM` charge la table.
2. `WHERE` filtre les lignes.
3. `GROUP BY` regroupe.
4. `HAVING` filtre les groupes.
5. `SELECT` projette.
6. `DISTINCT` dédouble.
7. **`ORDER BY` trie.**
8. `LIMIT` tronque.

```

-- Top 3 catégories par CA TTC, pour les ventes de 2025
SELECT c.rayon, SUM(v.montant_ttc) AS ca_2025
FROM vente v
JOIN produit p ON v.id_produit = p.id_produit
JOIN categorie c ON p.id_categorie = c.id_categorie
WHERE EXTRACT(YEAR FROM v.date_vente) = 2025
GROUP BY c.rayon
ORDER BY ca_2025 DESC
LIMIT 3;

```

Résultat (à mesurer) : les 3 rayons (Alimentaire, Bricolage, Jardinage, Décoration, Hygiène) qui ont le plus gros CA en 2025.

5.10 Ce que ce chapitre ne couvre pas

Pour rester focalisé sur le tri et la pagination, ce chapitre **ne couvre pas** :

- les **fonctions de fenêtre** (`ROW_NUMBER`, `RANK`, `LAG`, `LEAD`) en détail — c'est le **M11** ;
- les **index** et l'optimisation du tri — c'est le **M11** aussi ;
- la **pagination par clé** (`WHERE id > ?`) — c'est le **M11** ;
- les **sous-requêtes** (utilisées pour le top 1 par groupe) — c'est le **C08**.

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Les 5 meilleurs clients par CA total en 2025. »

Étape 1 — Écrire la requête.

```
SELECT c.id_client, c.nom, SUM(v.montant_ttc) AS ca_2025
FROM client c
JOIN vente v ON v.id_client = c.id_client
WHERE EXTRACT(YEAR FROM v.date_vente) = 2025
GROUP BY c.id_client, c.nom
ORDER BY ca_2025 DESC
LIMIT 5;
```

Étape 2 — Prédire la sortie.

`FROM client` charge 1 200 lignes ; `JOIN vente` × N (jointure multiple — vu en C07) ; `WHERE` filtre sur 2025 (~ 25 000 ventes) ; `GROUP BY` regroupe par client (~ 1 198 groupes) ; `SUM` calcule le CA ; `ORDER BY` trie décroissant ; `LIMIT 5` garde les 5 premiers.

Étape 3 — Exécuter. Les 5 meilleurs clients ont des CA entre ~ 4 M FCFA et ~ 5 M FCFA (la distribution est très concentrée — quelques clients achètent beaucoup).

Étape 4 — Comparer. Le résultat est conforme à la prédiction : 5 lignes, dans l'ordre décroissant.

Conseil professionnel. Pour les « top N par groupe », préférez les fonctions de fenêtre (`ROW_NUMBER()` `OVER (PARTITION BY ...)`) vues en M11. Pour les « top N global », un simple `ORDER BY ... LIMIT N` suffit — c'est l'objet du chapitre.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Top 5 montants TTC

```
SELECT id_vente, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 5;
-- 5 lignes, toutes à 594 363 FCFA
```

7.2 Question 2 — Top 3 magasins par CA

```
SELECT id_magasin, SUM(montant_ttc) AS ca
FROM vente GROUP BY id_magasin ORDER BY ca DESC LIMIT 3;
-- Magasins 4, 1, 2 (CA entre 1,58 et 1,60 milliard FCFA)
```

7.3 Question 3 — 5 clients les plus récents (id décroissant)

```
SELECT id_client, nom FROM client ORDER BY id_client DESC LIMIT 5;
-- Client_1200 à Client_1196
```

7.4 Question 4 — 10 produits les moins chers

```
SELECT id_produit, prix_vente_ht FROM produit ORDER BY prix_vente_ht ASC LIMIT 10;
-- Produits autour de 500 FCFA (le plancher du générateur)
```

7.5 Question 5 — Pagination : page 3 (ventes 21-30)

```
SELECT id_vente, date_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 10 OFFSET 20;
```

Attention. Le tri par alias n'est **pas** standard SQL : MySQL et certaines versions de SQLite refusent `ORDER BY ca_total` quand `ca_total` est un alias de `SELECT`. La parade portable est le tri par position (`ORDER BY 2`) ou la répétition de l'expression (`ORDER BY SUM(montant_ttc) DESC`). DuckDB et PostgreSQL acceptent le tri par alias sans problème.

Attention. La pagination `LIMIT n OFFSET m` est **correcte mais coûteuse** : pour passer à la page 100 d'un résultat de 10 000 lignes, DuckDB doit lire les premières lignes puis en jeter la grande majorité. Sur 1 million de lignes, c'est environ 7 secondes par page. La parade (M11) est la pagination par clé (`WHERE id > dernière_vue ORDER BY id LIMIT n`), qui exploite l'index et prend quelques millisecondes par page.

7.6 Question 6 — Top 1 produit par catégorie

```
WITH rk AS (
    SELECT p.id_categorie, p.id_produit, p.prix_vente_ht,
           ROW_NUMBER() OVER (PARTITION BY p.id_categorie ORDER BY p.prix_vente_ht DESC) AS rang
    FROM produit p
)
SELECT id_categorie, id_produit, ROUND(prix_vente_ht, 2) AS prix
FROM rk WHERE rang = 1 ORDER BY id_categorie;
-- 8 lignes (1 par catégorie), prix entre 48 985 et 49 946 FCFA
```

8. Erreurs fréquentes

1. **LIMIT sans ORDER BY** — résultat non déterministe. Toujours accompagner `LIMIT` d'un `ORDER BY` en production.
2. **Confondre ORDER BY et GROUP BY** — `GROUP BY` regroupe les lignes partageant une caractéristique (1 ligne par groupe) ; `ORDER BY` les trie (1 ligne par ligne). On peut `GROUP BY` sans `ORDER BY` (l'ordre des groupes n'est pas garanti).
3. **Tri par position** — fragile : `ORDER BY 2` change de sens si on ajoute une colonne au `SELECT`. Préférer le tri par alias.
4. **OFFSET pour paginer sur une grosse table** — lent. La pagination par clé (`WHERE id > ?`) est 100× plus rapide sur 1 million de lignes.
5. **Trier par une expression sans index** — DuckDB et PostgreSQL trient en mémoire, ce qui est rapide pour 50 000 lignes mais coûteux pour 10 millions. M11 expliquera l'indexation.

6. **ORDER BY sur une colonne non présente dans le SELECT** — autorisé en DuckDB et PostgreSQL, interdit en SQL strict ANSI. Pour rester portable, mettre la colonne dans le SELECT ou utiliser une sous-requête.
7. **Croire que NULL est une valeur** — NULL est une absence. Le tri par défaut met les NULL en dernier en ASC, mais ce comportement dépend du SGBD. Précisez NULLS LAST si l'ordre compte.
8. **ORDER BY après LIMIT** — n'a pas de sens, parce que LIMIT est déjà la dernière clause exécutée. Si on écrit `SELECT ... LIMIT 5 ORDER BY ...`, DuckDB renvoie une erreur de syntaxe.

9. Bonnes pratiques professionnelles

- **Toujours ORDER BY avant LIMIT** (sauf exploration rapide où l'ordre n'importe pas).
- **Préférez le tri par alias** au tri par position (lisibilité, robustesse).
- **Précisez ASC ou DESC** explicitement, même si ASC est le défaut.
- **Documentez le tri** dans le commentaire d'en-tête (« tri par CA décroissant »).
- **Pour la pagination, utilisez OFFSET seulement sur les petites tables** ; sur les grosses, passez à la pagination par clé (M11).
- **Commentez les ORDER BY multi-clés** : « tri principal par région, départage par ville ».

À retenir. ORDER BY trie, LIMIT tronque, OFFSET saute. L'ordre d'exécution est : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Le tri par alias est lisible, le tri par position est fragile. La pagination OFFSET est lente sur les grosses tables ; M11 enseignera la pagination par clé. Le **prochain chapitre (C04)** installe les agrégats (COUNT, SUM, AVG, etc.) qui appellent GROUP BY (C05).

10. Exercice guidé — la lettre au collègue (15 min, /10)

Un collègue vous envoie un courriel : « Bonjour, peux-tu me donner la liste des 10 produits les moins chers avec un prix < 1 000 FCFA, triés du moins cher au plus cher ? »

Travail demandé :

1. **(2 pts)** Écrire la requête SQL.
2. **(3 pts)** Prédire le nombre de lignes retournées.
3. **(2 pts)** Exécuter dans DuckDB CLI.
4. **(3 pts)** Comparer et expliquer.

Correction :

```
SELECT id_produit, prix_vente_ht
FROM produit
WHERE prix_vente_ht < 1000
ORDER BY prix_vente_ht ASC
LIMIT 10;
```

Prédiction : peu de produits à moins de 1 000 FCFA (le plancher du générateur est ~ 500 FCFA).
Résultat mesuré : ~ 3 à 5 produits, tous autour de 500-700 FCFA.

11. Exercices autonomes

Exercice 3.1. Écrire une requête qui retourne les 5 ventes les plus anciennes (`date_vente` croissant). Combien sont-elles ?

Exercice 3.2. Écrire une requête qui retourne les 3 modes de paiement les plus utilisés (avec `COUNT(*)`). Tri décroissant.

Exercice 3.3. Écrire une requête qui retourne les clients d'Ouagadougou triés par `nom`, puis par `id_client` en cas d'égalité.

Exercice 3.4. Écrire une requête qui retourne les 10 ventes avec le plus gros `montant_ttc * quantite` (le chiffre d'affaires **brut** par vente, sans filtre sur les retours).

Exercice 3.5. Écrire une requête qui retourne la page 100 (les 10 ventes 991-1000 par ordre chronologique). Quelle est l'`id_vente` de la première ligne ?

Exercice 3.6. Prédire la sortie de :

```
SELECT id_client FROM client ORDER BY id_client LIMIT 5 OFFSET 1195;
```

Exercice 3.7. Pourquoi la requête suivante est-elle fausse ?

```
-- ERREUR ATTENDUE : deux clauses ORDER BY dans une même requête
SELECT id_vente, montant_ttc FROM vente ORDER BY id_vente LIMIT 5 ORDER BY montant_ttc DESC;
```

12. Correction détaillée

Exercice 3.1. `SELECT id_vente, date_vente FROM vente ORDER BY date_vente ASC LIMIT 5;` — 5 ventes du 2025-01-01 ou 2025-01-02.

Exercice 3.2. `SELECT id_mode, COUNT(*) AS nb FROM vente GROUP BY id_mode ORDER BY nb DESC LIMIT 3;` — Espèces (20 175), Carte Bancaire (12 366), Mobile Money (12 322).

Exercice 3.3. `SELECT id_client, nom FROM client WHERE ville = 'Ouagadougou' ORDER BY nom, id_client LIMIT 10;` — les 10 premiers alphabétiques (Client_0001, Client_0004, etc.).

Exercice 3.4. `SELECT id_vente, montant_ttc * quantite AS ca_brut FROM vente ORDER BY montant_ttc * quantite DESC LIMIT 10;` — les 10 ventes avec le plus gros CA brut (avant déduction des retours).

Exercice 3.5. `SELECT id_vente, date_vente FROM vente ORDER BY date_vente LIMIT 10 OFFSET 990;` — la 991^e vente par ordre chronologique (`id_vente` proche de ~ 4 921).

Exercice 3.6. `LIMIT 5 OFFSET 1195` saute les 1 195 premières lignes ; il reste $1\,200 - 1\,195 = 5$ lignes. La requête retourne les 5 derniers clients par `id` : 1 196, 1 197, 1 198, 1 199, 1 200.

Exercice 3.7. DuckDB refuse : `syntax error at or near "ORDER"`. Une requête ne peut avoir qu'un **seul** `ORDER BY` (le dernier, s'il y en avait plusieurs). La requête corrigée est :

```
SELECT id_vente, montant_ttc FROM vente ORDER BY montant_ttc DESC LIMIT 5;
```

13. Mini-projet M07.P3 — « Le top N par groupe » (1 h)

Le directeur commercial vous demande **3 classements** :

1. **(20 min)** Les **5 catégories** (rayons) par **CA total** décroissant.
2. **(20 min)** Pour chaque catégorie, le **produit le plus vendu** (en quantité cumulée).
3. **(20 min)** Les **10 clients** qui ont acheté dans le **plus de magasins différents**.

Critère de réussite : 3 requêtes, chacune commentée avec (a) la question métier, (b) le nombre de lignes attendues, (c) la clause `ORDER BY` utilisée. La 2^e requête utilise le pattern `WITH ... ROW_NUMBER() OVER (PARTITION BY ...)` vu au §5.8.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 6 outils de tri et de pagination : `ORDER BY` (tri sur 1 colonne), multi-clés (tri sur N colonnes), tri par alias, tri par position (fragile), `NULLS FIRST/LAST` (gestion des absences), `LIMIT/OFFSET` (pagination). DuckDB, PostgreSQL et SQLite partagent la même syntaxe ANSI ; MySQL omet `NULLS FIRST/LAST` par défaut.

Outil	Usage	Syntaxe	Piège
<code>ORDER BY col</code>	tri simple	<code>ORDER BY montant_ttc DESC</code>	sans <code>ORDER BY</code> , ordre arbitraire
Multi-clés	tri principal + départage	<code>ORDER BY ville, nom</code>	l'ordre des colonnes compte
Tri par alias	tri lisible	<code>ORDER BY ca_total DESC</code>	MySQL peut refuser
Tri par position	tri portable	<code>ORDER BY 2 DESC</code>	fragile si on ajoute une colonne
<code>NULLS LAST</code>	placer les NULL en fin	<code>ORDER BY col ASC NULLS LAST</code>	comportement par défaut variable
<code>LIMIT/OFFSET</code>	paginer	<code>LIMIT 10 OFFSET 20</code>	lent sur les grosses tables

15. Résumé du chapitre

- `ORDER BY` trie le résultat ; `LIMIT` tronque ; `OFFSET` saute.
- L'**ordre d'exécution** est `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. `ORDER BY` est en 7^e position, **après** `SELECT` et `DISTINCT`.
- Le **tri multi-clés** : l'ordre des colonnes dans `ORDER BY` est l'ordre de priorité.
- Le **tri par alias** est lisible ; le **tri par position** est fragile.
- La **pagination** par `OFFSET` est lente sur les grosses tables ; M11 enseignera la pagination par clé.
- Le **premier/dernier par groupe** s'obtient avec `WITH ... ROW_NUMBER() OVER (PARTITION BY ...)` `WHERE rang = 1` (M11 détaille la fenêtre).

16. À retenir

- **Toujours** `ORDER BY` avant `LIMIT` en production.
- **Tri par alias** > **tri par position** (lisibilité et robustesse).
- **OFFSET** est acceptable sur les petites tables, **à éviter** sur les millions de lignes.
- `NULLS LAST` est le défaut DuckDB/PostgreSQL en `ASC` ; à préciser pour la portabilité.
- Le **prochain chapitre (C04)** installe les **agrégats** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`,

`ROUND`) qui appellent `GROUP BY` (C05).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Quelle est la position de `ORDER BY` dans l'ordre d'exécution ?

2. Pourquoi LIMIT sans ORDER BY est-il « non déterministe » ?
3. Quelle est la différence entre tri par alias et tri par position ?
4. Que fait NULLS LAST ?
5. Pourquoi la pagination OFFSET est-elle lente sur les grosses tables ?

Réponses :

1. 7^e position : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
2. Parce que sans ORDER BY, l'ordre des lignes est arbitraire (celui du stockage interne). Deux exécutions successives peuvent retourner des lignes différentes.
3. Le tri par alias utilise le nom déclaré dans le SELECT (par exemple ORDER BY ca_total DESC). Le tri par position utilise le numéro de la colonne (par exemple ORDER BY 2 DESC, qui trie la 2^e colonne du SELECT). Le tri par alias est lisible et robuste ; le tri par position est fragile (ajouter une colonne au SELECT change le sens).
4. NULLS LAST place les valeurs NULL à la fin du tri. Le défaut en DuckDB/PostgreSQL est déjà NULLS LAST en ASC, mais préciser est une bonne pratique pour la portabilité.
5. Parce que OFFSET m parcourt les m + n premières lignes puis en jette m. Sur 1 million de lignes, un OFFSET proche de la fin parcourt la quasi-totalité des lignes avant d'en afficher 10.

Module M07.C04 — Agréger : COUNT, SUM, AVG, MIN, MAX, ROUND

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C02 (WHERE) ; M07.C03 (ORDER BY, LIMIT).

L'idée du chapitre. Jusqu'ici, nos requêtes renvoyaient des **lignes** : une vente, un client, un produit. C04 change d'échelle : **réduire N lignes à un chiffre**. « Quel est le chiffre d'affaires total ? » « Quel est le panier moyen ? » « Quel est le plus gros montant ? » Ce sont des questions d'**agrégation** : COUNT, SUM, AVG, MIN, MAX, complétées par ROUND. Le chapitre couvre les cinq fonctions d'agrégat, le comptage de valeurs distinctes, le comportement particulier des NULL (ignorés par tous les agrégats sauf COUNT(*)), et la règle d'or qui prépare C05 : **une colonne non agrégée dans SELECT exige GROUP BY**. Le fil rouge est la base `commercial.duckdb` : on y mesure un CA brut de **7 908 259 731 FCFA**, un panier moyen de **158 140 FCFA**, et un montant minimum de **763 FCFA**.

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes (CA brut 7 908 259 731 FCFA), 1 200 clients, 5 modes de paiement (Espèces : 20 175 ventes ; Carte Bancaire : 12 366 ; Mobile Money : 12 322 ; Credit 30j : 2 593 ; Virement : 2 552), et des montants TTC compris entre **763** et **594 363 FCFA**.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **compter** les lignes avec COUNT(*), les valeurs non nulles avec COUNT(colonne), les valeurs distinctes avec COUNT(DISTINCT colonne) ;
- **totaliser** avec SUM, **moyenner** avec AVG, extraire le **minimum** et le **maximum** avec MIN / MAX ;
- **arrondir** un résultat avec ROUND(valeur, nb_decimales) ;
- **expliquer pourquoi** les NULL disparaissent des agrégats (sauf COUNT(*)) ;
- **combinaison** plusieurs agrégats dans une même requête pour produire une ligne de synthèse ;
- **identifier l'erreur** « colonne non agrégée sans GROUP BY » et la résoudre (anticipation de C05).

2. Pourquoi cette notion est importante

- Presque toute question métier est une question d'agrégat : « combien de ventes ? », « quel CA ? », « quel est le panier moyen ? », « quelle est la plus grosse facture ? ». C'est le passage de **l'exploration de données** (C01-C03) à **la mesure** (C04-C05).
- Le **comportement des NULL** dans les agrégats est l'une des sources d'erreur les plus classiques : AVG qui sous-estime, COUNT qui diverge de COUNT(*), SUM qui renvoie NULL sur une colonne tout entière vide.
- La **règle colonne non agrégée / GROUP BY** est la frontière entre C04 (une ligne de synthèse sur toute la table) et C05 (une ligne de synthèse par groupe). La comprendre maintenant évite les erreurs les plus fréquentes du module.

3. Explication simple — l'agrégat est une pliuse de lignes

Imaginez un tas de 50 008 tickets de caisse sur un bureau. La question « quel est le montant minimum ? » ne demande pas de lire les tickets un par un et de les recopier : on **scanne** le tas, on retient un seul ticket — le plus petit. La question « quel est le total ? » demande d'**additionner** — mais, encore une fois, un seul chiffre sort du tas.

C'est exactement ce que fait un agrégat : il **consomme N lignes** et **produit 1 valeur**. La requête

```
SELECT SUM(montant_ttc) AS ca_total
FROM vente;
```

lit les 50 008 lignes de `vente`, additionne la colonne `montant_ttc`, et renvoie **une seule ligne contenant un seul chiffre** : 7 908 259 731 FCFA (arrondi au franc). Le mot-clé `SELECT` n'a pas changé de syntaxe ; c'est le **comportement** qui change : dès qu'une fonction d'agrégat apparaît, le résultat est **réduit à une ligne** (sauf `GROUP BY`, C05).

Définition. Une **fonction d'agrégat** — aggregate function — est une fonction qui prend un ensemble de valeurs (les lignes d'un groupe) en entrée et produit une **unique valeur** en sortie. Les cinq agrégats standard sont `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. Ils s'écrivent `fonction(arg)`, par exemple `SUM(montant_ttc)`.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
agrégat	fonction qui consomme N valeurs et produit 1 valeur.	l'utiliser sur une colonne texte avec <code>SUM</code> (erreur de type).
COUNT(*)	nombre de lignes (tout contenu, <code>NULL</code> inclus).	le confondre avec <code>COUNT(colonne)</code> .
COUNT(DISTINCT ...)	nombre de valeurs différentes d'une colonne.	croire qu'il compte les lignes (il compte les valeurs uniques).
SUM	somme des valeurs non nulles.	attendre 0 sur une colonne 100 % <code>NULL</code> (on obtient <code>NULL</code>).
AVG	moyenne arithmétique = <code>SUM</code> / <code>COUNT(non-NULL)</code> .	croire que les <code>NULL</code> comptent dans le dénominateur (ils ne comptent pas).
MIN / MAX	valeur minimale / maximale (montants, dates, textes).	les appliquer sur un texte en oubliant l'ordre lexicographique.
ROUND(x, n)	arrondi de x à n décimales (défaut 0).	arrondir trop tôt (fausse les totaux en aval).
ligne de synthèse	résultat unique d'agrégats sans <code>GROUP BY</code> .	y mélanger une colonne non agrégée (erreur <code>GROUP BY</code>).
comptage conditionnel	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> : compter « les X qui ».	l'écrire en <code>WHERE</code> quand on veut plusieurs mesures à la fois.
null-skipping	les <code>NULL</code> sont ignorés par les agrégats (sauf <code>COUNT(*)</code>).	oublier que <code>SUM</code> renvoie <code>NULL</code> si tout est <code>NULL</code> .

5. Cours approfondi

5.1 COUNT(*) : compter les lignes

```
SELECT COUNT(*) AS nb_ventes
FROM vente;
```

COUNT(*) compte **toutes les lignes** du résultat, qu'elles contiennent des NULL ou non. Sur la base de travail : **50 008** lignes. C'est l'agrégat de vérification de premier rang : il confirme qu'une table « contient bien » le volume attendu.

5.2 COUNT(colonne) : compter les valeurs non nulles

```
SELECT COUNT(montant_ttc) AS nb_montants
FROM vente;
```

COUNT(colonne) ne compte **que les valeurs non nulles** de la colonne. Sur la base de travail, montant_ttc n'a aucun NULL : le résultat est **50 008**, identique à COUNT(*). La différence n'apparaît que quand des NULL existent : c'est alors un indicateur de qualité de données (« combien de lignes ont un montant ? »).

5.3 COUNT(DISTINCT colonne) : compter les valeurs différentes

```
SELECT COUNT(DISTINCT id_client) AS nb_clients_actifs
FROM vente;
```

COUNT(DISTINCT ...) compte les **valeurs différentes**. Sur les 50 008 ventes, il y a **1 200** clients distincts — chaque client a au moins une vente (la plupart, beaucoup). Cette écriture est le geste standard pour répondre à « combien de clients ont réellement acheté ? ».

Définition. Le **comptage distinct** — distinct count — compte les valeurs uniques d'une colonne au lieu de toutes les lignes. COUNT(DISTINCT id_client) répond à « combien de clients différents ? » ; il ignore les répétitions. C'est plus coûteux qu'un COUNT(*) car le moteur doit mémoriser (ou trier) les valeurs déjà vues.

5.4 SUM et AVG : totaliser et moyenner

```
SELECT SUM(montant_ttc) AS ca_total,
       AVG(montant_ttc) AS panier_moyen
FROM vente;
```

Le total s'élève à **7 908 259 731 FCFA** (arrondi au franc) — c'est le **CA brut**, retours inclus (piège P2, rappelé en C02). Le panier moyen vaut **158 140 FCFA** (arrondi). AVG est défini par le moteur comme SUM(colonne) / COUNT(colonne) : les NULL sont exclus du dénominateur comme du numérateur, ce qui évite la sous-estimation.

Définition. Le **panier moyen** — average order value (AOV) — est le montant moyen d'une vente, calculé par AVG(montant_ttc). Sur la base de travail : **158 140 FCFA**. Il est l'un des indicateurs commerciaux de premier rang, car il mesure la valeur d'une transaction type sans dépendre du volume de ventes.

5.5 MIN et MAX : les bornes

```
SELECT MIN(montant_ttc) AS montant_min,
       MAX(montant_ttc) AS montant_max
FROM vente;
```

Le montant minimum est de **763 FCFA** (arrondi) et le maximum de **594 363 FCFA** (plafond du générateur, mesuré en C03). MIN et MAX fonctionnent aussi sur les dates (MIN(date_vente) = première vente) et les textes (ordre lexicographique) — pas seulement sur les montants.

5.6 ROUND : arrondir pour présenter

Les agrégats produisent souvent des décimales inutiles en FCFA. ROUND sert à **présenter** le chiffre :

```
SELECT ROUND(SUM(montant_ttc))      AS ca_total,
       ROUND(AVG(montant_ttc))      AS panier_moyen,
       ROUND(AVG(montant_ttc), 2)   AS panier_precis
FROM vente;
```

ca_total = 7 908 259 731 ; panier_moyen = 158 140 ; panier_precis conserve deux décimales. Règle : **ROUND(valeur, n) arrondit à n décimales** ; sans n, l'arrondi est à l'entier.

5.7 Les NULL et les agrégats : la règle unique

Règle. Tous les agrégats **ignorent les NULL de leur colonne**, à une exception : COUNT(*), qui compte les lignes (pas les valeurs).

Agrégat	NULL dans la colonne	Colonne 100 % NULL
COUNT(*)	ignoré (compte les lignes)	nombre de lignes
COUNT(colonne)	ignoré (ne compte que non-NULL)	0
SUM(colonne)	ignoré (non compté)	NULL
AVG(colonne)	ignoré (non compté)	NULL
MIN / MAX	ignoré (non comparé)	NULL

Sur la base de travail, **aucune colonne n'a de NULL** (le générateur les exclut, comme noté en C03 §5.6) : SUM, AVG, MIN, MAX donnent donc des chiffres partout. La règle reste essentielle dès qu'on importe des données réelles — et c'est l'objet de la démonstration qui suit.

Définition. L'ignorance des NULL — null-skipping — est le comportement par défaut des agrégats SUM, AVG, MIN, MAX (et COUNT(colonne)) : les valeurs NULL ne participent ni à la somme, ni au dénombrement, ni à la comparaison. Seule l'écriture COUNT(*) dénombre les lignes quelle que soit leur contenu.

Attention. SUM et AVG renvoient NULL (pas 0) sur une colonne entièrement NULL. Si votre rapport affiche NULL là où il devrait afficher 0, c'est ce cas. La parade : COALESCE(SUM(x), 0) — fonction de M07.C06.

5.8 Position dans l'ordre d'exécution

Rappel de C01/C03 : FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Sans GROUP BY (ce chapitre), les agrégats du SELECT agissent sur **l'ensemble** des lignes survivantes après WHERE :

```
SELECT SUM(montant_ttc) AS ca_2025
FROM vente
WHERE date_vente >= '2025-01-01'
      AND date_vente < '2026-01-01';
```

WHERE filtre d'abord (le filtre est appliqué **avant** l'agrégation) : le CA 2025 ne tient compte que des ventes de 2025. C'est la différence fondamentale avec HAVING (C05) : WHERE filtre **les lignes**, HAVING filtre **les groupes**.

5.9 Plusieurs agrégats dans une même requête

Rien n'interdit de combiner les agrégats dans un seul SELECT — le résultat reste **une seule ligne** :

```
SELECT COUNT(*)           AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc))  AS ca_total,
       ROUND(AVG(montant_ttc))  AS panier_moyen,
       ROUND(MIN(montant_ttc))  AS montant_min,
       ROUND(MAX(montant_ttc))  AS montant_max
FROM vente;
```

nb_ventes	nb_clients	ca_total	panier_moyen	montant_min	montant_max
50 008	1 200	7 908 259 731	158 140	763	594 363

Une ligne, six chiffres : c'est le **format de synthèse** d'un rapport exécutif.

5.10 Agrégat conditionnel : compter « les X qui »

Question : « combien de ventes par Espèces, et combien par Carte Bancaire, **en une seule ligne** ? » Le filtre WHERE ne suffit pas (il ne permet qu'un filtre unique) ; on utilise une expression conditionnelle dans l'agrégat :

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS nb_especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS nb_cartes
FROM vente;
```

nb_especes = **20 175**, nb_cartes = **12 366**. Ce motif — agréguer une expression CASE — est un outil central de C05 (comptages conditionnels).

À retenir. Le motif SUM(CASE WHEN ... THEN 1 ELSE 0 END) transforme un comptage conditionnel en **agrégat ordinaire** : il s'additionne, se combine avec d'autres agrégats, et ne produit toujours qu'une ligne. C'est l'outil des « tableaux de bord » (une ligne, plusieurs colonnes de mesures).

5.11 Ce que ce chapitre ne couvre pas

- GROUP BY (une ligne de synthèse **par groupe**) : C05.
- HAVING (filtre sur les groupes) : C05.
- COALESCE, NULLIF (traiter les NULL avant l'agrégation) : C06.
- Les fonctions de fenêtre (SUM() OVER (PARTITION BY ...)) : M11.

6. Exemple concret — la requête qui répond à une question métier

La directrice financière demande : « Quel est notre CA brut, et quel est le panier moyen sur l'ensemble de l'historique ? »

Étape 1 — Écrire la requête.

```
SELECT ROUND(SUM(montant_ttc)) AS ca_brut,
       ROUND(AVG(montant_ttc)) AS panier_moyen,
       COUNT(*)                AS nb_ventes
FROM vente;
```

Étape 2 — Prédire la sortie. FROM vente charge 50 008 lignes ; aucun WHERE ; pas de GROUP BY ; SUM, AVG, COUNT réduisent le tout à **une ligne de synthèse**.

Étape 3 — Exécuter. Sortie : ca_brut = 7 908 259 731, panier_moyen = 158 140, nb_ventes = 50 008.

Étape 4 — Comparer. Trois chiffres, une ligne, tous cohérents : $7\,908\,259\,731 / 50\,008 \approx 158\,140$. Si le panier moyen ne redonnait pas le CA divisé par le nombre de ventes, il faudrait revoir la requête (filtre oublié, table erronée).

Conseil professionnel. Vérifiez **toujours la cohérence croisée** des agrégats d'une ligne de synthèse : $CA / \text{nombre de ventes} \approx \text{panier moyen}$, $MAX \geq AVG \geq MIN$. Une incohérence (par exemple $MIN > MAX$) signale une erreur de requête ou de type — elle se repère en 5 secondes, bien avant le rapport.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Combien de ventes au total ?

```
SELECT COUNT(*) AS nb_ventes
FROM vente;
```

→ **50 008** (dont 8 doublons et 200 retours inclus, cf. 6_pieges.md).

7.2 Question 2 — Quel est le CA brut total ?

```
SELECT ROUND(SUM(montant_ttc)) AS ca_brut
FROM vente;
```

→ **7 908 259 731 FCFA**.

7.3 Question 3 — Quel est le panier moyen ?

```
SELECT ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente;
```

→ **158 140 FCFA**.

7.4 Question 4 — Bornes des montants

```
SELECT ROUND(MIN(montant_ttc)) AS min_m,
       ROUND(MAX(montant_ttc)) AS max_m
FROM vente;
```

→ 763 / 594 363 FCFA (le maximum est le plafond du générateur, mesuré en C03).

7.5 Question 5 — Répartition par mode de paiement, en une ligne

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS carte,
       SUM(CASE WHEN id_mode = 3 THEN 1 ELSE 0 END) AS mobile,
       SUM(CASE WHEN id_mode = 4 THEN 1 ELSE 0 END) AS credit_30j,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS virement
FROM vente;
```

→ 20 175 / 12 366 / 12 322 / 2 593 / 2 552. Les Espèces représentent plus de 40 % des ventes ; le crédit 30j et le virement sont marginaux (5 % chacun).

7.6 Question 6 — CA par mode de paiement, en une ligne

```
SELECT ROUND(SUM(CASE WHEN id_mode = 1 THEN montant_ttc ELSE 0 END)) AS ca_especes,
       ROUND(SUM(CASE WHEN id_mode = 2 THEN montant_ttc ELSE 0 END)) AS ca_carte
FROM vente;
```

→ 3 184 889 570 FCFA (Especes) / 1 944 001 846 FCFA (Carte). Le même motif CASE s'applique au **montant** au lieu du comptage : c'est la forme générale du « tableau de bord en une ligne ».

À retenir. Le motif `SUM(CASE WHEN ... THEN montant ELSE 0 END)` est la forme générale du **tableau de bord en une ligne** : chaque colonne est une mesure conditionnelle. C'est exactement ce que `GROUP BY` (C05) généralisera en plusieurs lignes, une par groupe.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	<code>SELECT id_client, COUNT(*) FROM vente;</code> sans <code>GROUP BY</code>	erreur « must appear in the GROUP BY clause »	<code>GROUP BY id_client</code> (C05), ou retirer la colonne non agrégée.
2	Attendre 0 de <code>SUM</code> sur une colonne vide	résultat NULL dans le rapport	<code>COALESCE(SUM(x), 0)</code> .
3	Confondre <code>COUNT(*)</code> et <code>COUNT(colonne)</code>	décompte inexplicé	<code>COUNT(*)</code> = lignes ; <code>COUNT(col)</code> = valeurs non nulles.
4	Arrondir trop tôt	totaux incohérents en aval	<code>ROUND</code> uniquement dans le dernier <code>SELECT</code> affiché.
5	<code>SUM</code> sur une colonne texte	erreur de type	<code>SUM/AVG</code> sur des colonnes numériques uniquement.
6	<code>COUNT(DISTINCT a, b)</code> sur 2 colonnes	syntaxe refusée en DuckDB/PostgreSQL	<code>COUNT(DISTINCT a b)</code> ou table temporaire (C06+).

Attention. L'erreur 1 est **la plus fréquente** du module : on demande « une valeur par client » sans dire comment grouper. Le message du moteur est explicite (« must appear in the GROUP BY clause or be used in an aggregate function ») : dès qu'une colonne non agrégée est dans `SELECT`, le `GROUP BY` est obligatoire (C05).

9. Bonnes pratiques professionnelles

- **Un agrégat, un alias** : `SUM(montant_ttc) AS ca_total` — jamais de colonne sans nom dans un rapport.
- **ROUND à la présentation** : précision pleine en amont, arrondi au dernier `SELECT`.
- **Vérifier la cohérence croisée** : `CA / nb_ventes ≈ panier moyen` ; `MIN ≤ AVG ≤ MAX`.
- **COUNT(*) en contrôle** : avant tout agrégat, vérifier le volume de lignes attendues (sur vente : 50 008).
- **Documenter les NULL** : une colonne avec `NULL` doit être annotée dans le dictionnaire de données (M06), sinon `SUM/AVG` surprennent.
- **Commenter les comptages conditionnels** : « Espèces = id_mode 1, cf. mode_paiement ».

10. Exercice guidé — la synthèse mensuelle (15 min, /10)

Énoncé. Le chef de produit demande une **ligne de synthèse** de l'activité : nombre de ventes, nombre de clients actifs, CA total, panier moyen, montant minimum, montant maximum — le tout arrondi au franc, en une seule requête.

Grille de correction (/10).

Critère	Points
6 agrégats bien choisis (<code>COUNT(*)</code> , <code>COUNT(DISTINCT id_client)</code> , <code>SUM</code> , <code>AVG</code> , <code>MIN</code> , <code>MAX</code>)	3
6 alias lisibles	1
5 <code>ROUND</code> sur les montants	1
Une seule ligne produite (pas de colonne non agrégée)	2
<code>FROM vente</code> correct	1
Valeurs conformes (50 008 / 1 200 / 7 908 259 731 / 158 140 / 763 / 594 363)	2

Corrigé.

```
SELECT COUNT(*)           AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc))   AS ca_total,
       ROUND(AVG(montant_ttc))   AS panier_moyen,
       ROUND(MIN(montant_ttc))   AS min_m,
       ROUND(MAX(montant_ttc))   AS max_m
FROM vente;
```

→ 50 008 / 1 200 / 7 908 259 731 / 158 140 / 763 / 594 363.

11. Exercices autonomes

Exercice 1 — CA total, arrondi

Écrire la requête qui renvoie le CA brut total de `vente`, arrondi au franc.

Exercice 2 — Panier moyen à deux décimales

Exercice 3 — Nombre de clients distincts dans `vente`

Exercice 4 — Bornes des montants (minimum et maximum arrondis)

Exercice 5 — `COUNT(*)` VS `COUNT(montant_ttc)`

Expliquer pourquoi les deux renvoient 50 008 sur cette base, et dans quel cas ils divergeraient.

Exercice 6 — Comptage conditionnel par mode

Compter, dans une même requête, le nombre de ventes Espèces (id_mode 1) et Virement (id_mode 5).

Exercice 7 — CA par deux modes, en une ligne

Produire, en une ligne : le CA total Espèces et le CA total Carte Bancaire (arrondis).

Exercice 8 — Diagnostic d'erreur

Que renvoie ce `SELECT`, et pourquoi ?

```
SELECT SUM(plafond_credit) FROM client;
```

12. Correction détaillée

Exercice 1.

```
SELECT ROUND(SUM(montant_ttc)) AS ca_total  
FROM vente;
```

→ 7 908 259 731 FCFA.

Exercice 2.

```
SELECT ROUND(AVG(montant_ttc), 2) AS panier_moyen  
FROM vente;
```

→ le panier moyen avec deux décimales (arrondi de la moyenne exacte).

Exercice 3.

```
SELECT COUNT(DISTINCT id_client) AS nb_clients  
FROM vente;
```

→ 1 200.

Exercice 4.

```
SELECT ROUND(MIN(montant_ttc)) AS min_m,  
       ROUND(MAX(montant_ttc)) AS max_m  
FROM vente;
```

→ 763 / 594 363 FCFA.

Exercice 5. Sur `vente`, `montant_ttc` n'a aucun `NULL` : les deux comptent donc les mêmes lignes (50 008). Ils divergeraient dès qu'une ligne de `vente` aurait `montant_ttc = NULL` : `COUNT(*)` la compterait, `COUNT(montant_ttc)` non.

Exercice 6.

```
SELECT SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS nb_especes,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS nb_virement
FROM vente;
```

→ 20 175 / 2 552.

Exercice 7.

```
SELECT ROUND(SUM(CASE WHEN id_mode = 1 THEN montant_ttc ELSE 0 END)) AS ca_especes,
       ROUND(SUM(CASE WHEN id_mode = 2 THEN montant_ttc ELSE 0 END)) AS ca_carte
FROM vente;
```

→ 3 184 889 570 / 1 944 001 846 FCFA.

Exercice 8. Sur le socle M07, les 1 200 clients ont tous un plafond (le générateur exclut les `NULL`) : la somme additionne donc les 1 200 plafonds. Sur des données réelles contenant des `NULL`, les clients sans plafond seraient **ignorés** par `SUM` ; et si aucun client n'avait de plafond, le résultat serait `NULL` — pas 0. Pour afficher 0, écrire `COALESCE(SUM(plafond_credit), 0)`.

13. Mini-projet — « La fiche de synthèse commerciale » (30 min)

Produire la **fiche de synthèse** de la base commerciale : une requête unique qui renvoie, en une ligne, les 8 indicateurs ci-dessous. Vérifier chaque chiffre à la main (relever les valeurs dans un tableur puis comparer).

Indicateur	Valeur attendue (arrondie)
Nombre de ventes	50 008
Clients actifs	1 200
CA brut total (FCFA)	7 908 259 731
Panier moyen (FCFA)	158 140
Montant minimum (FCFA)	763
Montant maximum (FCFA)	594 363
Ventes Espèces	20 175
Ventes Virement	2 552

Corrigé.

```
SELECT COUNT(*) AS nb_ventes,
       COUNT(DISTINCT id_client) AS nb_clients,
       ROUND(SUM(montant_ttc)) AS ca_total,
       ROUND(AVG(montant_ttc)) AS panier_moyen,
       ROUND(MIN(montant_ttc)) AS min_m,
       ROUND(MAX(montant_ttc)) AS max_m,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS ventes_especes,
       SUM(CASE WHEN id_mode = 5 THEN 1 ELSE 0 END) AS ventes_virement
FROM vente;
```

Critères de validation. 8 colonnes, 1 ligne ; les 8 valeurs ci-dessus ; aucun avertissement `GROUP BY` (toutes les colonnes du `SELECT` sont agrégées).

Dans les faits. PostgreSQL accepte la même syntaxe avec le même résultat ; MySQL, en mode `ONLY_FULL_GROUP_BY` activé par défaut depuis la version 5.7, renvoie la même erreur qu'ici si une colonne non agrégée est présente, mais en mode permissif (ancien défaut) il **devine** une valeur arbitraire — la requête passe, et le chiffre est faux sans aucun signal. C'est pourquoi le mode permissif MySQL est à éviter en production.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 6 outils de mesure : `COUNT` (lignes, valeurs, distinct), `SUM` (total), `AVG` (moyenne), `MIN/MAX` (bornes), `ROUND` (présentation), plus le motif `SUM(CASE WHEN ...)` pour les comptages conditionnels. La règle d'or — colonne non agrégée $\Rightarrow$ `GROUP BY` — est la porte d'entrée de C05.

Outil	Usage	Syntaxe	Piège
<code>COUNT(*)</code>	nombre de lignes	<code>COUNT(*)</code>	confondre avec <code>COUNT(col)</code>
<code>COUNT(col)</code>	valeurs non nulles	<code>COUNT(montant_ttc)</code>	diverger de <code>COUNT(*)</code> si <code>NULL</code>
<code>COUNT(DISTINCT col)</code>	valeurs uniques	<code>COUNT(DISTINCT id_client)</code>	plus coûteux qu'un <code>COUNT(*)</code>
<code>SUM / AVG</code>	total / moyenne	<code>SUM(montant_ttc)</code>	renvoyer <code>NULL</code> si colonne 100 % <code>NULL</code>
<code>MIN / MAX</code>	bornes	<code>MAX(date_vente)</code>	ordre lexicographique sur texte
<code>ROUND(x, n)</code>	arrondi	<code>ROUND(AVG(montant_ttc), 2)</code>	arrondir trop tôt
<code>SUM(CASE ...)</code>	comptage / somme conditionnelle	<code>SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)</code>	<code>ELSE 0</code> obligatoire

15. Résumé du chapitre

- **Un agrégat consomme N lignes et produit 1 valeur** : `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, plus `ROUND` pour la présentation.
- `COUNT(*)` compte les lignes ; `COUNT(colonne)` les valeurs non nulles ; `COUNT(DISTINCT colonne)` les valeurs différentes.
- **Les NULL sont ignorés** par `SUM`, `AVG`, `MIN`, `MAX`, `COUNT(colonne)` ; sur une colonne 100 % `NULL`, `SUM/AVG/MIN/MAX` renvoient `NULL` (pas 0) — parade : `COALESCE(agrégat, 0)`.
- **WHERE filtre les lignes avant l'agrégation** ; `HAVING` (C05) filtrera les groupes.
- **Colonne non agrégée dans SELECT sans GROUP BY = erreur** : c'est la règle d'or qui sépare C04 (synthèse globale) de C05 (synthèse par groupe).
- Le motif `SUM(CASE WHEN ...)` produit les « tableaux de bord en une ligne » : une mesure conditionnelle par colonne.

16. À retenir

- **Un agrégat = N lignes $\rightarrow$ 1 valeur.** Toujours un alias par agrégat.
- `COUNT(*)` $\neq$ `COUNT(colonne)` dès qu'un `NULL` existe.
- **NULL ignorés** : `SUM/AVG` renvoient `NULL` sur une colonne vide — `COALESCE` pour forcer 0.

- **ROUND à la présentation** : précision pleine en amont, arrondi au dernier SELECT.
- **Colonne non agrégée sans GROUP BY = erreur** : la règle d'or, porte d'entrée de C05.

À retenir. $AVG(colonne) = SUM(colonne) / COUNT(colonne)$: le dénominateur exclut les NULL. C'est la définition même du panier moyen — et la raison pour laquelle une moyenne « qui baisse sans raison » est souvent une entrée de NULL en moins.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Quelle est la différence entre `COUNT(*)` et `COUNT(colonne)` ?
2. Que renvoie `SUM(colonne)` si toutes les valeurs de `colonne` sont NULL ?
3. `AVG` est-il défini comment par le moteur ?
4. Pourquoi `SELECT id_client, COUNT(*) FROM vente;` est-il une erreur ?
5. À quelle position de l'ordre d'exécution sont appliqués les agrégats du `SELECT` sans `GROUP BY` ?

Réponses :

1. `COUNT(*)` dénombre **les lignes** (tout contenu, NULL inclus) ; `COUNT(colonne)` dénombre **les valeurs non nulles** de la colonne. Les deux divergent dès qu'un NULL existe dans la colonne.
2. NULL — pas 0. L'agrégat n'a rien à sommer : l'absence de valeur s'exprime par NULL. La parade est `COALESCE(SUM(colonne), 0)`.
3. $AVG(colonne) = SUM(colonne) / COUNT(colonne)$ — le dénominateur exclut les NULL, donc la moyenne est calculée sur les seules valeurs présentes.
4. Parce qu'`id_client` n'est **pas agrégé** et n'est **pas groupé** : le moteur ne sait pas quelle valeur de `id_client` retourner pour la ligne de synthèse. Il faut `GROUP BY id_client` (C05).
5. Après le `WHERE` (le filtre de lignes) : l'agrégat sans `GROUP BY` consomme l'ensemble des lignes survivantes à ce stade, avant le `SELECT` et le `DISTINCT`.

Module M07.C05 — Résumer par groupe : GROUP BY, HAVING, comptages conditionnels

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C02 (WHERE) ; M07.C03 (ORDER BY, LIMIT) ; M07.C04 (agrégats).

L'idée du chapitre. C04 a réduit **toute la table** à une ligne de synthèse. C05 va plus loin : **une ligne de synthèse par groupe**. « Le CA de chaque magasin. » « Le nombre de ventes de chaque rayon. » « Le panier moyen par magasin et par année. » C'est **GROUP BY** — la clause la plus utile de l'analyse métier. Le chapitre couvre la syntaxe de base, le regroupement par **clé** (et non par libellé — le piège du rayon doublé), le regroupement multi-colonnes, **HAVING** (le filtre qui s'applique aux groupes, pas aux lignes), la distinction **WHERE / HAVING**, et les **comptages conditionnels** (**SUM(CASE WHEN ...)** combiné à **GROUP BY**) qui produisent les tableaux de bord croisés. Le fil rouge est la base `commercial.duckdb` : on y mesure le CA de chacun des **5 magasins** (entre 1 564 937 883 et 1 596 813 264 FCFA) et de chacune des **8 catégories** (entre 699 533 080 et 1 236 006 465 FCFA).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes répartis sur 5 magasins (de 9 873 à 10 099 ventes chacun), 8 catégories (de 4 764 à 7 166 ventes chacune), 2 années (2025 : 24 920 ventes ; 2026 : 25 088), et 5 modes de paiement. Le CA total reste 7 908 259 731 FCFA.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **regrouper** un résultat par une colonne avec **GROUP BY** et produire une ligne de synthèse par groupe ;
- **regrouper par clé** (id unique) et comprendre pourquoi regrouper par **libellé** est dangereux quand le libellé est doublé ;
- **regrouper sur plusieurs colonnes** (**GROUP BY a, b**) pour des coupes croisées ;
- **filtrer les groupes** avec **HAVING** (et distinguer **WHERE** de **HAVING**) ;
- **combiner** **ORDER BY** et **LIMIT** avec **GROUP BY** pour un « top N par groupe » ;
- **compter conditionnellement** avec **SUM(CASE WHEN ... THEN 1 ELSE 0 END)** dans un **GROUP BY**.

2. Pourquoi cette notion est importante

- « Le CA par magasin », « les ventes par catégorie », « le panier moyen par ville et par année » : ce sont **les questions d'un tableau de bord**. Sans **GROUP BY**, on ne sort d'une table qu'une seule ligne de synthèse (C04) ou des lignes brutes (C01–C03).
- Le **regroupement par libellé doublé** est un piège silencieux : le total est juste, mais les lignes se fondent les unes dans les autres sans avertissement. Le socle M07 en contient un de démonstration : la colonne `rayon` de `categorie` porte deux fois « Alimentaire », deux fois « Bricolage », deux fois « Jardinage » (les `sous_categorie` distinguent).
- La frontière **WHERE / HAVING** est l'un des points les plus demandés en entretien : **WHERE** filtre les **lignes avant** le regroupement ; **HAVING** filtre les **groupes après** l'agrégation.

3. Explication simple — GROUP BY est une plieuse par tiroirs

C04 : un tas de tickets, on scanne le tas, un seul chiffre sort. C05 : on met des **tiroirs** sur le bureau — un tiroir par magasin, un par rayon, un par année. Chaque ticket va dans son tiroir (selon sa valeur dans la colonne de regroupement). Puis on **applique la même plieuse à chaque tiroir** : un chiffre par tiroir.

Concrètement, `GROUP BY id_magasin` sur 50 008 ventes produit **5 lignes** (une par magasin), chaque ligne portant les agrégats calculés **sur les seules ventes de ce magasin**. Le total des 5 lignes de `SUM(montant_ttc)` redonne le CA global de C04 (7 908 259 731 FCFA) — c'est le **contrôle de cohérence** à faire systématiquement.

Définition. La clause `GROUP BY` — grouper par — subdivise les lignes du résultat en **groupes** selon les valeurs de la colonne (ou des colonnes) citées : les lignes qui partagent la même valeur forment un groupe, et chaque fonction d'agrégat du `SELECT` est calculée **séparément sur chaque groupe**. Le résultat contient une ligne par groupe.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
GROUP BY	une ligne de synthèse par groupe (par valeur distincte de la colonne).	oublier que chaque colonne du <code>SELECT</code> doit être agrégée ou dans le <code>GROUP BY</code> .
groupe	l'ensemble des lignes qui partagent la même valeur de la clé de regroupement.	confondre le nombre de groupes avec le nombre de lignes.
clé de regroupement	la colonne (ou les colonnes) du <code>GROUP BY</code> ; idéale : une clé unique (<code>id_*</code>).	regrouper par un libellé doublé (les groupes se fondent).
HAVING	filtre appliqué aux groupes , après les agrégats.	l'écrire dans <code>WHERE</code> (interdit : on ne filtre pas un agrégat là-dedans).
WHERE	filtre appliqué aux lignes , avant le regroupement.	mettre un agrégat dans <code>WHERE</code> (erreur d'exécution).
regroupement multi-colonnes	<code>GROUP BY a, b</code> : un groupe par couple (a, b).	croire que l'ordre des colonnes du <code>GROUP BY</code> change le total (il ne change que la granularité d'affichage).
comptage conditionnel	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> dans un <code>GROUP BY</code> .	oublier le <code>ELSE 0</code> (les <code>NULL</code> ne s'additionnent pas).
coupe croisée	regroupement sur 2 dimensions (magasin × année).	multiplier les colonnes sans comprendre que chaque ligne = une combinaison.

5. Cours approfondi

5.1 La syntaxe de base

```
SELECT id_magasin,
       COUNT(*)      AS nb_ventes,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
ORDER BY ca DESC;
```

id_magasin	nb_ventes	ca
4	10 061	1 596 813 264

id_magasin	nb_ventes	ca
1	10 099	1 595 223 423
2	10 033	1 578 987 708
5	9 942	1 572 297 452
3	9 873	1 564 937 883

Cinq groupes, cinq lignes. Chaque `nb_ventes` et `ca` est calculé **sur les seules ventes du magasin**. Le total des 5 `nb_ventes` redonne 50 008 ; le total des 5 `ca` redonne 7 908 259 731. Le magasin 4 (Bobo Sarfalao) est en tête, mais l'écart entre le 1^e et le 5^e est faible (moins de 2 %).

Règle de syntaxe : **toute colonne du SELECT doit être soit agrégée, soit dans le GROUP BY**. `id_magasin` est dans le GROUP BY ; `COUNT(*)` et `SUM(montant_ttc)` sont agrégés. C'est la règle d'or de C04, généralisée.

5.2 Regrouper par clé, pas par libellé

Le socle M07 contient 8 catégories, mais seulement **5 libellés** de `rayon` : « Alimentaire » apparaît deux fois (Boissons et Conserves), « Bricolage » deux fois (Outillage et quincaillerie), « Jardinage » deux fois (Plantes et Outillage).

Regrouper par `c.rayon` (le libellé) **fond les deux catégories** qui partagent le même rayon :

```
SELECT c.rayon, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon
ORDER BY nb_ventes DESC;
```

rayon	nb_ventes
Bricolage	12 418
Jardinage	12 037
Alimentaire	11 930
Hygiene	6 825
Decoration	6 798

Regrouper par `c.id_categorie` (la clé) garde **8 groupes distincts**. Le choix entre les deux est un **choix métier** (vu « par rayon » ou vu « par catégorie »), pas une coïncidence : si vous voulez 8 lignes, regroupez par la clé.

Définition. Une **clé de regroupement** — grouping key — est la colonne (ou la liste de colonnes) citée dans GROUP BY. Elle doit être **unique par groupe** : on regroupe idéalement par une clé (`id_categorie`) plutôt que par un libellé (`rayon`), car un libellé doublé fusionne silencieusement deux groupes différents.

Attention. Regrouper par un libellé doublé ne provoque **aucune erreur** : le moteur fusionne les groupes sans avertissement, et le total général reste juste. Le bug est donc silencieux — on ne le voit qu'en comptant les lignes du résultat (5 au lieu de 8). Vérifiez toujours le nombre de groupes attendus.

5.3 Regrouper sur plusieurs colonnes

GROUP BY `a, b` produit un groupe par **couple** (a, b). Coupe croisée magasin × année :

```
SELECT id_magasin,
       EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
WHERE id_magasin = 4
GROUP BY 1, 2
ORDER BY annee;
```

id_magasin	annee	nb_ventes	ca
4	2025	5 064	797 593 324
4	2026	4 997	799 219 940

Deux groupes (le couple magasin 4 × 2025, le couple magasin 4 × 2026). Sur les 5 magasins et 2 années, la requête sans `WHERE` produirait **10 groupes**.

5.4 HAVING — filtrer les groupes

`WHERE` ne voit que des lignes ; `HAVING` voit des **groupes déjà agrégés**. Question : « quelles catégories ont plus de 6 500 ventes ? »

```
SELECT c.id_categorie, c.rayon, c.sous_categorie, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
HAVING COUNT(*) > 6500
ORDER BY nb_ventes DESC;
```

id_categorie	rayon	sous_categorie	nb_ventes
1	Alimentaire	Boissons	7 166
4	Bricolage	Quincaillerie	7 037
8	Hygiene	Cosmétique	6 825
7	Decoration	Interieur	6 798

Quatre groupes passent le filtre. Le `HAVING` peut aussi s'appliquer à `SUM`, `AVG`, `MIN`, `MAX` — tout agrégat :

```
SELECT id_magasin, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
HAVING ROUND(SUM(montant_ttc)) > 1550000000
ORDER BY ca DESC;
```

Ici les **5 magasins** passent (le plus petit CA, 1 564 937 883 FCFA, dépasse le seuil) : le seuil est bas relativement à la répartition quasi uniforme du générateur.

Définition. La clause `HAVING` — ayant — est le filtre qui s'applique aux **groupes** après le calcul des agrégats. Elle s'écrit entre `GROUP BY` et `ORDER BY`, et seule elle permet de filtrer sur une valeur agrégée (`HAVING COUNT(*) > 10`). `WHERE` filtre les lignes **avant** le regroupement et ne peut pas référencer d'agrégat.

5.5 WHERE et HAVING ensemble

On peut (et on doit souvent) combiner les deux : `WHERE` réduit les lignes **avant** le regroupement, `HAVING` filtre les groupes **après** :

```
SELECT id_magasin, COUNT(*) AS nb_ventes_2026
FROM vente
WHERE EXTRACT(YEAR FROM date_vente) = 2026
GROUP BY id_magasin
HAVING COUNT(*) > 4900
ORDER BY nb_ventes_2026 DESC;
```

Ordre d'exécution (rappel C01/C04) : FROM → WHERE (lignes) → GROUP BY (groupes) → HAVING (groupes) → SELECT → ORDER BY → LIMIT.

À retenir. WHERE = filtre sur les **lignes** (avant agrégation, pas d'agrégat autorisé) ; HAVING = filtre sur les **groupes** (après agrégation, agrégats autorisés). Si vous filtrez sur un total, c'est HAVING.

5.6 Comptages conditionnels dans un GROUP BY

Le motif de C04 §5.10 (SUM(CASE WHEN ...)) combine parfaitement avec GROUP BY : chaque groupe porte plusieurs colonnes de mesures conditionnelles. Répartition Espèces / Carte **par magasin** :

```
SELECT id_magasin,
       COUNT(*) AS total,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

id_magasin	total	especes	cartes
1	10 099	4 109	2 502
2	10 033	4 043	2 532
3	9 873	3 973	2 414
4	10 061	4 093	2 469
5	9 942	3 957	2 449

Chaque ligne est un magasin ; chaque colonne especes / cartes est un sous-comptage. C'est le schéma des **tableaux de bord croisés** : lignes = dimension 1, colonnes = mesures conditionnelles.

Définition. Le **comptage conditionnel** — conditional count — est le motif SUM(CASE WHEN condition THEN 1 ELSE 0 END) : il convertit « compter les lignes qui vérifient la condition » en expression agrégable. Combiné à GROUP BY, il produit une colonne de comptage par condition, sur chaque groupe.

5.7 COUNT(DISTINCT ...) dans un groupe

Les agrégats de C04 fonctionnent aussi **par groupe**. Nombre de clients distincts qui ont acheté dans chaque magasin :

```
SELECT id_magasin, COUNT(DISTINCT id_client) AS nb_clients
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

→ 1 199, 1 199, 1 200, 1 200, 1 199. Quatre magasins servent 1 199 clients distincts, deux en servent 1 200 : la couverture est quasi totale (sur 1 200 clients au total).

5.8 Ce que ce chapitre ne couvre pas

- Les **fenêtres** (`SUM() OVER (PARTITION BY ...)`) : agrégat par groupe **sans** réduire les lignes) : M11.
- Les **sous-requêtes** et **CTE** (pré-agrégations nommées) : C08.
- L'optimisation des gros `GROUP BY` (index, plans d'exécution) : M11.

6. Exemple concret — la requête qui répond à une question métier

Le directeur des magasins demande : « Le CA de chaque magasin, le nombre de ventes, et le panier moyen — mais seulement les magasins qui font plus de 9 900 ventes. »

Étape 1 — Écrire la requête.

```
SELECT m.id_magasin,
       m.nom,
       COUNT(*)           AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.id_magasin, m.nom
HAVING COUNT(*) > 9900
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. `FROM vente` charge 50 008 lignes ; `JOIN magasin` apporte le nom ; `GROUP BY` forme 5 groupes ; `HAVING COUNT(*) > 9900` élimine les groupes de moins de 9 900 ventes. À l'œil, on prédit que 2 magasins sortent (les deux moins vendus) — il resterait 3 magasins.

Étape 3 — Exécuter. Quatre lignes : Bobo Sarfalao (10 061 ventes, 1 596 813 264 FCFA), Ouaga Centre (10 099, 1 595 223 423), Ouaga Patte d'Oie (10 033, 1 578 987 708), Koudougou (9 942, 1 572 297 452). Le panier moyen varie entre 157 379 et 158 713 FCFA.

Étape 4 — Comparer. La prédiction était fausse sur un point : Koudougou (9 942) passe le seuil de 9 900, seul Bobo Centre (9 873) sort. Le contrôle de cohérence (la somme des CA des 5 magasins = 7 908 259 731) reste le filet de sécurité.

Conseil professionnel. Prévoyez **toujours le cas limite** du seuil de `HAVING` : un groupe exactement **au seuil** passe ou non selon `>` ou `>=`. Avant de publier, vérifiez le nombre de groupes attendus — ici 5 au lieu de 4 — et ajustez le seuil ou l'opérateur. Une surprise de seuil est la première cause de « le rapport a 4 lignes au lieu de 5 ».

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — CA par magasin

```
SELECT id_magasin, COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY id_magasin
ORDER BY ca DESC;
```

→ 5 lignes : 1 596 813 264 / 1 595 223 423 / 1 578 987 708 / 1 572 297 452 / 1 564 937 883 FCFA.

7.2 Question 2 — CA par catégorie (la clé, pas le rayon)

```
SELECT c.id_categorie, c.rayon, c.sous_categorie,
       COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
ORDER BY ca DESC;
```

→ 8 lignes, du plus gros au plus petit : Decoration (1 236 006 465), Hygiene (1 163 845 193), Jardinage-Plantes (1 160 443 887), Alimentaire-Boissons (1 026 270 112), Jardinage-Outillage (950 960 303), Bricolage-Quincaillerie (939 911 483), Alimentaire-Conserves (731 289 207), Bricolage-Outillage (699 533 080) FCFA.

7.3 Question 3 — Panier moyen par magasin

```
SELECT id_magasin, ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente
GROUP BY id_magasin
ORDER BY panier_moyen DESC;
```

→ 158 713 (magasin 4) / 158 507 (3) / 158 147 (5) / 157 959 (1) / 157 379 (2) FCFA. L'écart total est de 1 334 FCFA, soit moins de 1 % : le panier moyen est quasi uniforme entre magasins.

7.4 Question 4 — Le CA par année

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 24 920 ventes, 3 925 215 671 FCFA ; 2026 : 25 088 ventes, 3 983 044 060 FCFA. Le 2026 dépasse le 2025 de 57 828 389 FCFA (+1,5 %).

7.5 Question 5 — Catégories actives (HAVING)

```
SELECT c.rayon, COUNT(*) AS nb_ventes
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon
HAVING COUNT(*) > 6500
ORDER BY nb_ventes DESC;
```

→ 4 rayons : Bricolage 12 418, Jardinage 12 037, Alimentaire 11 930, Hygiene 6 825, Decoration 6 798 — non, 5 lignes passent car Decoration (6 798) dépasse 6 500. Le HAVING garde 5 groupes sur 5 (seuil bas) ; le seuil 12 000 ne en garderait que 2 (Bricolage, Jardinage).

7.6 Question 6 — Espèces et cartes par magasin

```
SELECT id_magasin,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente
GROUP BY id_magasin
ORDER BY especes DESC;
```

→ Magasin 1 : 4 109 Espèces / 2 502 Carte ; magasin 4 : 4 093 / 2 469 ; magasin 2 : 4 043 / 2 532 ; magasin 3 : 3 973 / 2 414 ; magasin 5 : 3 957 / 2 449.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Colonne non agrégée et non groupée dans <code>SELECT</code>	erreur « must appear in the GROUP BY clause »	l'ajouter au <code>GROUP BY</code> ou l'agréger.
2	Filtrer un agrégat dans <code>WHERE</code>	erreur « WHERE clause cannot contain aggregates »	utiliser <code>HAVING</code> .
3	Regrouper par un libellé doublé	nombre de lignes inexplicé (5 au lieu de 8)	regrouper par la clé (<code>id_*</code>).
4	Oublier le <code>ELSE 0</code> dans un comptage conditionnel	colonnes de comptage trop faibles (les <code>NULL</code> ne s'additionnent pas)	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code> .
5	<code>ORDER BY</code> sur un alias inconnu du <code>GROUP BY</code>	ordre arbitraire ou erreur selon le SGBD	ordonner par l'agrégat ou la clé.
6	Attendre que <code>GROUP BY</code> supprime les doublons	doublons persistants	<code>GROUP BY</code> réduit par groupe ; pour supprimer des lignes, <code>DISTINCT (C01)</code> .

Attention. L'erreur 3 est la plus sournoise : **aucun message d'erreur**, un total juste, mais des groupes fusionnés. Le nombre de lignes du résultat est le premier contrôle : si vous attendez 8 catégories et vous en voyez 5, vérifiez la clé de regroupement.

Attention. `ORDER BY` dans une requête `GROUP BY` : on trie les **groupes**. `ORDER BY ca` (alias de l'agrégat) est accepté par DuckDB et PostgreSQL ; pour la portabilité maximale, répéter l'expression (`ORDER BY SUM(montant_ttc) DESC`) ou trier par position.

9. Bonnes pratiques professionnelles

- **Regrouper par la clé** (`id_*`), joindre le libellé dans le `SELECT` et le `GROUP BY`.
- **Vérifier le nombre de groupes** avant toute lecture : 5 magasins, 8 catégories, 10 couples magasin×année.
- **Contrôle de cohérence** : la somme des agrégats par groupe doit redonner l'agrégat global (C04).
- **Un seuil de `HAVING` doit être documenté** : « > 9 900 ventes » est une décision métier, pas une constante anonyme.
- **CASE conditionnel : toujours un `ELSE 0`** dans les comptages (jamais de `NULL` dans une addition).
- **Nommer les colonnes** : `AS nb_ventes`, `AS ca`, `AS panier_moyen` — le libellé du tableau de bord est celui des alias.

10. Exercice guidé — le tableau de bord des 5 magasins (15 min, /10)

Énoncé. Produire une ligne par magasin (5 lignes, triées par CA décroissant) avec : le nom du magasin, le nombre de ventes, le CA arrondi, le panier moyen arrondi, le nombre de ventes Espèces, le nombre de ventes Carte.

Grille de correction (/10).

Critère	Points
GROUP BY m.id_magasin, m.nom (clé + libellé)	2
COUNT(*), ROUND(SUM(...)), ROUND(AVG(...))	2
2 comptages conditionnels avec ELSE 0	2
JOIN magasin correct	1
ORDER BY ca DESC (alias de l'agrégat)	1
5 lignes, triées : 1 596 813 264 en tête	2

Corrigé.

```
SELECT m.id_magasin,
       m.nom,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen,
       SUM(CASE WHEN v.id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN v.id_mode = 2 THEN 1 ELSE 0 END) AS cartes
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.id_magasin, m.nom
ORDER BY ca DESC;
```

→ 5 lignes : Bobo Sarfalao (10 061 / 1 596 813 264 / 158 713 / 4 093 / 2 469), Ouaga Centre (10 099 / 1 595 223 423 / 157 959 / 4 109 / 2 502), Ouaga Patte d'Oie (10 033 / 1 578 987 708 / 157 379 / 4 043 / 2 532), Koudougou (9 942 / 1 572 297 452 / 158 147 / 3 957 / 2 449), Bobo Centre (9 873 / 1 564 937 883 / 158 507 / 3 973 / 2 414).

11. Exercices autonomes

Exercice 1 — CA par catégorie

Une ligne par catégorie (8 lignes) : id, rayon, sous-catégorie, nombre de ventes, CA arrondi, triée par CA décroissant.

Exercice 2 — Panier moyen par magasin

Une ligne par magasin : panier moyen arrondi, triée décroissante.

Exercice 3 — Le CA par année

Une ligne par année : nombre de ventes, CA arrondi.

Exercice 4 — Catégories actives

Les catégories (par clé) ayant plus de 6 500 ventes, avec leur nombre de ventes.

Exercice 5 — Rayon vs clé

Compter le nombre de groupes produits par `GROUP BY c.rayon` et par `GROUP BY c.id_categorie` sur la même requête de base. Expliquer l'écart.

Exercice 6 — Espèces et cartes par magasin

Deux comptages conditionnels par magasin, triés par nombre de ventes Espèces décroissant.

Exercice 7 — Couper par magasin et par année

Le nombre de ventes du magasin 4 en 2025 et en 2026 (2 lignes).

Exercice 8 — Diagnostic d'erreur

Que fait cette requête, et pourquoi est-elle une erreur ?

```
-- ERREUR ATTENDUE : un agrégat dans WHERE est interdit (cf. HAVING)
SELECT id_magasin, COUNT(*) AS nb
FROM vente
WHERE COUNT(*) > 9900;
```

12. Correction détaillée

Exercice 1.

```
SELECT c.id_categorie, c.rayon, c.sous_categorie,
       COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie, c.rayon, c.sous_categorie
ORDER BY ca DESC;
```

→ 8 lignes (cf. §7.2).

Exercice 2.

```
SELECT id_magasin, ROUND(AVG(montant_ttc)) AS panier_moyen
FROM vente
GROUP BY id_magasin
ORDER BY panier_moyen DESC;
```

→ 158 713 / 158 507 / 158 147 / 157 959 / 157 379 FCFA.

Exercice 3.

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee,
       COUNT(*) AS nb, ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 24 920 / 3 925 215 671 ; 2026 : 25 088 / 3 983 044 060 FCFA.

Exercice 4.

```
SELECT c.id_categorie, COUNT(*) AS nb
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.id_categorie
HAVING COUNT(*) > 6500
ORDER BY nb DESC;
```

→ 4 catégories : 1 (7 166), 4 (7 037), 8 (6 825), 7 (6 798).

Exercice 5. `GROUP BY c.rayon` produit **5 groupes** (Bricolage 12 418, Jardinage 12 037, Alimentaire 11 930, Hygiène 6 825, Decoration 6 798) ; `GROUP BY c.id_categorie` produit **8 groupes** (cf. §7.2). L'écart (3) vient des trois rayons doublés : Alimentaire, Bricolage, Jardinage — chaque libellé en fusionne deux.

Exercice 6. Cf. §7.6 → Magasin 1 : 4 109 / 2 502, puis 4 : 4 093 / 2 469, 2 : 4 043 / 2 532, 3 : 3 973 / 2 414, 5 : 3 957 / 2 449.

Exercice 7.

```
SELECT EXTRACT(YEAR FROM date_vente) AS annee, COUNT(*) AS nb
FROM vente
WHERE id_magasin = 4
GROUP BY 1
ORDER BY annee;
```

→ 2025 : 5 064 ; 2026 : 4 997.

Exercice 8. C'est une erreur : `COUNT(*)` est un **agrégat**, et `WHERE` ne peut pas en contenir (il filtre les lignes, avant le regroupement). Le bon écrit est `GROUP BY id_magasin HAVING COUNT(*) > 9900`.

13. Mini-projet — « Le rapport mensuel multi-dimensionnel » (45 min)

Produire, dans un seul fichier SQL exécuté sur `commercial.duckdb`, les 4 requêtes du rapport mensuel. Chaque requête est vérifiée à la main (nombre de lignes + cohérence des totaux).

Requête	Lignes attendues
R1 : CA, nb de ventes, panier moyen par magasin (trié CA décroissant)	5
R2 : CA par catégorie (par clé), trié CA décroissant	8
R3 : CA par année (2 lignes)	2
R4 : Espèces / Carte / Mobile Money (id 3) par magasin	5

Contrôles de cohérence. (i) La somme des CA de R1 = 7 908 259 731 FCFA. (ii) La somme des CA de R2 = 7 908 259 731 FCFA. (iii) La somme des CA de R3 = 7 908 259 731 FCFA. (iv) La somme des `nb_ventes` de R1 = 50 008.

Corrigé.

```
-- R1
SELECT m.nom, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca,
       ROUND(AVG(v.montant_ttc)) AS panier_moyen
FROM vente v JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.nom
ORDER BY ca DESC;

-- R2
SELECT c.rayon, c.sous_categorie, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
JOIN categorie c ON c.id_categorie = p.id_categorie
GROUP BY c.rayon, c.sous_categorie
ORDER BY ca DESC;

-- R3
SELECT EXTRACT(YEAR FROM date_vente) AS annee, COUNT(*) AS nb,
       ROUND(SUM(montant_ttc)) AS ca
FROM vente
GROUP BY 1
ORDER BY annee;

-- R4
SELECT id_magasin,
       SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END) AS especes,
       SUM(CASE WHEN id_mode = 2 THEN 1 ELSE 0 END) AS cartes,
       SUM(CASE WHEN id_mode = 3 THEN 1 ELSE 0 END) AS mobile
FROM vente
GROUP BY id_magasin
ORDER BY id_magasin;
```

Dans les faits. PostgreSQL exécute ces 4 requêtes avec le même résultat ; le `GROUP BY m.nom` (R1) est accepté car `nom` est fonctionnellement dépendant de `id_magasin` — DuckDB l’accepte aussi, mais la pratique portable reste de citer la clé (`GROUP BY m.id_magasin, m.nom`). MySQL en mode `ONLY_FULL_GROUP_BY` exigera que toute colonne du `SELECT` soit agrégée ou groupée ; en mode permissif, il devine — à éviter (cf. M07.C04 §13).

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 outils de regroupement : `GROUP BY` (une ligne de synthèse par groupe), le **regroupement par clé** (piège du libellé doublé), `HAVING` (filtre sur les groupes), et les **comptages conditionnels** (`SUM(CASE WHEN ... THEN 1 ELSE 0 END)` par groupe). La distinction `WHERE` / `HAVING` est le réflexe à acquérir.

Outil	Usage	Syntaxe	Piège
<code>GROUP BY col</code>	une ligne par groupe	<code>GROUP BY id_magasin</code>	colonne du <code>SELECT</code> ni agrégée ni groupée
Clé de regroupement	regrouper sans fusionner	<code>GROUP BY c.id_categorie</code>	le libellé doublé (rayon) fusionne
Multi-colonnes	coupe croisée	<code>GROUP BY id_magasin, annee</code>	chaque ligne = une combinaison
<code>HAVING</code>	filtrer les groupes	<code>HAVING COUNT(*) > 9900</code>	l’écrire dans <code>WHERE</code> (erreur)
<code>SUM(CASE ...)</code>	comptage conditionnel	<code>SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)</code>	oublier <code>ELSE 0</code>
Contrôle	cohérence totaux	des somme des groupes = total global	ne pas vérifier le nombre de groupes

15. Résumé du chapitre

- **GROUP BY** subdivise les lignes en groupes ; chaque agrégat est calculé par groupe ; le résultat contient une ligne par groupe.
- **Regrouper par la clé** (`id_*`), pas par le libellé : un libellé doublé fusionne silencieusement deux groupes (5 rayons au lieu de 8 catégories).
- **GROUP BY a, b** = une ligne par couple (a, b) : les coupes croisées.
- **WHERE** filtre les lignes (avant agrégation) ; **HAVING** filtre les groupes (après agrégation). Filtrer sur un total $\Rightarrow$ **HAVING**.
- Le **comptage conditionnel** `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` produit les tableaux de bord croisés : une colonne de mesure par condition, par groupe.
- **Contrôle systématique** : nombre de groupes + somme des agrégats = total global.

16. À retenir

- **Une ligne par groupe** : **GROUP BY** est la plieuse par tiroirs.
- **Clé, pas libellé** : le total juste ne suffit pas — le nombre de groupes doit être juste.
- **WHERE lignes / HAVING groupes** : si vous filtrez sur un total, c'est **HAVING**.
- **ELSE 0 obligatoire** dans les comptages conditionnels.
- **Cohérence** : somme des CA par groupe = CA global (7 908 259 731 FCFA).

À retenir. Le **HAVING** est le seul endroit où l'on peut **comparer un agrégat à un seuil** : `HAVING SUM(montant_ttc) > X`. Tout le reste de la requête (**WHERE**, **ORDER BY**, **LIMIT**) travaille sur des lignes ou sur des alias — jamais directement sur un agrégat dans **WHERE**.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Combien de lignes produit `SELECT id_magasin, COUNT(*) FROM vente GROUP BY id_magasin` ?
2. Pourquoi regrouper par `c.rayon` au lieu de `c.id_categorie` est-il risqué ici ?
3. Où écrire `COUNT(*) > 9900` : dans **WHERE** ou dans **HAVING** ? Pourquoi ?
4. Que fait `SUM(CASE WHEN id_mode = 1 THEN 1 ELSE 0 END)` dans un **GROUP BY** ?
5. Quel est le contrôle de cohérence à faire sur un **GROUP BY** ?

Réponses :

1. **5 lignes** — une par groupe, donc une par magasin (5 magasins dans le socle).
2. Parce que le libellé `rayon` est **doublé** (Alimentaire, Bricolage, Jardinage) : le regroupement fusionnerait silencieusement 8 catégories en 5 groupes, sans erreur.
3. Dans **HAVING** : `COUNT(*)` est un agrégat, et seuls les groupes sont filtrés par **HAVING**. Dans **WHERE**, c'est une erreur (« WHERE clause cannot contain aggregates »).
4. Il compte, **dans chaque groupe**, les lignes où `id_mode = 1` (les ventes Espèces) : une colonne de comptage conditionnel par groupe.
5. Deux contrôles : (i) le **nombre de groupes** est conforme (5 magasins, 8 catégories) ; (ii) la **somme des agrégats** par groupe redonne l'agrégat global (CA total 7 908 259 731 FCFA).

Module M07.C06 — Catégoriser : CASE, COALESCE, NULLIF

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 3 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C04 (agrégats) ; M07.C05 (GROUP BY).

L'idée du chapitre. Jusqu'ici, on a **lu** et **compté** les données. C06 les **transforme** : « segmenter les clients en 3 classes », « étiqueter chaque vente selon sa taille », « remplacer les absences par un texte neutre ». C'est le travail de **catégorisation** : CASE (la brique de logique conditionnelle du SELECT), COALESCE (le premier non nulle), NULLIF (le comparateur qui produit du NULL). Ces trois fonctions sont les outils de tout le travail de préparation de données : créer une colonne d'étiquette, normaliser un libellé, gérer les absences sans casser les agrégats. Le fil rouge est la base `commercial.duckdb` : on y segmente les 1 200 clients en 3 types (particuliers 850, entreprises 229, comptoirs 121), on étiquette les 50 008 ventes en 3 tranches de montant, et on mesure le CA des produits **inactifs** (471 206 245 FCFA — un produit « retiré » peut très bien avoir été vendu hier).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 1 200 clients (850 particuliers, 229 entreprises, 121 comptoirs ; Ouagadougou 512, Bobo-Dioulasso 441, Koudougou 247), 50 008 ventes (tranches : 12 753 / 21 509 / 15 746), 380 produits (362 actifs, 18 inactifs). Aucune colonne du socle ne contient de NULL (le générateur les exclut) : les fonctions COALESCE / NULLIF sont donc démontrées sur des expressions, pas sur des données réellement absentes.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **construire une colonne d'étiquette** avec CASE WHEN ... THEN ... ELSE ... END (forme simple et forme de recherche) ;
- **segmenter** un jeu de données en classes (types de clients, tranches de montant) et agréger sur ces classes ;
- **gérer les absences** avec COALESCE(x, défaut) et comprendre pourquoi un NULL dans une somme « disparaît » ;
- **produire du NULL volontairement** avec NULLIF(a, b) et l'exploiter (filtrer les zéros, détecter des égalités) ;
- **combinaison** ces fonctions dans un SELECT (et dans un GROUP BY) pour préparer des tableaux de bord lisibles.

2. Pourquoi cette notion est importante

- **CASE est l'outil n° 1 de la transformation de données** : 90 % des « colonnes créées » dans un entrepôt sont des CASE (segmentation, étiquettes, règles métier). C'est aussi la brique des comptages conditionnels de C04/C05 (SUM(CASE WHEN ...)).
- **COALESCE est le remède aux NULL** : un champ vide qui casse une somme, un libellé null qui devient « (non renseigné) » dans le rapport. Le maîtriser, c'est ne plus être surpris par les NULL en production.

- **NULLIF est le miroir de COALESCE** : il transforme une valeur en `NULL` quand elle est égale à une référence — un filtre de zéros sans `WHERE`, une détection d'égalité.

3. Explication simple — CASE est le carrefour du SELECT

Le `SELECT` de C01 copiait des colonnes. C06 lui ajoute une **logique de carrefour** : selon la valeur d'une colonne, on sort un libellé différent. Comme à un carrefour routier, on regarde une plaque (la condition) et on prend une sortie (le libellé) :

- plaque « `type_client = 'particulier'` » → sortie « Particulier » ;
- plaque « `type_client = 'entreprise'` » → sortie « Entreprise » ;
- plaque « `type_client = 'comptoir'` » → sortie « Comptoir » ;
- sinon → sortie « Autre » (le `ELSE`).

`COALESCE` et `NULLIF` sont les deux faces de la gestion des **absences** : `COALESCE(x, défaut)` dit « si `x` est absent, prends défaut » ; `NULLIF(a, b)` dit « si `a` vaut `b`, rends absent (`NULL`) ; sinon rends `a` ».

Définition. L'expression **CASE** — expression conditionnelle — est la brique de logique conditionnelle du `SELECT`. Sa forme de recherche est `CASE WHEN cond1 THEN r1 WHEN cond2 THEN r2 ... ELSE r0 END` : les conditions sont testées **dans l'ordre**, la première qui passe choisit le résultat ; `ELSE` est la valeur de repli si aucune ne passe.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
CASE (recherche)	<code>CASE WHEN c1 THEN r1 ... ELSE r0 END</code> : un libellé par condition, testées dans l'ordre.	croire que les branches sont évaluées toutes en même temps (non, dans l'ordre, la 1 ^{re} qui passe gagne).
CASE (simple)	<code>CASE x WHEN v1 THEN r1 ... END</code> : compare <code>x</code> à une valeur.	le confondre avec la forme de recherche (syntaxes différentes).
ELSE	valeur de repli si aucune condition ne passe.	l'oublier (le résultat est <code>NULL</code> pour les non-couverts).
segment	classe de lignes partageant une étiquette <code>CASE</code> .	des segments qui se chevauchent (les <code>WHEN</code> sont exclusifs par construction).
COALESCE	<code>COALESCE(a, b, ...)</code> : la première valeur non <code>NULL</code> de la liste.	croire qu'il remplace les zéros (il ne regarde que les <code>NULL</code>).
NULLIF	<code>NULLIF(a, b)</code> : <code>NULL</code> si <code>a = b</code> , sinon <code>a</code> .	le retourner (c'est <code>COALESCE</code> qui retourne).
étiquette	la colonne de texte produite par un <code>CASE</code> .	l'utiliser dans un <code>WHERE</code> avant qu'elle existe (il faut la recalculer ou une vue).

5. Cours approfondi

5.1 CASE forme de recherche — segmenter les clients

```
SELECT id_client,  
       nom,  
       type_client,  
       CASE  
         WHEN type_client = 'particulier' THEN 'Particulier'  
         WHEN type_client = 'entreprise' THEN 'Entreprise'  
         WHEN type_client = 'comptoir'   THEN 'Comptoir'  
         ELSE 'Autre'  
       END AS segment  
FROM client  
ORDER BY segment, nom  
LIMIT 6;
```

La colonne `segment` est une **étiquette** : elle est calculée à la volée, elle n'existe pas dans la table. Les 3 `WHEN` couvrent exactement les 3 valeurs du socle (850 particuliers, 229 entreprises, 121 comptoirs) : le `ELSE 'Autre'` n'attrapera rien ici, mais il protège le code contre une 4^e valeur qui arriverait demain.

Attention. Les branches `WHEN` sont testées **dans l'ordre écrit**, et **la première qui passe gagne**. Un `CASE` où la condition la plus générale vient avant la plus restrictive rend les branches suivantes mortes. Ici les 3 `WHEN` sont mutuellement exclusifs (une valeur de `type_client` ne peut en valoir qu'une), donc l'ordre est sans effet — mais sur des conditions qui se chevauchent (des intervalles), l'ordre change le résultat.

5.2 CASE forme simple — le même, en plus court

Quand on compare **la même colonne** à plusieurs valeurs constantes, la forme simple est plus lisible :

```
SELECT type_client,  
       CASE type_client  
         WHEN 'particulier' THEN 'Particulier'  
         WHEN 'entreprise' THEN 'Entreprise'  
         WHEN 'comptoir'   THEN 'Comptoir'  
         ELSE 'Autre'  
       END AS segment  
FROM client  
LIMIT 3;
```

`CASE x WHEN v1 ...` est un sucre syntaxique de `CASE WHEN x = v1 ...`. Pour des conditions arbitraires (comparaisons, `BETWEEN`, `LIKE`), seule la forme de recherche convient.

5.3 CASE pour étiqueter des tranches — la segmentation numérique

La question classique : « combien de ventes petites, moyennes, grosses ? » On découpe `montant_ttc` en 3 tranches :

```

SELECT CASE
    WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
    WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
    ELSE 'c) 200 000 et plus'
END AS tranche,
COUNT(*) AS nb_ventes,
ROUND(MIN(montant_ttc)) AS min_m,
ROUND(MAX(montant_ttc)) AS max_m
FROM vente
GROUP BY 1
ORDER BY tranche;

```

tranche	nb_ventes	min_m	max_m
a) moins de 50 000	12 753	763	49 964
b) 50 000 à 199 999	21 509	50 008	199 920
c) 200 000 et plus	15 746	200 017	594 363

Trois segments, disjoints, qui se recouvrent totalement ($12\,753 + 21\,509 + 15\,746 = 50\,008$). C'est le contrôle de toute segmentation : **la somme des segments redonne le total**. Remarquez l'astuce de la 2^e branche : `montant_ttc < 200000` suffit car la 1^{re} a déjà attrapé `< 50000` — les branches précédentes étant passées, on est sûr d'être au-dessus de 50 000.

À retenir. Dans un CASE d'intervalle, **on n'écrit que la borne haute** de chaque branche (`< 50000`, puis `< 200000`, sinon) : la borne basse est implicitement couverte par la branche précédente. C'est ce qui rend les tranches **disjointes** sans écrire `BETWEEN 50000 AND 199999` — et c'est ce qu'il faut vérifier en relecture.

Définition. Une **tranche** — bucket — est un intervalle de valeurs découpé par des CASE WHEN successifs (ici `< 50 000`, `< 200 000`, sinon). Les tranches sont **disjointes** (une ligne n'en vaut qu'une) et **exhaustives** (avec un ELSE, toute ligne est classée). La segmentation en tranches est la base des histogrammes et des « classements par taille ».

5.4 CASE combiné à GROUP BY — agréger sur les étiquettes

L'étiquette se réutilise dans le GROUP BY (via sa position, C03) ou en répétant l'expression. Le CA par segment de client :

```

SELECT CASE
    WHEN c.type_client = 'particulier' THEN 'Particulier'
    WHEN c.type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
END AS segment,
COUNT(*) AS nb_ventes,
ROUND(AVG(v.montant_ttc)) AS panier_moyen,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;

```

segment	nb_ventes	panier_moyen	ca
Particulier	35 377	158 181	5 595 962 715
Entreprise	9 471	156 542	1 482 606 616
Comptoir	5 160	160 793	829 690 400

Le panier moyen des **comptoirs** (160 793 FCFA) est le plus élevé — ils achètent moins (5 160 ventes) mais plus gros. Le total des 3 ca redonne 7 908 259 731 FCFA (contrôle de cohérence de C05).

5.5 COALESCE — le premier non nulle

`COALESCE(a, b, c, ...)` renvoie la **première** valeur de la liste qui n'est pas `NULL`. C'est l'outil de **repli** :

```
-- Si plafond_credit est absent, afficher 0 (au lieu de NULL)
SELECT id_client, nom,
       COALESCE(plafond_credit, 0) AS plafond_sur
FROM client
ORDER BY id_client
LIMIT 3;
```

Sur le socle M07, `plafond_credit` n'a aucun `NULL` : `COALESCE(plafond_credit, 0)` renvoie donc toujours `plafond_credit`. Mais la démonstration est valide — et c'est le geste standard dès qu'une colonne réelle contient des absences.

Cas d'usage n° 2 : un **libellé de repli** dans un rapport :

```
SELECT nom,
       COALESCE(ville, '(non renseignée)') AS ville_affichee
FROM client
ORDER BY nom
LIMIT 3;
```

Un client sans ville n'affichera pas une case vide mais « (non renseignée) » — le rapport reste lisible.

Définition. La fonction **COALESCE** — coalescer — renvoie la **première** valeur de sa liste d'arguments qui n'est pas `NULL`. `COALESCE(x, défaut)` est le substitut portable de l'opérateur `??` (MySQL) et de `x IS NOT NULL THEN x` (un `CASE`) elle transforme une absence en valeur de repli. Elle ne regarde que les `NULL` — un zéro ou un texte vide sont des valeurs présentes, pas des absences.

Attention. `COALESCE` ne remplace **pas les zéros** et **pas les textes vides** : `COALESCE(0, 1)` renvoie 0, `COALESCE('', 'défaut')` renvoie '' (le tiret médian est une valeur, pas un `NULL`). Pour « 0 ou vide ⇒ défaut », il faut un `CASE` explicite.

5.6 NULLIF — produire du NULL volontairement

`NULLIF(a, b)` renvoie `NULL` **si** `a = b`, sinon `a`. C'est le miroir de `COALESCE` : là où `COALESCE` transforme un `NULL` en valeur, `NULLIF` transforme une valeur en `NULL`.

Usage n° 1 : **filtrer les zéros d'une somme** sans `WHERE` :

```
-- La marge d'un produit à prix nul : NULL au lieu de 0 (signal d'absence)
SELECT NULLIF(0, 0)      AS zero_devenu_null,
       NULLIF(5, 0)      AS cinq_garde_cinq,
       NULLIF('a', 'a')  AS egal_devenu_null;
```

→ `NULL`, 5, `NULL`. Les deux égalités produisent du `NULL`, l'inégalité garde la valeur.

Usage n° 2 : **combinaison COALESCE + NULLIF** pour « vider les zéros puis mettre un repli » (mêmes types des deux côtés du `COALESCE` — ici des textes) :

```
SELECT COALESCE(NULLIF('0', '0'), 'aucun') AS zero_vide_puis_repli,
       COALESCE(NULLIF('5', '0'), 'aucun') AS cinq_garde_cinq;
```

→ « aucun » (le « 0 » devient NULL, le COALESCE met le repli) / « 5 ». Sur des nombres, le repli doit être un nombre : COALESCE(NULLIF(0, 0), -1).

Définition. La fonction **NULLIF** — nullifier si égal — renvoie NULL quand ses deux arguments sont égaux, sinon le premier argument. NULLIF(a, 0) est le filtre de zéros portable : il transforme un zéro (valeur « pas de donnée ») en NULL (absence), ce qui permet ensuite à COALESCE de poser un repli. C’est le miroir exact de COALESCE : l’un sort des NULL, l’autre y entre.

5.7 Le CASE dans le WHERE — filtrer sur une étiquette

On ne peut pas filtrer sur une étiquette du SELECT (elle n’existe pas encore au moment du WHERE) : on **recalcule** la condition, ou on encapsule dans une vue / CTE (C08). Ici, la recalcul direct :

```
SELECT COUNT(*) AS nb_ventes_grosses
FROM vente
WHERE montant_ttc >= 200000;
```

→ 15 746 ventes (la tranche c). Le WHERE ne voit que des expressions sur les colonnes réelles ; l’étiquette du SELECT est calculée **après** (ordre d’exécution C01).

5.8 Ce que ce chapitre ne couvre pas

- Les **fonctions de texte** (UPPER, LOWER, TRIM, SUBSTR, LIKE) : M11.
- Les **vues** et **CTE** pour encapsuler une étiquette réutilisable : C08.
- L’optimisation des **CASE** imbriqués et des fonctions scalaires : M11.

6. Exemple concret — la requête qui répond à une question métier

La responsable fidélité demande : « Combien de ventes par type de client, et quel est le CA de chaque type ? Et si un type avait un libellé vide, que faudrait-il afficher ? »

Étape 1 — Écrire la requête (segmentation + COALESCE de repli).

```
SELECT COALESCE(CASE c.type_client
                 WHEN 'particulier' THEN 'Particulier'
                 WHEN 'entreprise' THEN 'Entreprise'
                 WHEN 'comptoir'   THEN 'Comptoir'
                 ELSE 'Autre'
               END, '(inconnu)') AS segment,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. 3 segments (les 3 types du socle) ; le COALESCE de repli ne s’activera que si un type_client serait NULL (aucun ici). Particulier en tête (le plus de ventes).

Étape 3 — Exécuter. Particulier 35 377 ventes / 5 595 962 715 FCFA ; Entreprise 9 471 / 1 482 606 616 ; Comptoir 5 160 / 829 690 400.

Étape 4 — Comparer. 3 segments, aucun « (inconnu) » (pas de NULL dans le socle), total 7 908 259 731 FCFA. Le COALESCE est un filet de sécurité, pas un acteur ici.

Conseil professionnel. Dans une requête de production, **toute étiquette CASE qui peut tomber dans le vide doit avoir un ELSE**, et ce ELSE doit être un libellé explicite (« (inconnu) », « (non classé) ») plutôt qu'un NULL : un NULL dans une colonne de segment disparaît des GROUP BY lisibles et des tableaux de bord. Le COALESCE autour du CASE est le filet de dernier recours.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Les 3 types de clients

```
SELECT type_client, COUNT(*) AS nb_clients
FROM client
GROUP BY type_client
ORDER BY nb_clients DESC;
```

→ particulier 850, entreprise 229, comptoir 121.

7.2 Question 2 — Le CA par type de client

```
SELECT c.type_client,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY c.type_client
ORDER BY ca DESC;
```

→ particulier 5 595 962 715 FCFA, entreprise 1 482 606 616, comptoir 829 690 400.

7.3 Question 3 — Les 3 villes et leur CA

```
SELECT c.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c ON c.id_client = v.id_client
GROUP BY c.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 ventes / 3 391 679 290 FCFA, Bobo-Dioulasso 18 364 / 2 895 549 769, Koudougou 10 354 / 1 621 030 672.

7.4 Question 4 — La segmentation en 3 tranches

```
SELECT CASE
       WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
       WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
       ELSE 'c) 200 000 et plus'
END AS tranche,
       COUNT(*) AS nb_ventes
FROM vente
GROUP BY 1
ORDER BY tranche;
```

→ 12 753 / 21 509 / 15 746 (total 50 008).

7.5 Question 5 — Les ventes des comptoirs, par ville

```
SELECT c.ville, COUNT(*) AS nb_ventes
FROM vente v
JOIN client c ON c.id_client = v.id_client
WHERE c.type_client = 'comptoir'
GROUP BY c.ville
ORDER BY nb_ventes DESC;
```

→ Ouagadougou 2 621, Bobo-Dioulasso 1 534, Koudougou 1 005.

7.6 Question 6 — Le CA des produits inactifs (piège silencieux)

```
SELECT COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
WHERE p.aktif = FALSE;
```

→ 2 434 ventes, 471 206 245 FCFA. Les 18 produits **inactifs** ont tout de même généré cette CA (sur l'historique 2025-2026) : « inactif » signifie « plus proposé », pas « jamais vendu ». C'est le piège d'interprétation classique d'une colonne de statut.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Oublier le <code>ELSE</code> dans un <code>CASE</code>	lignes non couvertes = <code>NULL</code> dans l'étiquette	toujours un <code>ELSE</code> explicite.
2	Branche <code>WHEN</code> trop générale en 1 ^{re} position	branches suivantes jamais atteintes (mortes)	conditions de la plus spécifique à la plus générale.
3	Filtrer sur une étiquette dans le <code>WHERE</code>	erreur ou résultat inattendu	recalculer la condition, ou vue/CTE (C08).
4	Attendre que <code>COALESCE</code> remplace un 0	le 0 passe tel quel	<code>COALESCE(NULLIF(x, 0), défaut)</code> pour vider les zéros.
5	Confondre <code>CASE x WHEN v</code> (simple) et <code>CASE WHEN c</code> (recherche)	syntaxe erronée	simple = comparaison d'une valeur ; recherche = conditions arbitraires.
6	Croire que « inactif » = « jamais vendu »	sous-estimer l'historique d'un statut	vérifier le CA réel par statut (cf. 7.6).

Attention. L'erreur 2 est la plus subtile : un `CASE` où la branche la plus générale est en premier **rend les autres branches mortes** sans erreur. Sur des intervalles, toujours du plus étroit au plus large, ou utiliser des bornes fermées explicites (`BETWEEN`).

Attention. Un `NULL` dans un `CASE` : `WHEN NULL = 'x'` est `NULL` (ni vrai ni faux), donc la branche ne passe pas. Pour tester l'absence, écrire `WHEN x IS NULL THEN` — jamais `WHEN x = NULL`.

9. Bonnes pratiques professionnelles

- **Toujours un `ELSE`** dans un `CASE` de segmentation, avec un libellé explicite.
- **L'ordre des `WHEN`** : du plus spécifique au plus général (surtout sur des intervalles).
- **Contrôler la segmentation** : la somme des segments redonne le total (ici 50 008).
- **`COALESCE` sur toute colonne affichée** qui peut être absente en production.

- **Documenter le statut** : « actif / inactif » doit être défini (plus proposé vs jamais vendu) — sinon les rapports mentent par omission.
- **Préférer une vue/CTE (C08)** quand l'étiquette `CASE` est réutilisée plusieurs fois.

10. Exercice guidé — le rapport de segmentation (15 min, /10)

Énoncé. Produire, par type de client (3 lignes, triées par CA décroissant) : le segment (étiquette lisible), le nombre de ventes, le panier moyen arrondi, le CA arrondi. Ajouter un `COALESCE` de repli « (inconnu) » autour du `CASE`.

Grille de correction (/10).

Critère	Points
<code>CASE</code> 3 branches + <code>ELSE</code>	2
<code>COALESCE(..., '(inconnu)')</code>	1
<code>COUNT(*)</code> , <code>ROUND(AVG(...))</code> , <code>ROUND(SUM(...))</code>	2
<code>JOIN client</code> + <code>GROUP BY 1</code>	2
<code>ORDER BY ca DESC</code>	1
3 lignes, Particulier en tête (5 595 962 715)	2

Corrigé. Cf. §6, étape 1. → Particulier 35 377 / 158 181 / 5 595 962 715 ; Entreprise 9 471 / 156 542 / 1 482 606 616 ; Comptoir 5 160 / 160 793 / 829 690 400.

11. Exercices autonomes

Exercice 1 — Les 3 types de clients

Compter les clients par `type_client`, trié décroissant.

Exercice 2 — Le CA par ville

Une ligne par ville : nombre de ventes, CA arrondi, trié par CA décroissant.

Exercice 3 — La segmentation en tranches

Les 3 tranches de montant (< 50 000 / 50 000–199 999 / ≥ 200 000) avec leur nombre de ventes. Vérifier que le total redonne 50 008.

Exercice 4 — `NULLIF` en solo

Quel est le résultat de `SELECT NULLIF(7, 7), NULLIF(7, 8), NULLIF('x', 'x');` ?

Exercice 5 — `COALESCE` + `NULLIF`

Réécrire « si `plafond_credit` vaut 0 ou est `NULL`, afficher 500 000 ; sinon le `plafond_credit` » avec `COALESCE` et `NULLIF`.

Exercice 6 — Comptoirs par ville

Les ventes des comptoirs, une ligne par ville, triées décroissant.

Exercice 7 — Le CA des produits inactifs

Nombre de ventes et CA des produits `actif = FALSE`.

Exercice 8 — Diagnostic d'erreur

Pourquoi cette requête est-elle une erreur, et comment la corriger ?

```
SELECT segment, COUNT(*)
FROM (
  SELECT CASE
    WHEN type_client = 'particulier' THEN 'Particulier'
    WHEN type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
  END AS segment
  FROM client
) t
GROUP BY segment;
```

12. Correction détaillée

Exercice 1.

```
SELECT type_client, COUNT(*) AS nb_clients
FROM client
GROUP BY type_client
ORDER BY nb_clients DESC;
```

→ particulier 850, entreprise 229, comptoir 121.

Exercice 2. Cf. §7.3 → Ouagadougou 21 290 / 3 391 679 290 ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672 FCFA.

Exercice 3. Cf. §7.4 → 12 753 / 21 509 / 15 746 ; total 50 008 ✓.

Exercice 4. `NULL`, `7`, `NULL` : les deux égalités (`7=7`, `'x'='x'`) produisent du `NULL` ; l'inégalité (`7≠8`) garde `7`.

Exercice 5.

```
SELECT COALESCE(NULLIF(plafond_credit, 0), 500000) AS plafond_final
FROM client
ORDER BY id_client
LIMIT 3;
```

`NULLIF(plafond_credit, 0)` transforme le `0` en `NULL` ; `COALESCE(..., 500000)` met le repli pour les `NULL` (et les `0`). Sur le socle, aucun plafond n'est `0` ni `NULL` : le `plafond_final` vaut toujours le `plafond_credit`.

Exercice 6. Cf. §7.5 → Ouagadougou 2 621, Bobo-Dioulasso 1 534, Koudougou 1 005.

Exercice 7. Cf. §7.6 → 2 434 ventes, 471 206 245 FCFA.

Exercice 8. Cette requête est en fait **correcte** (une sous-requête nommée `t` qui produit `segment`, puis un `GROUP BY segment` valide) — le piège est de chercher l'erreur alors qu'il n'y en a pas. Le cas qui **serait** une erreur, c'est de vouloir `GROUP BY segment` **sans** la sous-requête (l'alias du `SELECT` n'est pas groupable telle quelle dans tous les SGBD). La bonne pratique : encapsuler l'étiquette (sous-requête ici, vue ou CTE en C08) avant de la regrouper.

13. Mini-projet — « La grille de segmentation complète » (40 min)

Produire la **grille de segmentation** de la base : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes attendues
G1 : segment par type de client (nb, panier moyen, CA)	3
G2 : tranche de montant (nb, min, max)	3
G3 : CA par ville (nb, CA)	3
G4 : statut produit (actif/inactif) : nb de produits, nb de ventes, CA	2

Contrôles de cohérence. (i) La somme des `nb_ventes` de G2 = 50 008. (ii) La somme des `ca` de G1 = 7 908 259 731 FCFA. (iii) G4 : actif 362 produits, inactif 18 ; le CA inactif = 471 206 245 FCFA.

Corrigé.

```
-- G1
SELECT COALESCE(CASE c.type_client
                WHEN 'particulier' THEN 'Particulier'
                WHEN 'entreprise' THEN 'Entreprise'
                WHEN 'comptoir'   THEN 'Comptoir'
                ELSE 'Autre'
                END, '(inconnu)') AS segment,
COUNT(*) AS nb,
ROUND(AVG(v.montant_ttc)) AS panier_moyen,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN client c ON c.id_client = v.id_client
GROUP BY 1
ORDER BY ca DESC;

-- G2
SELECT CASE
    WHEN montant_ttc < 50000 THEN 'a) moins de 50 000'
    WHEN montant_ttc < 200000 THEN 'b) 50 000 à 199 999'
    ELSE 'c) 200 000 et plus'
END AS tranche,
COUNT(*) AS nb,
ROUND(MIN(montant_ttc)) AS min_m,
ROUND(MAX(montant_ttc)) AS max_m
FROM vente
GROUP BY 1
ORDER BY tranche;

-- G3
SELECT c.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN client c ON c.id_client = v.id_client
GROUP BY c.ville
ORDER BY ca DESC;

-- G4
SELECT p.actif,
COUNT(DISTINCT p.id_produit) AS nb_produits,
COUNT(*) AS nb_ventes,
ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v JOIN produit p ON p.id_produit = v.id_produit
GROUP BY p.actif
ORDER BY p.actif DESC;
```

Dans les faits. PostgreSQL et SQLite partagent cette syntaxe `CASE / COALESCE / NULLIF` (elles sont **standard SQL**) : la grille s'exécute telle quelle sur les deux. MySQL aussi, mais son opérateur `??` (équivalent `COALESCE` à 2 arguments) est un ajout non standard : pour la portabilité, écrire `COALESCE`.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 3 outils de transformation : `CASE` (la brique de segmentation, formes simple et recherche), `COALESCE` (le repli des `NULL`), `NULLIF` (le producteur de `NULL`). Ensemble, ils couvrent la quasi-totalité du travail de « création de colonnes d'étiquette » d'un entrepôt. Le `ELSE` obligatoire et le contrôle « somme des segments = total » sont les deux réflexes.

Outil	Usage	Syntaxe	Piège
<code>CASE WHEN ... END</code>	étiquette condition	par <code>CASE WHEN x < 50000 THEN 'petit' ELSE 'grand' END</code>	<code>ELSE</code> oublié, branches mortes
<code>CASE x WHEN v END</code>	étiquette valeur	par <code>CASE type WHEN 'pro' THEN 'Pro' END</code>	le confondre avec la forme recherche
<code>COALESCE</code>	repli des <code>NULL</code>	<code>COALESCE(ville, '(non renseignée)')</code>	ne remplace pas 0 / vide
<code>NULLIF</code>	produire du <code>NULL</code>	<code>NULLIF(plafond, 0)</code>	le retourner (c'est <code>COALESCE</code>)
<code>COALESCE(NULLIF(...))</code>	vider les zéros puis repli	<code>COALESCE(NULLIF(x, 0), 0)</code>	l'oublier sur une somme
Contrôle	cohérence	somme des segments = total	ne pas vérifier le total

15. Résumé du chapitre

- **CASE** est la brique de segmentation : forme de recherche (conditions arbitraires) et forme simple (comparaison d'une valeur). Les `WHEN` sont testés **dans l'ordre**, la 1^{re} qui passe gagne ; **toujours un ELSE**.
- **Segmenter** = découper en tranches disjointes et exhaustives ; le contrôle : la somme des segments redonne le total (50 008).
- **COALESCE(a, b, ...)** = la première valeur non `NULL` : le repli des absences. Il ne regarde que les `NULL` (pas les 0, pas les vides).
- **NULLIF(a, b)** = `NULL` si `a = b`, sinon `a` : le producteur de `NULL` (le miroir de `COALESCE`). `COALESCE(NULLIF(x, 0), défaut)` vide les zéros puis met le repli.
- **Filtrer sur une étiquette** : on recalcule la condition dans le `WHERE`, ou on encapsule l'étiquette (sous-requête, vue, CTE — C08).
- **Interpréter les statuts** : « inactif » $\neq$ « jamais vendu » (ici 471 206 245 FCFA de CA sur 18 produits inactifs).

16. À retenir

- **CASE** : **toujours un ELSE**, ordre des `WHEN` du spécifique au général.
- **Segmentation** : tranches disjointes + exhaustives ; somme des segments = total.
- **COALESCE** = **repli des NULL** (pas des 0 / vides) ; **NULLIF** = **producteur de NULL**.
- **COALESCE(NULLIF(x, 0), défaut)** : le duo pour vider les zéros puis replier.
- **Statut $\neq$ historique** : vérifier le CA réel par statut avant d'interpréter.

À retenir. `CASE`, `COALESCE`, `NULLIF` sont **standard SQL** : elles s'écrivent à l'identique sur DuckDB, PostgreSQL et SQLite. C'est l'un des rares pans du langage où la portabilité est totale — à condition de ne pas employer les raccourcis non standard (`??`, `IFNULL` MySQL, `DECODE` Oracle).

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. À quoi sert le `ELSE` dans un `CASE`, et que se passe-t-il s'il est absent ?
2. Quelle est la différence entre `CASE x WHEN v1 ...` et `CASE WHEN c1 ...` ?
3. Que renvoie `COALESCE(NULL, 0, 'x')` ? Et `COALESCE(0, 'x')` ?
4. Que renvoie `NULLIF(5, 5)` ? Et `NULLIF(5, 0)` ?
5. Pourquoi « inactif » ne veut-il pas dire « jamais vendu » ?

Réponses :

1. Le `ELSE` est la valeur de repli si aucune condition ne passe. S'il est absent, les lignes non couvertes renvoient `NULL` — des segments invisibles dans les rapports.
2. `CASE x WHEN v1 ...` compare **une valeur** `x` à des constantes (`x = v1`, `x = v2...`) — c'est un sucre. `CASE WHEN c1 ...` teste des **conditions arbitraires** (comparaisons, `BETWEEN`, `LIKE`). Pour un intervalle, seule la forme de recherche convient.
3. `COALESCE(NULL, 0, 'x') → 0` (la 1^{re} non nulle). `COALESCE(0, 'x') → 0` (le 0 est une valeur, pas un `NULL`, donc il est retenu).
4. `NULLIF(5, 5) → NULL` (égalité). `NULLIF(5, 0) → 5` (inégalité, on garde le 1^{er} argument).
5. Parce que le statut « inactif » décrit l'état **actuel** du produit (plus proposé), pas son historique : ici, les 18 produits inactifs ont généré 2 434 ventes pour 471 206 245 FCFA sur 2025-2026. Un statut ne résume pas le passé.

Module M07.C07 — Joindre : INNER, LEFT, FULL, CROSS, jointures multiples

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 5 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM, alias) ; M07.C05 (GROUP BY) ; M07.C06 (CASE).

L'idée du chapitre. C04–C06 travaillaient sur **une table à la fois** (ou sur des agrégats d'une table). C07 brise cette limite : **relier plusieurs tables** par leurs clés. « Le CA de chaque vente, avec le nom du client et la ville du magasin. » « Les magasins qui n'ont pas encore d'objectif. » C'est la **jointure** — join : l'opération qui fait d'un schéma en 9 tables un ensemble interrogeable. Le chapitre couvre **INNER JOIN** (les correspondances des deux côtés), **LEFT JOIN** (garder le côté gauche, **NULL** à droite), **FULL OUTER JOIN** (les deux côtés), **CROSS JOIN** (le produit cartésien), les **jointures multiples en étoile**, et les deux pièges du module — **P4** (la jointure qui double les lignes) et **P5** (le **DISTINCT** qui masque un mauvais **JOIN**). Le fil rouge est la base `commercial.duckdb` : on y relie 50 008 ventes à leurs 1 200 clients, 380 produits, 5 magasins et 5 modes de paiement, et on montre qu'une jointure mal conçue gonfle 50 008 lignes en **2 131 064**.

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes, 1 200 clients (tous présents dans `vente`), 380 produits (tous vendus au moins une fois), 5 magasins, 5 modes de paiement, 8 catégories, et la table `objectif_magasin` **vide** (0 lignes) — un objet de démonstration idéal pour le **LEFT JOIN** (elle produit des **NULL** réels).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **relier deux tables** par une clé avec **INNER JOIN ... ON** et comprendre le résultat (les correspondances des deux côtés) ;
- **garder le côté gauche** avec **LEFT JOIN** et interpréter les **NULL** du côté droit (les lignes sans correspondance) ;
- **garder les deux côtés** avec **FULL OUTER JOIN** ;
- **produire le produit cartésien** avec **CROSS JOIN** et savoir quand c'est voulu ;
- **enchaîner plusieurs jointures** en étoile autour d'une table centrale (`vente`) ;
- **détecter** une jointure qui double les lignes (piège P4) et un **DISTINCT** qui masque un mauvais **JOIN** (piège P5).

2. Pourquoi cette notion est importante

- **Toute interrogation métier multi-table est une jointure** : « le CA par client avec son nom », « les ventes de chaque ville ». Un schéma en 9 tables (le socle M07) ne devient lisible qu'au travers de jointures.
- **Le choix de type de jointure est une décision sémantique** : **INNER** répond à « les ventes qui ont un client » ; **LEFT** répond à « tous les magasins, avec leur objectif s'il existe ». Se tromper de type, c'est répondre à une autre question.
- **Les pièges P4/P5 sont les bugs les plus coûteux en production** : un total gonflé par une jointure qui double, ou un **DISTINCT** qui « répare » un mauvais **JOIN** sans que personne ne le sache.

3. Explication simple — la jointure est un accord de clés

Imaginez deux fiches : une **vente** (qui note `id_client = 73`) et un fichier des **clients** (une fiche par `id_client`). La jointure `vente JOIN client ON vente.id_client = client.id_client` fait l'**accord** : pour chaque vente, on retrouve la fiche du client de même `id`, et on met les deux côte à côte. Le résultat a **les colonnes des deux tables**, et **autant de lignes qu'il y a de paires accordées**.

`INNER JOIN` = « ne gardez que les paires qui se sont accordées ». `LEFT JOIN` = « gardez toutes les ventes, même celles qui n'ont trouvé aucun client (elles seront complétées de `NULL` à droite) ». C'est tout.

Définition. Une **jointure** — `join` — est l'opération qui met côte à côte les lignes de deux tables (ou plus) qui **s'accordent sur une clé**, via `ON` (par exemple `v.id_client = c.id_client`). Le résultat combine les colonnes des deux tables. Le type de jointure (`INNER`, `LEFT`, `FULL`, `CROSS`) décide quelles paires sont conservées.

4. Vocabulaire essentiel

Français — English		Définition simple	Piège à éviter
INNER JOIN		les paires qui se sont accordées des deux côtés.	croire qu'elle garde toutes les lignes (elle les filtre).
LEFT JOIN		toutes les lignes du gauche , <code>NULL</code> à droite si aucune correspondance.	mettre le filtre du côté droit dans <code>WHERE</code> (il transforme le <code>LEFT</code> en <code>INNER</code>).
FULL JOIN	OUTER	toutes les lignes des deux côtés, <code>NULL</code> où il manque.	le confondre avec <code>LEFT</code> (il garde aussi le droit seul).
CROSS JOIN		le produit cartésien : chaque ligne du gauche x chaque ligne du droit.	l'employer par accident (oubli de <code>ON</code>) : explosion de lignes.
clé de jointure		la (les) colonne(s) comparée(s) dans <code>ON</code> .	joindre sur une colonne non unique (doublons) sans le savoir.
jointure étoile	en	une table centrale (vente) reliée à N tables de référence.	joindre les tables de référence entre elles (inutile, et source de doublons).
explosion jointure	de	une jointure qui multiplie les lignes au lieu de les enrichir (P4).	ne pas vérifier <code>COUNT(*)</code> avant/après la jointure.
ON vs WHERE		<code>ON</code> définit l'accord ; <code>WHERE</code> filtre après (pour <code>LEFT</code> , le sens change).	mettre la condition du côté droit d'un <code>LEFT</code> dans <code>WHERE</code> (il filtre les <code>NULL</code>).

5. Cours approfondi

5.1 INNER JOIN — les correspondances des deux côtés

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
INNER JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 3;
```

Chaque vente retrouve le client de même `id_client`. Sur le socle, **toutes** les ventes ont un client existant (1 200 clients, tous présents dans `vente`) : le `INNER JOIN` retourne donc 50 008 lignes, chacune enrichie du `nom` du client.

```
SELECT COUNT(*) AS nb_lignes
FROM vente v
INNER JOIN client c ON c.id_client = v.id_client;
```

→ **50 008** (aucune vente orpheline dans le socle). Le `INNER JOIN` n’a rien filtré parce que la clé `id_client` de `vente` pointe toujours vers un client existant — c’est la propriété d’intégrité référentielle que le socle garantit.

Définition. La **clé étrangère** — foreign key (FK) — est la colonne d’une table qui pointe vers la clé primaire d’une autre table (`vente.id_client` → `client.id_client`). Une jointure `INNER` sur une FK intègre ne supprime jamais de ligne de la table centrale ; une FK **orpheline** (qui pointe vers une clé inexistante) serait, elle, éliminée par le `INNER JOIN`.

5.2 LEFT JOIN — garder le côté gauche, NULL à droite

La table `objectif_magasin` est **vide** (0 lignes) dans le socle. Le `LEFT JOIN` depuis `magasin` est donc l’objet de démonstration idéal : chaque magasin reste, et son objectif apparaît en `NULL`.

```
SELECT m.id_magasin, m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
ORDER BY m.id_magasin;
```

id_magasin	nom	ca_objectif_ttc
1	Ouaga Centre	NULL
2	Ouaga Patte d’Oie	NULL
3	Bobo Centre	NULL
4	Bobo Sarfalao	NULL
5	Koudougou	NULL

Cinq magasins, cinq lignes, **tous les objectifs en NULL** (la table de droite étant vide). C’est exactement le comportement voulu : « tous les magasins, avec leur objectif **s’il existe** ». Un `INNER JOIN` ici retournerait **0 ligne** (aucune paire accordée), ce qui serait une erreur de réponse.

5.3 Le piège du WHERE sur le côté droit d’un LEFT JOIN

```
-- MAUVAIS : le filtre sur o élimine les NULL et transforme le LEFT en INNER
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc > 0;
```

```
-- JUSTE : la condition sur le côté droit va dans ON
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
    AND o.ca_objectif_ttc > 0
ORDER BY m.id_magasin;
```

Dans le `WHERE`, les lignes dont `ca_objectif_ttc` est `NULL` sont **éliminées** (car `NULL > 0` est `NULL`, ni vrai ni faux) : le `LEFT JOIN` se comporte comme un `INNER`. Dans l’`ON`, la condition ne filtre pas le côté gauche : les magasins sans objectif restent, `NULL` à droite.

Attention. Mettre la condition du **côté droit** d'un `LEFT JOIN` dans le `WHERE` transforme silencieusement la jointure en `INNER` (les `NULL` du droit sont éliminés). Pour filtrer le côté droit sans perdre le gauche, écrire la condition dans l'`ON`. C'est le piège de jointure le plus fréquent en production.

5.4 FULL OUTER JOIN — les deux côtés

`FULL OUTER JOIN` garde **toutes** les lignes des deux tables : celles qui s'accordent, celles du gauche sans correspondance (`NULL` à droite), celles du droit sans correspondance (`NULL` à gauche).

```
SELECT m.id_magasin, m.nom, o.annee
FROM magasin m
FULL OUTER JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
ORDER BY m.id_magasin;
```

→ 5 lignes (les 5 magasins, `annee` en `NULL`) — ici identique au `LEFT JOIN` car la table de droite est vide (il n'y a pas de ligne du droit « seule »). Sur deux tables toutes deux peuplées et partiellement disjointes, le `FULL` ajouterait les lignes orphelines du droit.

5.5 CROSS JOIN — le produit cartésien

`CROSS JOIN` produit **toutes les paires** (chaque ligne du gauche × chaque ligne du droit), sans condition. 5 magasins × 5 modes de paiement = **25 lignes** :

```
SELECT m.nom, mp.libelle
FROM magasin m
CROSS JOIN mode_paiement mp
ORDER BY m.id_magasin, mp.id_mode;
```

→ 25 combinaisons (chacun des 5 magasins avec chacun des 5 modes). C'est voulu quand on veut une **grille complète** (tous les couples, même ceux à zéro) ; c'est un bug quand on l'obtient par oubli de `ON` (une jointure qui explose).

Définition. Le **produit cartésien** — cross product — est l'ensemble de toutes les paires (ligne du gauche × ligne du droit), sans condition : $|gauche| \times |droit|$ lignes. `CROSS JOIN` l'explique (grille complète voulue) ; une `JOIN` sans `ON` (ou un `ON` toujours vrai) le produit par accident (explosion de lignes, cf. P4).

5.6 Jointures multiples en étoile

La table centrale `vente` se relie à **quatre** tables de référence (`client`, `produit`, `magasin`, `mode_paiement`) — la **jointure en étoile**. On n'enchaîne que des `N → 1` (chaque vente pointe vers une seule valeur de référence) : le résultat garde 50 008 lignes.

```

SELECT v.id_vente,
       c.nom AS client,
       p.designation AS produit,
       m.nom AS magasin,
       mp.libelle AS mode,
       v.montant_ttc
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
ORDER BY v.id_vente
LIMIT 3;

```

```

SELECT COUNT(*) AS nb_lignes
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode;

```

→ **50 008** (chaque vente enrichie de 4 libellés, sans doublon). C'est le motif de référence de tout « rapport détaillé ».

À retenir. Une jointure en étoile ne **double jamais** les lignes tant que chaque relation est $N \rightarrow 1$ (une vente pointe vers un seul client, un seul produit, un seul magasin, un seul mode). Le contrôle est immédiat : `COUNT(*)` après l'étoile doit redonner le `COUNT(*)` de la table centrale. S'il ne redonne pas, une relation a multiplié — c'est P4.

5.7 Le piège P4 — la jointure qui double

Le piège P4 : une jointure qui **multiplie** les lignes. Le cas d'école : relier `vente` à elle-même via `id_client` (chaque vente « s'accorde » avec **toutes** les ventes du même client) :

```

SELECT COUNT(*) AS nb_lignes_explosees
FROM vente v
JOIN vente v2 ON v2.id_client = v.id_client;

```

→ **2 131 064 lignes** pour 50 008 ventes uniques. Chaque vente est comptée une fois par chacune des autres ventes de son client (son nombre de ventes + 1). Le `COUNT(*)` est **gonflé d'un facteur ~42**. La bonne pratique : compter les ventes uniques (`COUNT(DISTINCT v.id_vente)` → 50 008), et ne joindre que des relations $N \rightarrow 1$ (jamais $N \rightarrow N$ sur la même entité sans raison).

Définition. L'**explosion de jointure** — join fan-out — est le gonflement du nombre de lignes quand une jointure accorde **plusieurs lignes d'un côté à une ligne de l'autre** (relation $N \rightarrow N$, ou jointure sur une colonne non unique). Le contrôle : comparer `COUNT(*)` **avant** et **après** la jointure — s'il explose, la jointure double (piège P4).

5.8 Le piège P5 — le `DISTINCT` qui masque un mauvais `JOIN`

Le piège P5 : un `COUNT(DISTINCT ...)` peut donner le **bon chiffre par hasard**, alors que la jointure est fautive. Le `DISTINCT` « répare » en dédoublonnant, sans que personne ne sache que la jointure était mauvaise.

```
-- P5 : le DISTINCT "répare" le doublon de la jointure – chiffre juste, jointure fausse
SELECT COUNT(DISTINCT c.id_client) AS nb_clients_bobo
FROM vente v
JOIN client c ON c.id_client = v.id_client
WHERE c.ville = 'Bobo-Dioulasso';
```

Ici le chiffre (441 clients de Bobo) est juste **malgré** le fait que la jointure a déjà multiplié les lignes (chaque client apparaît une fois par vente). Le `DISTINCT` masque le problème. La bonne pratique : **vérifier le nombre de lignes avant l'agrégation** (`SELECT COUNT(*)` sur la jointure seule), pas après le `DISTINCT`.

Attention. Un `COUNT(DISTINCT col)` peut cacher une jointure qui double : le chiffre sort juste, mais la requête est fausse (et coûteuse). La règle P5 : **toujours contrôler `COUNT(*)` sur la jointure nue, avant tout `DISTINCT` ou agrégat**. Si le `COUNT(*)` explose, c'est la jointure qu'il faut corriger, pas le `DISTINCT` qu'il faut ajouter.

5.9 ON et alias — la lecture d'une jointure

La lecture d'une jointure : `FROM vente v JOIN client c ON c.id_client = v.id_client` = « pour chaque vente `v`, cherche le client `c` de même `id_client`, et mets les deux côte à côte ». Les alias (`v`, `c`) abrègent et évitent les homonymes (deux tables avec une colonne `nom` — ici `client.nom` et `magasin.nom` : c'est l'alias qui distingue).

6. Exemple concret — la requête qui répond à une question métier

Le directeur commercial demande : « Pour chaque vente, le nom du client, la ville du magasin et le mode de paiement — et le CA total par ville. »

Étape 1 — Écrire la requête (jointure en étoile, puis regroupement).

```
SELECT m.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
GROUP BY m.ville
ORDER BY ca DESC;
```

Étape 2 — Prédire la sortie. 3 villes (Ouagadougou, Bobo-Dioulasso, Koudougou) ; le `COUNT(*)` de la jointure étoilée = 50 008 (pas d'explosion, toutes les relations sont `N → 1`) ; Ouagadougou en tête (le plus de clients).

Étape 3 — Exécuter. Ouagadougou 21 290 ventes / 3 391 679 290 FCFA ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672.

Étape 4 — Comparer. 3 villes, total 50 008 ventes, total 7 908 259 731 FCFA. La jointure étoilée n'a rien doublé (contrôle P4 passé).

Conseil professionnel. Avant tout `GROUP BY` après une jointure, **vérifiez que la jointure ne double pas** : `SELECT COUNT(*)` sur la jointure nue doit redonner le nombre de lignes de la table centrale (ici 50 008). Si c'est le cas, le `GROUP BY` est fiable ; sinon, corrigez la jointure d'abord. C'est le réflexe anti-P4/P5.

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Ventes enrichies du nom du client

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 3;
```

→ 50 008 lignes disponibles (toutes les ventes ont un client).

7.2 Question 2 — Magasins sans objectif (LEFT JOIN sur table vide)

```
SELECT m.nom, o.ca_objectif_ttc
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc IS NULL;
```

→ 5 magasins (tous, l'objectif étant NULL partout) : c'est la requête standard « les X sans Y ».

7.3 Question 3 — La grille complète magasin × mode (CROSS JOIN)

```
SELECT COUNT(*) AS nb_combinaisons
FROM magasin m
CROSS JOIN mode_paiement mp;
```

→ 25 (5 × 5).

7.4 Question 4 — Le rapport détaillé en étoile

```
SELECT v.id_vente, c.nom AS client, m.nom AS magasin,
       mp.libelle AS mode, v.montant_ttc
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paiement mp ON mp.id_mode = v.id_mode
ORDER BY v.id_vente
LIMIT 3;
```

→ 50 008 lignes, chacune enrichie de 3 libellés.

7.5 Question 5 — Le piège P4 (jointure qui double)

```
SELECT COUNT(*) AS nb_explose, COUNT(DISTINCT v.id_vente) AS nb_uniques
FROM vente v
JOIN vente v2 ON v2.id_client = v.id_client;
```

→ 2 131 064 lignes explosees, 50 008 uniques. Le COUNT(*) est gonflé d'un facteur ~42 : c'est l'explosion de jointure.

7.6 Question 6 — Le CA par ville (jointure étoilée + GROUP BY)

```
SELECT m.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 / 3 391 679 290 ; Bobo-Dioulasso 18 364 / 2 895 549 769 ; Koudougou 10 354 / 1 621 030 672 FCFA.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	WHERE sur le côté droit d'un LEFT JOIN	les NULL disparaissent, le LEFT devient INNER	la condition dans l'ON.
2	Jointure sur une colonne non unique (P4)	COUNT(*) explose (2 131 064 au lieu de 50 008)	joindre sur une clé unique, ou COUNT(DISTINCT).
3	DISTINCT ajouté « pour réparer » (P5)	chiffre juste, jointure fausse, requête lente	corriger la jointure, contrôler COUNT(*) avant.
4	JOIN sans ON (oubli)	produit cartésien par accident	ajouter l'ON, ou CROSS JOIN si voulu.
5	Colonnes homonymes sans alias	erreur « column is ambiguous »	alias sur chaque table (v., c.).
6	INNER JOIN là où il faut LEFT JOIN	les lignes orphelines du gauche disparaissent	choisir le type selon la question.

Attention. Les erreurs 2 et 3 sont les pièges P4 et P5 du module : une jointure qui **double** (P4) et un DISTINCT qui **masque** un mauvais JOIN (P5). Les deux se contrôlent par le même geste : COUNT(*) sur la jointure **nue**, avant tout agrégat.

Attention. Un JOIN sans ON (ou un ON toujours vrai, comme ON 1=1) produit le **produit cartésien** : |gauche| × |droit| lignes. Sur 50 008 ventes et 1 200 clients, ce serait un peu plus de 60 milliards de lignes — à ne jamais laisser passer en production.

9. Bonnes pratiques professionnelles

- **Joindre sur des clés uniques** (PK/FK) ; jamais sur une colonne libre (nom, libellé).
- **COUNT(*) avant/après** chaque jointure : le contrôle P4 est systématique.
- **Le type de jointure suit la question** : « les X qui ont Y » → INNER ; « tous les X, avec Y si existe » → LEFT.
- **Condition du côté droit d'un LEFT dans l'ON**, jamais dans le WHERE.
- **Alias sur chaque table** : v., c., m. — obligatoire dès deux colonnes homonymes.
- **Étoile, pas grappe** : relier la table centrale aux références, pas les références entre elles.

10. Exercice guidé — le rapport détaillé par ville (15 min, /10)

Énoncé. Produire, par ville (3 lignes, triées par CA décroissant) : le nombre de ventes, le CA arrondi, le nombre de clients distincts. Utiliser une jointure en étoile.

Grille de correction (/10).

Critère	Points
Jointures vente → magasin (et client pour le distinct) correctes	3
COUNT(*), ROUND(SUM(...)), COUNT(DISTINCT c.id_client)	3
GROUP BY m.ville + ORDER BY ca DESC	2
3 lignes, Ouagadougou en tête (3 391 679 290)	2

Corrigé.

```
SELECT m.ville,
       COUNT(*) AS nb_ventes,
       ROUND(SUM(v.montant_ttc)) AS ca,
       COUNT(DISTINCT c.id_client) AS nb_clients
FROM vente v
JOIN client c ON c.id_client = v.id_client
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

→ Ouagadougou 21 290 / 3 391 679 290 / 512 ; Bobo-Dioulasso 18 364 / 2 895 549 769 / 441 ; Koudougou 10 354 / 1 621 030 672 / 247.

11. Exercices autonomes

Exercice 1 — Ventes + nom du client

Les 10 premières ventes avec le nom du client (jointure `INNER`).

Exercice 2 — Magasins sans objectif

Les magasins dont l'objectif est `NULL` (`LEFT JOIN + WHERE ... IS NULL`).

Exercice 3 — La grille magasin × mode

Le nombre de combinaisons `CROSS JOIN` (magasin × mode_paiement).

Exercice 4 — Le rapport étoilé

Les 5 premières ventes avec client, magasin, mode (3 libellés).

Exercice 5 — Mesurer l'explosion (P4)

`COUNT(*)` et `COUNT(DISTINCT v.id_vente)` de la jointure `vente × vente` via `id_client`. Donner le facteur de gonflement.

Exercice 6 — Le CA par ville

Une jointure `vente → magasin`, `GROUP BY ville`, CA arrondi, trié décroissant.

Exercice 7 — `LEFT` vs `INNER` sur table vide

Combien de lignes renvoient `INNER JOIN` et `LEFT JOIN` depuis `magasin` vers `objectif_magasin` ? Expliquer.

Exercice 8 — Diagnostic d'erreur

Pourquoi cette requête est-elle une erreur, et comment la corriger ?

```
SELECT v.id_vente, c.nom, m.nom
FROM vente v
JOIN client c ON c.id_client = v.id_client
JOIN magasin m ON m.id_magasin = v.id_magasin;
```

12. Correction détaillée

Exercice 1.

```
SELECT v.id_vente, c.nom, v.montant_ttc
FROM vente v
JOIN client c ON c.id_client = v.id_client
ORDER BY v.id_vente
LIMIT 10;
```

→ 10 lignes (les 10 premières ventes, enrichies du nom du client).

Exercice 2. Cf. §7.2 → 5 magasins (tous, l'objectif étant `NULL` partout).

Exercice 3. Cf. §7.3 → **25** combinaisons.

Exercice 4. Cf. §7.4 → 5 lignes, chacune avec 3 libellés.

Exercice 5. `COUNT(*)` = **2 131 064**, `COUNT(DISTINCT v.id_vente)` = **50 008**. Facteur de gonflement $\approx 2\,131\,064 / 50\,008 \approx 42,6$: chaque vente est comptée une fois par vente du même client. C'est l'explosion P4.

Exercice 6. Cf. §7.6 → Ouagadougou 3 391 679 290 ; Bobo-Dioulasso 2 895 549 769 ; Koudougou 1 621 030 672 FCFA.

Exercice 7. `INNER JOIN` → **0 ligne** (aucune paire accordée, `objectif_magasin` est vide) ; `LEFT JOIN` → **5 lignes** (les 5 magasins, objectif en `NULL`). Le choix dépend de la question : « les magasins qui ont un objectif » (`INNER`, 0 ici) vs « tous les magasins avec leur objectif si existe » (`LEFT`, 5).

Exercice 8. Piège de relecture : la requête est en fait **valide** (les deux `nom` sont préfixés `c.` et `m.`, donc pas d'ambiguïté) — c'est la version « non préfixée » qui serait une erreur. Si on écrivait `SELECT nom` sans alias, DuckDB renverrait « column "nom" is ambiguous » (la colonne existe dans `client` et dans `magasin`). La règle : **toujours préfixer les colonnes homonymes par l'alias** dès qu'une jointure met deux tables ayant la même colonne.

13. Mini-projet — « L'audit de jointures du rapport détaillé » (45 min)

Produire l'audit de jointures du rapport détaillé : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes attendues
J1 : le rapport étoilé (client, magasin, mode) — <code>COUNT(*)</code>	50 008
J2 : magasins sans objectif (<code>LEFT JOIN + IS NULL</code>)	5
J3 : grille magasin × mode (<code>CROSS JOIN</code>)	25
J4 : CA par ville (étoile + <code>GROUP BY</code>)	3

Contrôles de cohérence. (i) J1 : `COUNT(*)` = 50 008 (pas d'explosion, P4 passé). (ii) J4 : la somme des `ca` = 7 908 259 731 FCFA. (iii) J3 : $25 = 5 \times 5$.

Corrigé.

```
-- J1
SELECT COUNT(*) AS nb_lignes
FROM vente v
JOIN client c      ON c.id_client = v.id_client
JOIN produit p     ON p.id_produit = v.id_produit
JOIN magasin m     ON m.id_magasin = v.id_magasin
JOIN mode_paieement mp ON mp.id_mode = v.id_mode;

-- J2
SELECT m.nom
FROM magasin m
LEFT JOIN objectif_magasin o ON o.id_magasin = m.id_magasin
WHERE o.ca_objectif_ttc IS NULL;

-- J3
SELECT COUNT(*) AS nb_combinaisons
FROM magasin m
CROSS JOIN mode_paieement mp;

-- J4
SELECT m.ville, COUNT(*) AS nb, ROUND(SUM(v.montant_ttc)) AS ca
FROM vente v
JOIN magasin m ON m.id_magasin = v.id_magasin
GROUP BY m.ville
ORDER BY ca DESC;
```

Dans les faits. La syntaxe `JOIN ... ON` est **identique** sur DuckDB, PostgreSQL et SQLite pour `INNER/LEFT/CROSS`. Le `FULL OUTER JOIN` est supporté par DuckDB et PostgreSQL ; **SQLite ne le supporte pas** (il faut le simuler par `LEFT JOIN + UNION + LEFT JOIN`). C'est la principale différence de portabilité du chapitre.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 types de jointure (`INNER`, `LEFT`, `FULL`, `CROSS`), la **jointure en étoile** (la table centrale reliée aux références), et les deux contrôles anti-pièges (P4 : `COUNT(*)` avant/après ; P5 : ne jamais « réparer » un `JOIN` avec un `DISTINCT`). Le choix du type suit la question ; la condition du côté droit d'un `LEFT` va dans l'`ON`.

Outil	Usage	Syntaxe	Piège
<code>INNER JOIN</code>	les paires accordées	<code>v JOIN c ON c.id=v.id</code>	filtre les orphelines du gauche
<code>LEFT JOIN</code>	tout le gauche, <code>NULL</code> à droite	<code>m LEFT JOIN o ON ...</code>	<code>WHERE</code> sur le droit = <code>INNER</code>
<code>FULL OUTER JOIN</code>	les deux côtés	<code>m FULL JOIN o ON ...</code>	absent de SQLite
<code>CROSS JOIN</code>	produit cartésien	<code>m CROSS JOIN mp</code>	explosion si par accident
Étoile	table centrale + références	<code>vente JOIN c JOIN m JOIN mp</code>	joindre les références entre elles
Contrôle P4	explosion	<code>COUNT(*)</code> avant/après	2 131 064 au lieu de 50 008
Contrôle P5	<code>DISTINCT</code> qui masque	<code>COUNT(*)</code> sur la jointure nue	chiffre juste, jointure fausse

15. Résumé du chapitre

- **`INNER JOIN`** = les paires qui s'accordent des deux côtés (filtre les orphelines du gauche). **`LEFT JOIN`** = tout le gauche, `NULL` à droite. **`FULL OUTER JOIN`** = les deux côtés. **`CROSS JOIN`** = le produit cartésien (voulu en grille, bug par oubli d'`ON`).

- **La jointure en étoile** relie la table centrale (`vente`) aux références (client, produit, magasin, mode) en relations $N \rightarrow 1$: elle enrichit sans doubler (50 008 lignes conservées).
- **Le type de jointure suit la question** : « les X qui ont Y » → `INNER` ; « tous les X, avec Y si existe » → `LEFT`.
- **P4** : une jointure sur une colonne non unique **double** les lignes (2 131 064 au lieu de 50 008) — contrôle par `COUNT(*)` avant/après.
- **P5** : un `DISTINCT` peut **masquer** un mauvais `JOIN` (chiffre juste, requête fausse) — contrôler la jointure nue avant tout `DISTINCT`.
- **ON vs WHERE** : la condition du côté droit d'un `LEFT JOIN` va dans l'`ON`, jamais dans le `WHERE`.

16. À retenir

- **Le type suit la question** : `INNER` = « qui a », `LEFT` = « tous, avec si existe ».
- **Étoile, pas grappe** : relier la centrale aux références, jamais les références entre elles.
- `COUNT(*)` avant/après chaque jointure (P4) ; **jamais de `DISTINCT` pour réparer un `JOIN` (P5)**.
- **Condition du côté droit d'un `LEFT` dans l'`ON`**, pas dans le `WHERE`.
- **Alias + préfixe** sur chaque table : obligatoire dès deux colonnes homonymes.

À retenir. Une jointure est un **accord de clés** : `ON v.id_client = c.id_client`. Le type (`INNER/LEFT/FULL/CROSS`) décide **qui reste** quand l'accord échoue. Le bon réflexe n'est pas la syntaxe (facile) mais le **contrôle du nombre de lignes** (difficile) : `COUNT(*)` avant et après, et la question « est-ce que ma jointure devrait doubler ? ».

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Différence entre `INNER JOIN` et `LEFT JOIN` ?
2. Pourquoi la condition du côté droit d'un `LEFT JOIN` doit-elle aller dans l'`ON` ?
3. Que produit un `CROSS JOIN` entre 5 magasins et 5 modes de paiement ?
4. C'est quoi l'explosion de jointure (P4), et comment la détecter ?
5. Pourquoi un `DISTINCT` peut-il masquer un mauvais `JOIN` (P5) ?

Réponses :

1. `INNER JOIN` ne garde que les paires qui s'accordent **des deux côtés** (les orphelines du gauche disparaissent) ; `LEFT JOIN` garde **toutes** les lignes du gauche, et complète de `NULL` à droite celles sans correspondance.
2. Parce que dans le `WHERE`, les lignes dont la colonne du droit est `NULL` sont **éliminées** (`NULL > 0` est `NULL`), ce qui transforme le `LEFT JOIN` en `INNER`. Dans l'`ON`, la condition ne filtre pas le côté gauche : les lignes sans correspondance restent.
3. Le **produit cartésien** : $5 \times 5 = 25$ lignes (chaque magasin avec chaque mode), sans condition.
4. C'est le gonflement du nombre de lignes quand la jointure accorde plusieurs lignes d'un côté à une ligne de l'autre (relation $N \rightarrow N$). Détection : `COUNT(*)` **avant** (50 008) et **après** la jointure (2 131 064) — si ça explose, la jointure double.
5. Parce que le `DISTINCT` **dédoublonne** le résultat : le chiffre final peut être juste **même si** la jointure a doublé les lignes. On ne voit pas le bug (chiffre correct), mais la requête est fausse et lente. D'où le contrôle de la jointure nue avant l'agrégat.

Module M07.C08 — Sous-requêtes, CTE, vues, EXISTS vs IN

Outils comparés : DuckDB 1.5.5 (CLI + Python), SQLite natif Python, PostgreSQL cité.
Durée indicative : 4 h. **Niveau :** N2 → N3. **Prérequis :** M07.C01 (SELECT, FROM) ; M07.C05 (GROUP BY) ; M07.C07 (jointures).

L'idée du chapitre. C07 a relié les tables par des jointures. C08 montre comment **structurer** une requête quand la question est composée : d'abord calculer quelque chose, puis filtrer sur le résultat. C'est le passage de la requête « plate » à la requête « en couches » : la **sous-requête** (une requête dans une autre), la **CTE** (WITH — une sous-requête nommée), la **vue** (une sous-requête enregistrée), et la famille **EXISTS / IN** (exister ou non dans un ensemble) qui inclut le pattern canonique de la **double négation** (piège P6 : « les clients qui ont acheté **tous** les produits X »). Le fil rouge est la base `commercial.duckdb` : on y identifie le client n° 1 par CA (Client_0073, 11 561 026 FCFA), on lit la vue `v_ca_mensuel_magasin` (120 lignes), et on mesure que **119 clients** ont acheté les 2 produits les plus chers (double négation).

Base de travail — `commercial.duckdb` (empreinte `0b9c12397d9c8af2...`). Le chapitre mobilise : 50 008 ventes, 1 200 clients (tous ont au moins une vente — les « clients sans vente » sont donc 0), 380 produits (55 dans la catégorie Alimentaire-Boissons, id 1), et la vue `v_ca_mensuel_magasin` (120 lignes : 5 magasins × 24 mois).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **écrire une sous-requête scalaire** (une valeur) dans le `SELECT` ou le `WHERE` ;
- **écrire une sous-requête dans le `FROM`** (table dérivée) et l'agréger ;
- **nommer une sous-requête** avec `WITH` (CTE) pour lisibilité et réutilisation ;
- **créer une vue** et comprendre sa différence avec une CTE (enregistrée vs éphémère) ;
- **distinguer `IN` de `EXISTS`** (et leurs négations) pour tester l'appartenance à un ensemble ;
- **écrire le pattern de double négation** (`NOT EXISTS ... NOT EXISTS`) pour les questions « **tous les** » (piège P6).

2. Pourquoi cette notion est importante

- **Les questions métier réelles sont composées** : « les ventes au-dessus du panier moyen », « le client qui a le plus dépensé », « les clients qui ont tout acheté ». Aucune ne se pose en une seule requête plate — il faut d'abord **calculer**, puis **utiliser le résultat**.
- **La CTE (`WITH`) est l'outil de lisibilité** : une requête de 40 lignes devient 4 blocs nommés. C'est la différence entre une requête lisible par un collègue et une requête qu'on n'ose pas toucher.
- **`EXISTS` vs `IN`** est l'un des points les plus demandés en entretien, et la **double négation** est le pattern canonique des questions « tous les » — celles que `IN` et `COUNT` ne savent pas poser.

3. Explication simple — la sous-requête est une question dans une question

Une sous-requête, c'est poser **une question à l'intérieur d'une autre**. « Trouve-moi les ventes **au-dessus du panier moyen** » se décompose en deux questions : (1) « quel est le panier moyen ? » (2) « quelles ventes dépassent ce chiffre ? ». En SQL, la question (1) s'écrit **dedans** la question (2) :

```
SELECT ... FROM vente
WHERE montant_ttc > ( SELECT AVG(montant_ttc) FROM vente )
                    ↑ la question dans la question
```

Le moteur exécute d'abord la question de l'intérieur (le panier moyen : 158 140 FCFA), puis la question de l'extérieur avec ce résultat. **WITH** (la CTE) fait la même chose, mais **nomme** la question intérieure : ``WITH panier AS (SELECT AVG(...) ...) SELECT ... WHERE ...`

(SELECT ... FROM panier)`. Et la **vue** enregistre cette CTE nommée dans la base, pour la réutiliser.

Définition. Une **sous-requête** — subquery — est une requête SQL **implantée à l'intérieur** d'une autre requête, dans le **SELECT**, le **WHERE** ou le **FROM**. Elle est exécutée d'abord (ou à la demande), et son résultat alimente la requête extérieure. On distingue la sous-requête **scalaire** (renvoie 1 valeur) et la sous-requête **de liste** (renvoie une colonne de valeurs, pour un **IN**).

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
sous-requête scalaire	une sous-requête qui renvoie 1 valeur (1 ligne, 1 colonne).	l'utiliser quand elle renvoie plusieurs lignes (erreur).
sous-requête de liste	renvoie une colonne de valeurs, pour un IN / NOT IN .	NOT IN avec des NULL dans la liste (rien ne passe).
CTE (WITH)	une sous-requête nommée , déclarée en tête de requête.	croire qu'une CTE est optimisée comme une vue (c'est du sucre de lisibilité).
vue	une requête enregistrée dans la base, interrogée comme une table.	croire qu'elle fige les données (elle se recalcule à chaque lecture).
table dérivée	une sous-requête dans le FROM (avec un alias obligatoire).	oublier l'alias (erreur de syntaxe).
EXISTS	teste s'il existe au moins une ligne dans une sous-requête corrélée.	le confondre avec IN (EXISTS regarde l'existence, IN l'appartenance).
corrélé	une sous-requête qui référence une colonne de la requête extérieure.	le confondre avec une sous-requête indépendante (exécutée une fois).
double négation	NOT EXISTS (... NOT EXISTS (...)) : le pattern des questions « tous les ».	essayer de le faire avec IN ou COUNT (faux ou illisible).

5. Cours approfondi

5.1 La sous-requête scalaire dans le **SELECT**

La sous-requête scalaire renvoie **une valeur** qu'on place dans le **SELECT** :

```
SELECT v.id_vente, v.montant_ttc,
       (SELECT AVG(montant_ttc) FROM vente) AS panier_moyen_global
FROM vente v
ORDER BY v.id_vente
LIMIT 3;
```

→ Chaque ligne affiche le montant de la vente **et** le panier moyen global (158 140 FCFA, recalculé). C'est le motif « contexte » : afficher une valeur de référence à côté de chaque ligne.

5.2 La sous-requête scalaire dans le `WHERE`

« Les ventes au-dessus du panier moyen » :

```
SELECT COUNT(*) AS nb_ventes
FROM vente
WHERE montant_ttc > (SELECT AVG(montant_ttc) FROM vente);
```

Le moteur calcule d'abord `AVG(montant_ttc)` = 158 140 FCFA, puis filtre les ventes qui le dépassent : **20 289** ventes. La comparaison est faite contre la valeur calculée, pas contre une constante — c'est la force de la sous-requête scalaire dans le `WHERE`.

Définition. Une **sous-requête scalaire** — scalar subquery — est une sous-requête qui doit renvoyer **exactement une valeur** (1 ligne, 1 colonne). Elle s'utilise dans le `SELECT` (valeur de référence par ligne) ou le `WHERE` (filtre contre une valeur calculée). Si elle renvoie plus d'une ligne, DuckDB/PostgreSQL renvoient une erreur ; c'est un garde-fou, pas un bug.

5.3 La sous-requête dans le `FROM` — la table dérivée

Quand la question intérieure renvoie **plusieurs lignes**, on la met dans le `FROM` (avec un **alias obligatoire**) :

```
SELECT seg, COUNT(*) AS nb
FROM (
  SELECT CASE
    WHEN c.type_client = 'particulier' THEN 'Particulier'
    WHEN c.type_client = 'entreprise' THEN 'Entreprise'
    ELSE 'Comptoir'
  END AS seg
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
) t
GROUP BY seg
ORDER BY nb DESC;
```

La sous-requête (alias `t`) produit 50 008 lignes avec une colonne `seg` ; l'extérieur les regroupe. C'est l'alternative à la CTE — même chose, sans nom.

5.4 La CTE (`WITH`) — nommer la question intérieure

Le `WITH` **nomme** la sous-requête du `FROM` (ou d'une étape intermédiaire) :

```
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 3;
```

→ Client_0073 en tête avec **11 561 026 FCFA** (le client n° 1 par CA, id 73). Le `WITH` ne change pas le résultat par rapport à la table dérivée — il change la **lisibilité** : chaque étape a un nom, on peut la relire séparément.

Définition. Une **CTE** — Common Table Expression — est une sous-requête **nommée**, déclarée en tête de requête par `WITH nom AS (SELECT ...)`, puis référencée comme une table dans le reste de la requête. Elle est **éphémère** (n'existe que durant l'exécution de la requête) et sert surtout à la **lisibilité** : découper une requête complexe en étapes nommées. Elle se distingue d'une **vue** (enregistrée dans la base, réutilisable).

5.5 La vue — une CTE enregistrée

La base contient déjà la vue `v_ca_mensuel_magasin` (120 lignes : 5 magasins × 24 mois). Une vue est une requête **enregistrée** dans le schéma, interrogée comme une table :

```
SELECT id_magasin, annee, mois, nb_ventes, ROUND(ca_ttc) AS ca_ttc
FROM v_ca_mensuel_magasin
ORDER BY ca_ttc DESC
LIMIT 3;
```

→ Le plus gros mois : magasin 3, 2025-08, 444 ventes, 78 965 529 FCFA. La vue **ne fige pas** les données : elle se **recalcule** à chaque lecture (si vente change, la vue change). C'est le motif de C05/C06 « l'étiquette réutilisable » : on calcule une fois (la vue), on relit partout.

Définition. Une **vue** — view — est une requête SQL **enregistrée** dans le schéma de la base, interrogeable comme une table (`SELECT ... FROM vue`). Contrairement à une CTE (éphémère, liée à une requête), une vue **persiste** et est **réutilisable** par n'importe quelle requête. Elle ne stocke pas de données : elle **se recalcule** à chaque lecture. C'est l'outil d'encapsulation des étiquettes et des agrégats de base de travail.

5.6 IN — appartenir à un ensemble

`WHERE x IN (sous-requête)` teste si `x` appartient à la liste renvoyée :

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
JOIN produit p ON p.id_produit = v.id_produit
WHERE p.id_categorie IN (SELECT id_categorie FROM produit WHERE actif = FALSE);
```

Ou, plus simplement, « les clients qui ont acheté au moins un produit de la catégorie 1 » :

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);
```

→ **1 196 clients** (sur 1 200) ont acheté au moins un des 55 produits de la catégorie Alimentaire-Boissons. Le `IN` est l'opérateur « **au moins un** ».

À retenir. Le sens de `IN` est « **au moins un** » : `WHERE x IN (...) = « il existe au moins une valeur de la liste égale à x »`. C'est pourquoi `IN` **ne pose jamais** une question « tous les » — pour celle-ci, il faut la double négation (§5.9). Confondre « au moins un » et « tous » est l'erreur la plus fréquente des requêtes d'appartenance.

5.7 NOT IN et le piège des NULL

`NOT IN` est le « **aucun** » : « les clients qui n'ont acheté **aucun** produit de la catégorie 1 » :

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE c.id_client NOT IN (SELECT id_client FROM vente v
                          JOIN produit p ON p.id_produit = v.id_produit
                          WHERE p.id_categorie = 1);
```

→ **4 clients** (1 200 – 1 196). Attention au piège : si la sous-requête du `NOT IN` renvoie **un seul NULL**, le `NOT IN` ne renvoie **aucune ligne** (car `x NOT IN (a, NULL)` est `NULL` si `x ≠ a` — ni vrai ni faux). C'est le défaut classique du `NOT IN`.

5.8 EXISTS / NOT EXISTS — exister ou non

`EXISTS` teste s'il **existe** au moins une ligne dans une sous-requête **corrélée** (qui référence la ligne extérieure) :

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **1 200** (tous les clients ont au moins une vente). Le `NOT EXISTS` est le « **aucun** » :

```
SELECT COUNT(*) AS nb_clients_sans_vente
FROM client c
WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **0** (aucun client sans vente dans le socle). Le `EXISTS/NOT EXISTS` est souvent plus lisible (et plus sûr face aux `NULL`) que `IN/NOT IN` pour les tests d'existence.

Définition. `EXISTS` teste s'il **existe** au moins une ligne dans sa sous-requête (généralement **corrélée** : elle référence une colonne de la requête extérieure). `NOT EXISTS` est sa négation. Contrairement à `IN/NOT IN`, ils sont **insensibles aux NULL** de la liste et court-circuitent dès qu'une ligne existe — c'est l'outil de référence pour les tests d'existence et la double négation.

5.9 Le pattern de double négation — les « tous les » (P6)

La question « les clients qui ont acheté **tous** les produits les plus chers » (ou « tous les produits d'une catégorie ») ne se pose **ni** avec `IN` (qui dit « au moins un ») **ni** avec `COUNT` (fragile). Le pattern canonique est la **double négation** : « il n'existe **aucun** produit X que le client n'a **pas** acheté » :

```

SELECT c.id_client, c.nom
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Logique : on cherche les clients **c** tels qu'il **n'existe aucun** produit du top 2 que **c n'a pas** acheté. C'est-à-dire les clients qui ont acheté les **deux**. → **119 clients**.

Définition. Le **pattern de double négation** — double negation — est l'écriture `NOT EXISTS (SELECT ... WHERE NOT EXISTS (SELECT ...))` qui exprime « **tous les** » : on nie l'existence d'un contre-exemple. « Le client a acheté **tous** les produits X » équivaut à « il n'existe **aucun** produit X que le client n'a **pas** acheté ». C'est le pattern canonique (piège P6) des questions universelles, là où **IN** (« au moins un ») et **COUNT** (fragile) échouent.

Attention. Ne pas poser une question « **tous les** » avec **IN** : `WHERE id_client IN (...)` renvoie ceux qui ont acheté **au moins un**, pas **tous**. Et ne pas le faire avec un **COUNT** comparé au total (fragile, et faux dès qu'un produit est acheté en double). La double négation est l'écriture sûre.

6. Exemple concret — la requête qui répond à une question métier

Le directeur demande : « Quel est le client qui a le plus dépensé, et combien de clients ont acheté les 2 produits les plus chers ? »

Étape 1 — Écrire les deux requêtes (une CTE pour le top client, une double négation pour les 2 plus chers).

```

-- (a) Le client n°1 par CA
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;

-- (b) Les clients qui ont acheté les 2 produits les plus chers
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Étape 2 — Prédire. (a) 1 ligne, le client n° 1 par CA (id 73). (b) Un nombre < 1 200 (ceux qui ont les 2 produits du top).

Étape 3 — Exécuter. (a) Client_0073, 11 561 026 FCFA. (b) **119** clients.

Étape 4 — Comparer. Le top client est cohérent avec l'ATTENDU (id 73). Les 119 clients du (b) sont plausibles (les 2 produits les plus chers sont vendus, mais pas par tous).

Conseil professionnel. Quand une question contient « **tous les** », « **chaque** », « **aucun** », passez directement à la **double négation** (NOT EXISTS ... NOT EXISTS). Ne perdez pas de temps à essayer **IN** ou **COUNT** : ils donnent le mauvais sens (« au moins un ») ou un résultat fragile. Le « tous les » = « aucun contre-exemple ».

7. Démonstration pas à pas — 6 requêtes sur le fil rouge

7.1 Question 1 — Panier moyen comme référence (scalaire dans SELECT)

```
SELECT id_vente, montant_ttc,
       (SELECT ROUND(AVG(montant_ttc)) FROM vente) AS panier_moyen
FROM vente
ORDER BY id_vente
LIMIT 3;
```

→ 3 lignes, chacune avec le panier moyen global (158 140 FCFA) en référence.

7.2 Question 2 — Le client n° 1 par CA (CTE)

```
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT id_client, nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;
```

→ id 73, Client_0073, 11 561 026 FCFA.

7.3 Question 3 — La vue mensuelle (120 lignes)

```
SELECT COUNT(*) AS nb_lignes,
       ROUND(MAX(ca_ttc)) AS plus_gros_mois
FROM v_ca_mensuel_magasin;
```

→ 120 lignes, plus gros mois 78 965 529 FCFA (magasin 3, 2025-08).

7.4 Question 4 — Les clients ayant acheté la catégorie 1 (IN)

```
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);
```

→ 1 196 clients.

7.5 Question 5 — Les clients sans vente (NOT EXISTS)

```
SELECT COUNT(*) AS nb_clients_sans_vente
FROM client c
WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client);
```

→ **0** (tous les clients ont au moins une vente).

7.6 Question 6 — Les 2 produits les plus chers (double négation)

```
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);
```

→ **119** clients.

8. Erreurs fréquentes

#	Erreur	Symptôme	Correction
1	Sous-requête scalaire qui renvoie plusieurs lignes	erreur « more than one row returned »	LIMIT 1, ou transformer en sous-requête de liste.
2	Table dérivée sans alias	erreur de syntaxe	toujours un alias () t).
3	NOT IN avec des NULL dans la liste	0 ligne (au lieu du résultat attendu)	NOT EXISTS, ou filtrer les NULL de la liste.
4	Poser « tous les » avec IN	renvoie « au moins un » (trop de clients)	double négation NOT EXISTS ... NOT EXISTS.
5	Croire qu'une vue fige les données	la vue change quand la base change	une vue se recalcule à chaque lecture.
6	CTE complexe sans nom	requête illisible, impossible à maintenir	découper en CTE nommées (une par étape).

Attention. Le piège du NOT IN avec NULL est silencieux : si la sous-requête renvoie un NULL, x NOT IN (... , NULL) est NULL pour tout x qui n'est pas dans la partie non nulle — **aucune ligne ne passe**. C'est le bug le plus classique des « exclusions ». Privilégier NOT EXISTS (insensible aux NULL).

Attention. Une CTE n'est **pas une optimisation** : c'est du sucre de lisibilité. Sur certaines bases, une CTE peut même être **matérialisée** (calculée une fois, stockée) alors qu'une table dérivée serait inline. Ne pas écrire une CTE « pour aller plus vite » : on écrit une CTE **pour être lisible**.

9. Bonnes pratiques professionnelles

- Une CTE **par étape** de la logique : WITH brut AS (...), agrégé AS (...), résultat AS (...) puis SELECT ... FROM résultat.
- La **sous-requête scalaire** pour une valeur de référence (panier moyen, seuil) ; la **sous-requête de liste** pour un IN.

- **NOT EXISTS** plutôt que **NOT IN** dès que la liste peut contenir des **NULL**.
- « **Tous les** » = **double négation** : **NOT EXISTS (... NOT EXISTS (...))**.
- **Vue pour les étiquettes réutilisées** (CA mensuel, segmentation) ; CTE pour les étapes éphémères d'une seule requête.
- **Nommer les CTE** par leur rôle métier (`ca_par_client`, `top_produits`), pas `t1`, `t2`.

10. Exercice guidé — le top client et les acheteurs du top (15 min, /10)

Énoncé. (a) Renvoyer le client n° 1 par CA (id, nom, CA) avec une CTE. (b) Renvoyer le nombre de clients qui ont acheté les 2 produits les plus chers (double négation).

Grille de correction (/10).

Critère	Points
(a) CTE <code>ca_par_client</code> avec <code>GROUP BY + ORDER BY ca DESC LIMIT 1</code>	3
(a) Résultat : id 73, Client_0073, 11 561 026 FCFA	2
(b) Double négation <code>NOT EXISTS ... NOT EXISTS</code> correcte	4
(b) Résultat : 119 clients	1

Corrigé. Cf. §6, étape 1. (a) → 73, Client_0073, 11 561 026. (b) → 119.

11. Exercices autonomes

Exercice 1 — Le panier moyen en référence

Afficher les 5 premières ventes avec le panier moyen global (sous-requête scalaire dans le SELECT).

Exercice 2 — Le client n° 1 par CA

Avec une CTE, renvoyer l'id, le nom et le CA du client le plus dépensier.

Exercice 3 — La vue mensuelle

Nombre de lignes de `v_ca_mensuel_magasin` et le plus gros mois (en CA).

Exercice 4 — Les acheteurs de la catégorie 1

Le nombre de clients distincts ayant acheté au moins un produit de la catégorie 1 (**IN**).

Exercice 5 — Les clients sans vente

Le nombre de clients n'ayant **aucune** vente (**NOT EXISTS**).

Exercice 6 — Les acheteurs du top 2

Le nombre de clients ayant acheté les 2 produits les plus chers (double négation).

Exercice 7 — **NOT IN** VS **NOT EXISTS**

Réécrire l'exercice 5 avec **NOT IN** et expliquer en quoi **NOT EXISTS** est plus sûr.

Exercice 8 — Un COUNT à 0 : bug ou bon résultat ?

Cette requête renvoie **0**. Est-ce un bug ? Vérifiez par une méthode indépendante et expliquez pourquoi le 0 est (ou non) un résultat cohérent.

```
SELECT COUNT(*) FROM client c
WHERE c.id_client NOT IN (
  SELECT v.id_client FROM vente v
  WHERE v.montant_ttc > (SELECT AVG(montant_ttc) FROM vente)
);
```

12. Correction détaillée

Exercice 1. Cf. §7.1 → 5 lignes, chacune avec 158 140 FCFA en référence.

Exercice 2. Cf. §7.2 → id 73, Client_0073, 11 561 026 FCFA.

Exercice 3. Cf. §7.3 → 120 lignes, plus gros mois 78 965 529 FCFA.

Exercice 4. Cf. §7.4 → **1 196** clients.

Exercice 5. Cf. §7.5 → **0** client.

Exercice 6. Cf. §7.6 → **119** clients.

Exercice 7.

```
SELECT COUNT(*) FROM client c
WHERE c.id_client NOT IN (SELECT id_client FROM vente);
```

Sur ce socle, les deux renvoient 0 (tous les clients ont une vente). Mais `NOT IN` est fragile : si une seule `id_client` de la sous-requête était `NULL`, le `NOT IN` renverrait 0 ligne **même s'il existe des clients sans vente**. `NOT EXISTS` est insensible aux `NULL` et toujours correct.

Exercice 8. Ce 0 est un **bon résultat**, pas un bug. La requête compte les clients n'ayant **aucune** vente au-dessus de la moyenne globale (158 140 FCFA). Chaque client ayant en moyenne 42 ventes (50 008 / 1 200) et des montants répartis de 763 à 594 363 FCFA, il est quasi-certain que chacun a au moins une vente au-dessus de la moyenne : le `COUNT` renvoie donc légitimement 0. La vérification par méthode indépendante : la même question en `NOT EXISTS` renvoie aussi 0. Leçon : un `COUNT` à 0 n'est un bug que s'il contredit les données — on le contrôle par une requête indépendante (ici `NOT EXISTS`), et on se méfie du `NOT IN` seulement quand la liste sous-jacente peut contenir des `NULL` (pas le cas ici : `id_client` est non nul).

13. Mini-projet — « L'audit des requêtes composées » (40 min)

Produire l'audit des requêtes composées : 4 requêtes, chacune vérifiée à la main.

Requête	Lignes / valeur attendue
S1 : le client n° 1 par CA (CTE)	73, 11 561 026
S2 : le plus gros mois de la vue mensuelle	78 965 529
S3 : les acheteurs de la catégorie 1 (IN)	1 196
S4 : les acheteurs du top 2 (double négation)	119

Contrôles de cohérence. (i) $S3 \leq 1\,200$ (nombre de clients). (ii) $S4 \leq S3$ (les acheteurs du top 2 sont un sous-ensemble). (iii) $S2 \leq 7\,908\,259\,731 / 120$ (un mois ne dépasse pas le moyenne mensuelle du CA total).

Corrigé.

```

-- S1
WITH ca_par_client AS (
  SELECT c.id_client, c.nom, ROUND(SUM(v.montant_ttc)) AS ca
  FROM vente v
  JOIN client c ON c.id_client = v.id_client
  GROUP BY c.id_client, c.nom
)
SELECT id_client, nom, ca
FROM ca_par_client
ORDER BY ca DESC
LIMIT 1;

-- S2
SELECT ROUND(MAX(ca_ttc)) AS plus_gros_mois
FROM v_ca_mensuel_magasin;

-- S3
SELECT COUNT(DISTINCT v.id_client) AS nb_clients
FROM vente v
WHERE v.id_produit IN (SELECT id_produit FROM produit WHERE id_categorie = 1);

-- S4
SELECT COUNT(*) AS nb_clients
FROM client c
WHERE NOT EXISTS (
  SELECT 1
  FROM (SELECT id_produit FROM vente ORDER BY montant_ttc DESC LIMIT 2) top
  WHERE NOT EXISTS (
    SELECT 1 FROM vente v
    WHERE v.id_client = c.id_client AND v.id_produit = top.id_produit
  )
);

```

Dans les faits. WITH (CTE), EXISTS/NOT EXISTS, IN et les vues sont **standard SQL** : identiques sur DuckDB, PostgreSQL et SQLite. Seule nuance : la **matérialisation des CTE** varie (PostgreSQL peut choisir de matérialiser ou d'inline ; DuckDB gère les CTE récursives ; SQLite aussi depuis la version 3.8.3). Pour la portabilité, éviter les CTE **récursives** dans du code qui doit tourner sur SQLite ancien.

14. Boîte à outils du chapitre

Boîte à outils. Ce chapitre installe 4 outils de structuration : la **sous-requête** (scalaire et de liste), la **CTE** (WITH, la sous-requête nommée), la **vue** (la CTE enregistrée), et la famille **EXISTS/IN** avec le pattern de **double négation** pour les « tous les ». La règle : une CTE par étape, **NOT EXISTS** plutôt que **NOT IN**, et la double négation pour les questions universelles.

Outil	Usage	Syntaxe	Piège
Sous-requête scalaire	1 valeur de référence	(SELECT AVG(...) FROM vente)	plusieurs lignes = erreur
Sous-requête de liste	un IN	WHERE x IN (SELECT ...)	NOT IN + NULL = 0 ligne
Table dérivée	sous-requête FROM	du FROM (SELECT ...) t	alias obligatoire
CTE (WITH)	étape nommée	WITH nom AS (SELECT ...)	sucré de lisibilité, pas d'optimisation
Vue	CTE enregistrée	CREATE VIEW ... AS SELECT ...	se recalcule, ne fige pas

Outil	Usage	Syntaxe	Piège
EXISTS NOT EXISTS	/ test d'existence	WHERE EXISTS (SELECT 1 ...)	corrélée, court-circuit
Double négation	les « tous les »	NOT EXISTS (... NOT EXISTS (...))	P6 : ne pas le faire avec IN

15. Résumé du chapitre

- **Sous-requête scalaire** (1 valeur, dans le `SELECT` ou le `WHERE`) ; **sous-requête de liste** (une colonne, pour un `IN`).
- **CTE (WITH)** = sous-requête **nommée**, éphémère, outil de **lisibilité** (une par étape). **Vue** = CTE **enregistrée**, réutilisable, **se recalcule** (ne fige pas).
- **IN** = « au moins un » ; **NOT IN** = « aucun », mais **fragile face aux NULL** (renvoie 0 ligne si la liste contient un `NULL`).
- **EXISTS/NOT EXISTS** = test d'existence, **insensible aux NULL**, souvent plus sûr que `IN/NOT IN`.
- **Double négation** (`NOT EXISTS ... NOT EXISTS`) = le pattern canonique des « **tous les** » (P6) : « aucun contre-exemple ». `IN` et `COUNT` ne le font pas.
- **Nuance de portabilité** : CTE, vues, `EXISTS` standard ; la matérialisation des CTE et les CTE récursives varient entre SGBD.

16. À retenir

- Une CTE **par étape**, nommée par son rôle métier : lisibilité avant tout.
- **Sous-requête scalaire** = 1 valeur de référence ; **de liste** = un `IN`.
- **NOT EXISTS plutôt que NOT IN** (sécurité face aux `NULL`).
- « **Tous les** » = **double négation** : `NOT EXISTS (... NOT EXISTS (...))` — jamais `IN`.
- Une **vue se recalcule** : c'est une encapsulation, pas un snapshot.

À retenir. La sous-requête est une **question dans une question** ; la CTE la **nomme** ; la vue l'**enregistre** ; `EXISTS/IN` testent l'**appartenance**. Et dès que la question dit « **tous les** », on écrit la **double négation** : nier l'existence d'un contre-exemple est la seule écriture sûre.

17. Évaluation formative (auto-correction, 8 min)

Sans document, en 8 minutes, répondre aux 5 questions. Auto-correction en bas.

1. Différence entre une sous-requête scalaire et une sous-requête de liste ?
2. Quelle est la différence entre une CTE et une vue ?
3. Pourquoi `NOT IN` peut-il renvoyer 0 ligne, et comment éviter ce bug ?
4. Comment écrit-on « les clients qui ont acheté **tous** les produits X » ?
5. Une vue fige-t-elle les données ?

Réponses :

1. La sous-requête **scalaire** renvoie **1 valeur** (1 ligne, 1 colonne) — elle s'utilise dans le `SELECT` ou le `WHERE`. La sous-requête **de liste** renvoie une **colonne de valeurs** — elle s'utilise dans un `IN / NOT IN`.
2. Une **CTE** est une sous-requête **nommée, éphémère** (n'existe que durant la requête), déclarée par `WITH`. Une **vue** est une requête **enregistrée** dans le schéma, **réutilisable** par n'im-

porte quelle requête, interrogée comme une table. Les deux se recalculent ; la vue persiste, la CTE non.

3. Parce que si la sous-requête du `NOT IN` renvoie **un NULL**, alors `x NOT IN (... , NULL)` est `NULL` (ni vrai ni faux) pour tout `x` non présent — **aucune ligne ne passe**. Éviter avec `NOT EXISTS` (insensible aux `NULL`) ou en filtrant les `NULL` de la liste.
4. Par la **double négation** : `WHERE NOT EXISTS (SELECT 1 FROM produit X WHERE NOT EXISTS (SELECT 1 FROM vente v WHERE v.id_client = c.id_client AND v.id_produit = p.id_produit))` — « il n'existe **aucun** produit X que le client **n'a pas** acheté ». `IN` (« au moins un ») et `COUNT` (fragile) ne le font pas.
5. **Non**. Une vue **se recalcule** à chaque lecture : si la table sous-jacente change, la vue change. C'est une encapsulation de requête, pas un snapshot de données.